

ระบบรวมศูนย์ข้อความและการจัดการคำสั่งซื้อสำหรับธุรกิจออนไลน์

Centralized Chat and Order Management System for Online Business

โดย

วชิรวิทย์ ชื่นจิตร

รายงานนี้เป็นส่วนหนึ่งของวิชาสหกิจศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต

สาขาวิชาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตขอนแก่น

พ.ศ. 2569

ลิขสิทธิ์ของคณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน



ระบบรวมศูนย์ข้อความและการจัดการคำสั่งซื้อสำหรับธุรกิจออนไลน์

Centralized Chat and Order Management System for Online Business

โดย

วชิรวิทย์ ชื่นจิตร

ปฏิบัติงาน ณ

บริษัทไฮเฟนพลัสจำกัด

71 ซอยรังสิต-นครนายก 59 ตำบลประชาธิปัตย์ อำเภอธัญบุรี จังหวัดปทุมธานี 12130

หัวข้อปริญญานิพนธ์	ระบบรวมศูนย์ข้อความและการจัดการคำสั่งซื้อสำหรับธุรกิจออนไลน์
จัดทำโดย	วชิรวิทย์ ชื่นจิตร
ปีที่ปริญญานิพนธ์สำเร็จ	พ.ศ. 2569
สาขาวิชา	วิศวกรรมคอมพิวเตอร์
ประธานที่ปรึกษา	ผู้ช่วยศาสตราจารย์ ดร.อติราช สุขสวัสดิ์

บทคัดย่อ

โครงงานนี้มีวัตถุประสงค์เพื่อพัฒนาระบบเว็บของ Chatshop ซึ่งเป็นระบบรวมศูนย์ข้อความและการจัดการคำสั่งซื้อสำหรับธุรกิจออนไลน์ ร้านค้าที่รับลูกค้าจากหลายช่องทางต้องสลับแอปพลิเคชันไปมา ทำให้ข้อความตกหล่น ตอบลูกค้าล่าช้า และข้อมูลลูกค้ากับคำสั่งซื้อกระจัดกระจาย ระบบนี้จึงรวมแชทจาก LINE Official Account, Facebook Messenger, Instagram และ WebChat ไว้ในกล่องข้อความเดียว

ระบบพัฒนาด้วย Laravel จัดเก็บข้อมูลด้วย MySQL และ MongoDB รับข้อความผ่าน Webhook ประมวลผลงานเบื้องหลังด้วยคิวงาน และแสดงข้อความใหม่แบบเรียลไทม์ผ่าน Laravel Reverb ระบบรองรับร้านค้าหลายร้านโดยข้อมูลแยกจากกัน มีระบบทีมที่มอบหมายบทสนทนาและกำหนดสิทธิ์ตามบทบาท แชทบอทที่ตอบคำถามจากฐานความรู้ของร้านค้าด้วยเทคนิค RAG และส่งต่อให้เจ้าหน้าที่เมื่อตอบไม่ได้ ระบบจัดการสินค้า ข้อมูลลูกค้า และคำสั่งซื้อที่เปิดได้จากหน้าแชท การแจ้งเตือนแชทที่ค้างตอบ และรายงานสรุปผลการให้บริการ

ผลการทดสอบแบบ Black Box ตามกรณีทดสอบ 18 กรณี พบว่าระบบทำงานได้ถูกต้องทุกกรณี และผลการประเมินความพึงพอใจของผู้ใช้งานจำนวน 16 คน มีค่าเฉลี่ยรวมเท่ากับ 4.25 อยู่ในระดับมาก

คำสำคัญ : ระบบรวมศูนย์ข้อความ, ร้านค้าออนไลน์, แชทบอท, RAG, Laravel

Project Title	Centralized Chat and Order Management System for Online Business
Proposed by	Wachirawit Chuenchit
Year	2026
Department	Computer Engineering
Advisor	Assistant Professor, Atirarj Suksawad, D.Eng

Abstract

This project aims to develop the web system of Chatshop, a centralized chat and order management system for online businesses. Shops that serve customers through several channels must switch between applications, which causes missed messages, slow replies, and scattered customer and order data. The system therefore gathers chats from LINE Official Account, Facebook Messenger, Instagram, and WebChat into a single inbox.

The system is developed with Laravel, stores data in MySQL and MongoDB, receives messages through Webhooks, processes background jobs with queues, and pushes new messages to the browser in real time through Laravel Reverb. It supports multiple shops with isolated data and provides team management with conversation assignment and role-based permissions, a chatbot that answers questions from the shop knowledge base using RAG and hands over to staff when it cannot answer, product, customer, and order management with orders created directly from the chat screen, alerts for unanswered chats, and service summary reports.

Black-box testing with 18 test cases showed that the system worked correctly in every case. The satisfaction survey of 16 users gave an overall mean score of 4.25, which is at the high level.

Keywords: Centralized Inbox, Online Shop, Chatbot, RAG, Laravel

กิตติกรรมประกาศ

การที่ข้าพเจ้าได้มาปฏิบัติงานสหกิจศึกษา ณ บริษัทไฮเฟนพลัสจำกัด ตั้งแต่วันที่ 22 มิถุนายน พ.ศ. 2569 ถึงวันที่ 9 ตุลาคม พ.ศ. 2569 ข้าพเจ้าได้รับความรู้ทั้งในด้านการปฏิบัติงานจริง การแก้ปัญหาที่หน้างาน และประสบการณ์ในการทำงานร่วมกับผู้อื่น ซึ่งเป็นประสบการณ์ที่สำคัญและเป็นประโยชน์อย่างยิ่ง สำหรับโครงงานสหกิจศึกษาฉบับนี้ จะสำเร็จลุล่วงไปไม่ได้หากปราศจากความช่วยเหลือและสนับสนุนจากหลายฝ่าย ข้าพเจ้าขอแสดงความ ขอบคุณอย่างสูงต่อบุคคลต่อไปนี้

1. **คุณปริญญา จันทร์ครุธ** **ตำแหน่ง กรรมการบริษัท**
2. **คุณนพรัตร์ คำโกแก้ว** **ตำแหน่ง นักพัฒนาซอฟแวร์**
3. **ผู้ช่วยศาสตราจารย์ ดร.อติราช สุขสวัสดิ์** **ตำแหน่ง อาจารย์นิเทศสหกิจศึกษา**
4. **อาจารย์ชัย ดร. ชัย วันคัน** **ตำแหน่ง อาจารย์ประสานงานสหกิจ**

นอกจากนี้ ข้าพเจ้ายังขอขอบคุณเจ้าหน้าที่สำนักเทคโนโลยีดิจิตอลมหาวิทยาลัยขอนแก่นและทีมช่างทุกท่าน ที่ได้ให้คำแนะนำและความช่วยเหลือตลอดระยะเวลาการฝึกงาน รวมถึงการให้ข้อมูลและความรู้ที่เป็นประโยชน์ต่อการพัฒนาโครงงานนี้

สุดท้ายนี้ ข้าพเจ้าขอขอบพระคุณอาจารย์ทุกท่านจากมหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน ที่ได้มอบโอกาสและสนับสนุนการศึกษาของข้าพเจ้า ข้าพเจ้าขอขอบพระคุณทุกท่านไว้ ณ โอกาสนี้

วชิรวิทย์ ชื่นจิตร

สารบัญ

หน้า

บทคัดย่อ ง

กิตติกรรมประกาศ ฉ

สารบัญตาราง ฎ

สารบัญรูป ฏ

บทที่ 1 บทนำ 1

1.1 สถานประกอบการ 1

1.2 ความเป็นมาและความสำคัญของปัญหา 2

1.3 วัตถุประสงค์ของโครงงาน 2

1.4 ขอบเขตของโครงงาน 3

1.5 ประโยชน์ที่คาดว่าจะได้รับ 3

1.6 เทคโนโลยีและเครื่องมือที่ใช้ในการพัฒนา 4

1.7 นิยามศัพท์เฉพาะ 4

บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง 5

2.1 LINE Messaging API 5

2.2 Meta Graph API 6

2.3 Webhook 7

2.4 Laravel 8

2.5 Blade และ React 10

2.6 Queue และ Task Scheduling 10

2.7 WebSocket และ Laravel Reverb 12

2.8 MySQL 13

2.9 MongoDB 13

2.10 สถาปัตยกรรม Multi-Tenant 14

2.11 การควบคุมสิทธิ์ตามบทบาท (RBAC) 15

สารบัญ (ต่อ)

หน้า

2.12 OAuth 2.0 15
2.13 REST API 16
2.14 แชทบอท 17
2.15 แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM) 18
2.16 ผู้ให้บริการแบบจำลองภาษา (AI Provider) 19
2.17 RAG (Retrieval-Augmented Generation) 20
2.18 Embedding และความคล้ายโคไซน์ 21
2.19 งานวิจัยที่เกี่ยวข้อง 21
บทที่ 3 วิธีดำเนินงาน 24
3.1 กำหนดปัญหาที่เกิดขึ้นกับระบบงาน 24
3.2 การวิเคราะห์ระบบ 24
3.3 การพัฒนาระบบ 27
3.4 โครงสร้างระบบเว็บ Chatshop 33
3.5 การออกแบบ API 34
3.6 การพัฒนาเว็บแอปพลิเคชัน 35
3.7 การเชื่อมต่อบริการภายนอก 44
3.8 การเชื่อมต่อผู้ให้บริการแบบจำลองภาษา 53
บทที่ 4 ผลการดำเนินงานและการทดสอบ 55
4.1 รูปแบบการทดสอบ 56
4.2 ผลการทดสอบการสมัครและเข้าสู่ระบบด้วยอีเมล 56
4.3 ผลการทดสอบการเข้าสู่ระบบด้วย Google และ Facebook 57
4.4 ผลการทดสอบการเชื่อมต่อ LINE Official Account 58
4.5 ผลการทดสอบการเชื่อมต่อ Facebook Page และ Instagram 59
4.6 ผลการทดสอบการรับข้อความแบบทันที 59

สารบัญ (ต่อ)

หน้า

4.7 ผลการทดสอบการปฏิเสธ Webhook ที่ลายเซ็นไม่ถูกต้อง 60
4.8 ผลการทดสอบการตอบข้อความจากกล่องแชท 61
4.9 ผลการทดสอบการจัดการแท็ก 62
4.10 ผลการทดสอบการเชิญสมาชิกเข้าทีม 63
4.11 ผลการทดสอบสิทธิ์ตามบทบาท 63
4.12 ผลการทดสอบการตอบอัตโนมัติด้วยคีย์เวิร์ด 64
4.13 ผลการทดสอบการตอบของแชทบอทจากฐานความรู้ 65
4.14 ผลการทดสอบการส่งต่อบทสนทนาให้เจ้าหน้าที่ 66
4.15 ผลการทดสอบการเปิดคำสั่งซื้อจากแชท 66
4.16 ผลการทดสอบการส่งบรอดแคสต์แบบตั้งเวลา 67
4.17 ผลการทดสอบการนำเข้าประวัติแชท 68
4.18 ผลการทดสอบการแจ้งเตือนแชทที่ค้างตอบ 69
4.19 ผลการทดสอบ WebChat Widget 69
4.20 แบบประเมินความพึงพอใจในการใช้งานระบบเว็บ 70
บทที่ 5 สรุปผลและข้อเสนอแนะ 72
5.1 สรุปผลการดำเนินงาน 72
5.2 ประโยชน์ที่สถานประกอบการได้รับ 73
5.3 ข้อจำกัดของระบบ 75
5.4 ปัญหาและอุปสรรค 76
5.5 ข้อเสนอแนะ 76
บรรณานุกรม 78
ภาคผนวก ก คู่มือการใช้งานระบบเว็บ Chatshop 85
ก.1 ข้อกำหนดและผู้ใช้งานระบบ 85
ก.2 การสมัครใช้งานและเข้าสู่ระบบ 86

สารบัญ (ต่อ)

หน้า

ก.3 การเชื่อมต่อช่องทางแชท 87

ก.4 การตอบแชทในกล่องแชท 91

ก.5 ทีมงานและสิทธิ์การใช้งาน 94

ก.6 การตั้งค่าการตอบกลับ 95

ก.7 การตั้งค่าแชทบอท 96

ก.8 สินค้า ลูกค้า และคำสั่งซื้อ 99

ก.9 การส่งบรอดแคสต์ 101

ก.10 การนำเข้าประวัติแชท 102

ก.11 การแจ้งเตือนและรายงานสรุปผล 102

ประวัติผู้เขียน 104

สารบัญตาราง

ตาราง	หน้า
ตารางที่ 3.1 โหมดการทำงานของแชทบอท	31
ตารางที่ 4.1 หัวข้อที่ใช้บันทึกกรณีทดสอบ	56
ตารางที่ 4.2 การทดสอบการสมัครและเข้าสู่ระบบด้วยอีเมล	56
ตารางที่ 4.3 การทดสอบการเข้าสู่ระบบด้วย Google และ Facebook	57
ตารางที่ 4.4 การทดสอบการเชื่อมต่อ LINE Official Account	58
ตารางที่ 4.5 การทดสอบการเชื่อมต่อ Facebook Page และ Instagram	59
ตารางที่ 4.6 การทดสอบการรับข้อความแบบทันที	59
ตารางที่ 4.7 การทดสอบการปฏิเสธ Webhook ที่ลายเซ็นไม่ถูกต้อง	60
ตารางที่ 4.8 การทดสอบการตอบข้อความจากกล่องแชท	61
ตารางที่ 4.9 การทดสอบการจัดการแท็ก	62
ตารางที่ 4.10 การทดสอบการเชิญสมาชิกเข้าทีม	63
ตารางที่ 4.11 การทดสอบสิทธิ์ตามบทบาท	63
ตารางที่ 4.12 การทดสอบการตอบอัตโนมัติด้วยคีย์เวิร์ด	64
ตารางที่ 4.13 การทดสอบการตอบของแชทบอทจากฐานความรู้	65
ตารางที่ 4.14 การทดสอบการส่งต่อบทสนทนาให้เจ้าหน้าที่	66
ตารางที่ 4.15 การทดสอบการเปิดคำสั่งซื้อจากแชท	66
ตารางที่ 4.16 การทดสอบการส่งบรอดแคสต์แบบตั้งเวลา	67
ตารางที่ 4.17 การทดสอบการนำเข้าประวัติแชท	68
ตารางที่ 4.18 การทดสอบการแจ้งเตือนแชทที่ค้างตอบ	69
ตารางที่ 4.19 การทดสอบ WebChat Widget	69
ตารางที่ 4.20 เกณฑ์การแปลผลค่าเฉลี่ยความพึงพอใจ	70
ตารางที่ 4.21 ผลการประเมินความพึงพอใจในการใช้งานระบบเว็บ	71
ตารางที่ 5.1 ผลที่วัดได้จากการพัฒนาและการทดสอบ	73
ตารางที่ 5.2 สมมติฐานที่ใช้ประมาณการมูลค่า	74
ตารางที่ 5.3 มูลค่าเทียบเท่าค่าจ้างแอดมินช่วงนอกเวลาทำงาน	74

สารบัญรูป

รูป	หน้า
รูปที่ 1.1 ที่ตั้งบริษัท ไฮเฟน พลัส จำกัด	1
รูปที่ 2.1 ขั้นตอนการทำงานของ Webhook	7
รูปที่ 2.2 การทำงานของ Laravel ตามรูปแบบ MVC	9
รูปที่ 2.3 การทำงานของระบบคิวงาน	11
รูปที่ 2.4 การรับส่งข้อมูลแบบ HTTP เทียบกับ WebSocket	12
รูปที่ 2.5 ขั้นตอนการขอสิทธิ์แบบ Authorization Code ของ OAuth 2.0	16
รูปที่ 2.6 ขั้นตอนการทำงานของ RAG	20
รูปที่ 3.1 Use Case Diagram ของระบบเว็บ Chatshop	27
รูปที่ 3.2 ภาพรวมการจัดเก็บข้อมูลของระบบเว็บ Chatshop	28
รูปที่ 3.3 ขั้นตอนการรับข้อความจาก Webhook	29
รูปที่ 3.4 ขั้นตอนการสร้างคำตอบของแชทบอท	30
รูปที่ 3.5 สถาปัตยกรรมของระบบเว็บ Chatshop	33
รูปที่ 3.6 หน้าเข้าสู่ระบบ	36
รูปที่ 3.7 หน้ากล่องแชท	36
รูปที่ 3.8 หน้าออเดอร์	37
รูปที่ 3.9 หน้าภาพรวม	38
รูปที่ 3.10 หน้าทีมงาน	38
รูปที่ 3.11 แท็บช่องทางในหน้าตั้งค่า	39
รูปที่ 3.12 แท็บอัตโนมัติในหน้าตั้งค่า	40
รูปที่ 3.13 แท็บ AI Chatbot ในหน้าตั้งค่า	41
รูปที่ 3.14 หน้าสินค้า	42
รูปที่ 3.15 หน้าลูกค้า	42
รูปที่ 3.16 หน้ารายการบรอดแคสต์	43
รูปที่ 3.17 หน้าสร้างบรอดแคสต์	43
รูปที่ 3.18 การสร้าง OAuth client ID ใน Google Cloud Console	44
รูปที่ 3.19 การตั้งค่า Facebook Login ของแอปประเภท Consumer	45
รูปที่ 3.20 การสร้าง Configuration ของ Facebook Login for Business	47
รูปที่ 3.21 การตั้งค่า Webhooks ของ Messenger	47
รูปที่ 3.22 หน้าเลือก Facebook Page ที่ต้องการเชื่อมต่อ	48

สารบัญรูป (ต่อ)

รูป	หน้า
รูปที่ 3.23 การตั้งค่า Messaging API ใน LINE Developers Console	49
รูปที่ 3.24 หน้าเชื่อมต่อ LINE Official Account ในระบบ	50
รูปที่ 3.25 หน้าต่างโค้ดฝัง WebChat Widget	51
รูปที่ 3.26 หน้า App Review ของแอปประเภท Business	52
รูปที่ 3.27 การเพิ่มบัญชีผู้ทดสอบในบทบาทของแอป	52
รูปที่ 3.28 ฟอร์มเพิ่มผู้ให้บริการแบบจำลองภาษา	54
รูปที่ ก.1 แถบเมนูหลักของระบบเว็บ	85
รูปที่ ก.2 หน้าสมัครใช้งาน	86
รูปที่ ก.3 หน้าเข้าสู่ระบบ	87
รูปที่ ก.4 หน้าต่างเลือกแพลตฟอร์มที่ต้องการเชื่อมต่อ	88
รูปที่ ก.5 หน้าเชื่อมต่อ LINE Official Account	89
รูปที่ ก.6 หน้าเลือก Facebook Page ที่ต้องการเชื่อมต่อ	90
รูปที่ ก.7 หน้าเลือก Instagram ที่ต้องการเชื่อมต่อ	90
รูปที่ ก.8 หน้าต่างโค้ดฝัง Web Chat Widget	91
รูปที่ ก.9 หน้ากล่องแชท	92
รูปที่ ก.10 ช่องพิมพ์ข้อความและการแนบไฟล์	93
รูปที่ ก.11 แผงข้อมูลลูกค้าในหน้ากล่องแชท	93
รูปที่ ก.12 หน้าต่างเชิญสมาชิกใหม่	94
รูปที่ ก.13 หน้าแก้ไขบทบาทและสิทธิ์การใช้งาน	95
รูปที่ ก.14 หน้าเพิ่มกฎตอบกลับอัตโนมัติ	96
รูปที่ ก.15 ตัวช่วยตั้งค่าบอท	97
รูปที่ ก.16 ส่วนคลังความรู้ของแชทบอท	98
รูปที่ ก.17 ฟอร์มเพิ่มผู้ให้บริการแบบจำลองภาษา	99
รูปที่ ก.18 หน้าจัดการสินค้า	99
รูปที่ ก.19 หน้าต่างสร้างคำสั่งซื้อจากหน้าแชท	100
รูปที่ ก.20 หน้ารายการคำสั่งซื้อ	101
รูปที่ ก.21 หน้าสร้างบรอดแคสต์	101
รูปที่ ก.22 หน้าต่างนำเข้าประวัติบทสนทนา	102
รูปที่ ก.23 หน้าภาพรวมประสิทธิภาพ	103

บทที่ 1
บทนำ

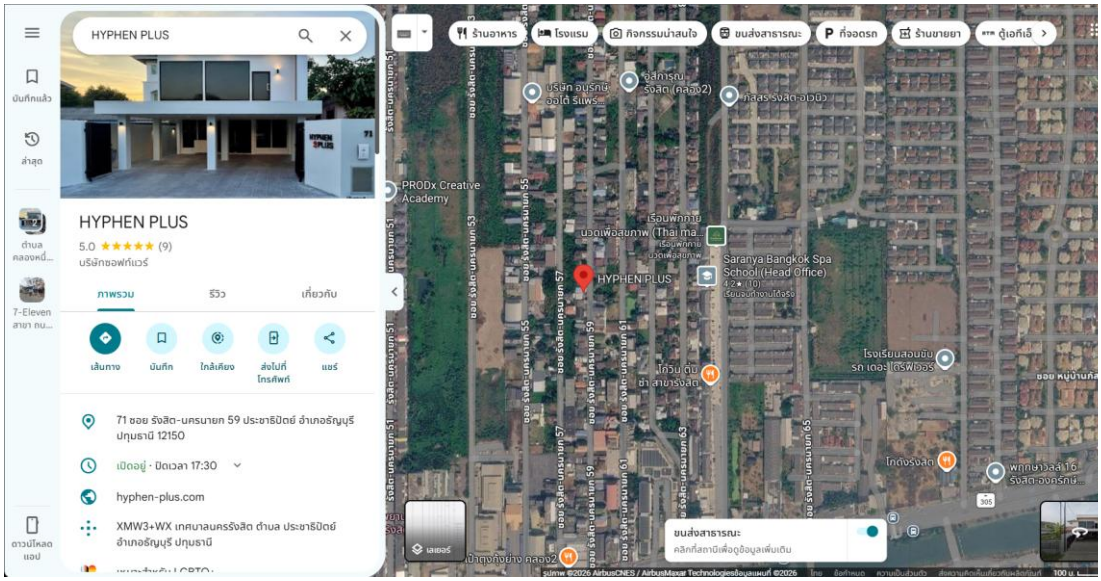
1.1 สถานประกอบการ

1.1.1 ชื่อและที่ตั้งสถานประกอบการ

หน่วยงาน: บริษัท ไฮเฟน พลัส จำกัด (Hyphen Plus Co., Ltd.)

ที่ตั้ง: 71 ซอยรังสิต-นครนายก 59 ตำบลประชาธิปัตย์ อำเภอธัญบุรี จังหวัดปทุมธานี 12130 (71 Soi Rangsit-Nakhon Nayok 59, Prachathipat, Thanyaburi, Pathum Thani 12130)

ที่ตั้งของบริษัท ไฮเฟน พลัส จำกัด แสดงดังรูปที่ 1.1



รูปที่ 1.1 ที่ตั้งบริษัท ไฮเฟน พลัส จำกัด

1.1.2 ประวัติและความเป็นมาของสถานประกอบการ

บริษัท ไฮเฟน พลัส จำกัด เป็นบริษัทพัฒนาซอฟต์แวร์ตั้งอยู่ในจังหวัดปทุมธานี ก่อตั้งเมื่อปี พ.ศ. 2563 ให้บริการพัฒนาซอฟต์แวร์ตามความต้องการของลูกค้า โดยเน้นระบบอัตโนมัติที่ธุรกิจ

ใช้งานประจำวัน เช่น ระบบขายสินค้าผ่านการถ่ายทอดสด ระบบตรวจสอบการชำระเงิน และระบบประมวลผลเอกสาร เพื่อเปลี่ยนงานที่ต้องทำด้วยมือให้ระบบทำงานแทนได้อย่างต่อเนื่อง

บริษัทเป็น Meta Business Partner และได้รับการรับรองเป็น Meta Tech Provider รวมทั้งผ่านการรับรองด้านการจัดการข้อมูลลูกค้าจาก Meta จึงพัฒนาระบบบนแพลตฟอร์มของ Meta ตามมาตรฐานที่ Meta กำหนด นอกจากงานตามความต้องการของลูกค้าแล้ว บริษัทยังพัฒนาผลิตภัณฑ์ของตนเอง ได้แก่ MegaPic, CheckSlip, Edocify และ CF Shop

งานทั้งหมดพัฒนาโดยทีมงานภายในบริษัท และบริษัทเปิดรับนักศึกษาฝึกงานเข้าร่วมปฏิบัติงานพัฒนาซอฟต์แวร์จริงร่วมกับทีมงาน เพื่อให้นักศึกษามีทักษะพร้อมทำงานในสายอาชีพ

1.2 ความเป็นมาและความสำคัญของปัญหา

การซื้อขายสินค้าผ่านแชทได้รับความนิยมอย่างมากในประเทศไทย ลูกค้าสอบถามรายละเอียดสินค้า ขอดูรูปภาพ และตกลงซื้อขายกับร้านค้าผ่าน LINE Official Account, Facebook Messenger และ Instagram ร้านค้าออนไลน์จึงต้องเปิดหลายช่องทางพร้อมกันเพื่อเข้าถึงลูกค้า มูลค่าพาณิชย์อิเล็กทรอนิกส์ของประเทศไทยปี 2566 คาดว่าสูงถึง 5.96 ล้านล้านบาท และการขายผ่านสื่อสังคมออนไลน์คิดเป็นร้อยละ 22.25 ของช่องทางการขายออนไลน์ โดย Facebook เป็นแพลตฟอร์มที่ใหญ่ที่สุด (สำนักงานพัฒนาธุรกรรมทางอิเล็กทรอนิกส์, 2567)

เมื่อแต่ละช่องทางมีแอปพลิเคชันของตัวเอง เจ้าหน้าที่ต้องสลับหน้าจอไปมา ข้อความตกหล่นและตอบลูกค้าไม่ทัน เมื่อร้านมีเจ้าหน้าที่หลายคนก็ตอบซ้ำกัน และไม่มีการแบ่งสิทธิ์การเข้าถึงข้อมูล คำถามเดิมต้องพิมพ์ตอบซ้ำทุกวัน การเปิดคำสั่งซื้อยังทำด้วยตนเองในแชท ข้อมูลลูกค้าและคำสั่งซื้อจึงกระจัดกระจาย และร้านค้าไม่รู้ว่าบทสนทนาใดค้างตอบนานเกินไป

ผู้จัดทำร่วมกับผู้ร่วมพัฒนาจึงพัฒนาระบบ Chatshop ซึ่งประกอบด้วยระบบเว็บและแอปพลิเคชันบนสมาร์ตโฟน โดยผู้จัดทำรับผิดชอบระบบเว็บทั้งหมดที่พัฒนาด้วย Laravel ได้แก่ กล่องข้อความที่รวมแชทจากทุกช่องทางไว้ที่เดียว ระบบทีมและสิทธิ์การใช้งาน แชทบอทที่ตอบคำถามจากฐานความรู้ของร้านค้า ระบบจัดการสินค้า ลูกค้า และคำสั่งซื้อ การแจ้งเตือนแชทที่ค้างตอบ และรายงานสรุปผล

1.3 วัตถุประสงค์ของโครงงาน

1.3.1 เพื่อพัฒนาระบบเว็บที่รวบรวมและตอบข้อความจาก LINE Official Account, Facebook Messenger, Instagram และ WebChat ได้ในหน้าจอเดียวแบบเรียลไทม์

1.3.2 เพื่อพัฒนาระบบรองรับร้านค้าหลายร้าน พร้อมระบบทีม การมอบหมายบทสนทนา และการกำหนดสิทธิ์การใช้งานตามบทบาท

1.3.3 เพื่อพัฒนาแชทบอทที่ตอบคำถามลูกค้าจากฐานความรู้ของร้านค้า และส่งต่อบทสนทนาให้เจ้าหน้าที่เมื่อไม่สามารถตอบได้

1.3.4 เพื่อพัฒนาระบบจัดการสินค้า ข้อมูลลูกค้า และคำสั่งซื้อที่เชื่อมกับหน้าแชท

1.3.5 เพื่อพัฒนารายงานสรุปผลการให้บริการของร้านค้า

1.4 ขอบเขตของโครงงาน

1.4.1 ระบบเว็บพัฒนาด้วย Laravel จัดเก็บข้อมูลด้วย MySQL และ MongoDB และประมวลผลงานที่ใช้เวลานานในคิวงานเบื้องหลัง

1.4.2 ระบบเชื่อมต่อ LINE Official Account, Facebook Messenger, Instagram และ WebChat ผ่าน Webhook และส่งข้อความตอบกลับผ่าน API ของแต่ละแพลตฟอร์ม

1.4.3 เจ้าหน้าที่ตอบข้อความ ส่งรูปภาพและไฟล์ ติดแท็ก และมอบหมายบทสนทนาได้จากกล่องข้อความเดียว พร้อมค้นหาบทสนทนาย้อนหลังได้

1.4.4 ร้านค้าส่งข้อความบรอดแคสต์ตามกลุ่มลูกค้าได้ และนำเข้าประวัติแชทเดิมจาก Facebook และ Instagram ย้อนหลังได้ไม่เกิน 90 วัน

1.4.5 ร้านค้าเชิญสมาชิกเข้าทีม และกำหนดบทบาทและสิทธิ์ของสมาชิกได้

1.4.6 ร้านค้าตั้งกฎตอบกลับอัตโนมัติตามคีย์เวิร์ดได้

1.4.7 ระบบมีแชทบอทที่ตอบคำถามลูกค้าจากฐานความรู้ของร้านค้า และส่งต่อบทสนทนาให้เจ้าหน้าที่ดูแลต่อเมื่อคำถามอยู่นอกเหนือข้อมูลที่แชทบอทตอบได้

1.4.8 ร้านค้าจัดการสินค้าและข้อมูลลูกค้าได้ และเปิดคำสั่งซื้อจากหน้าแชทได้

1.4.9 ระบบแจ้งเตือนแชทที่ค้างตอบ และแสดงรายงานสรุปผล

1.4.10 แอปพลิเคชันบนสมาร์ตโฟนเป็นขอบเขตของผู้ร่วมพัฒนา ไม่อยู่ในโครงงานนี้

1.5 ประโยชน์ที่คาดว่าจะได้รับ

1.5.1 เจ้าหน้าที่ตอบลูกค้าจากทุกช่องทางได้ในหน้าจอเดียว

1.5.2 ร้านค้าแบ่งงานเจ้าหน้าที่ได้ชัดเจน และควบคุมสิทธิ์ตามบทบาท

1.5.3 ลูกค้าได้รับคำตอบของคำถามที่พบบ่อยทันทีจากแชทบอท

1.5.4 ลดความผิดพลาดในการเปิดคำสั่งซื้อ และบันทึกข้อมูลลูกค้าเป็นระบบ

1.5.5 ผู้ประกอบการติดตามภาพรวมการให้บริการของร้านได้จากรายงานสรุปผล

1.6 เทคโนโลยีและเครื่องมือที่ใช้ในการพัฒนา

1.6.1 Visual Studio Code ใช้เป็นเครื่องมือหลักในการเขียนโปรแกรม

1.6.2 Laravel ใช้พัฒนาระบบเว็บส่วนหลังบ้าน

1.6.3 Blade และ React ใช้พัฒนาหน้าเว็บ

1.6.4 MySQL และ MongoDB ใช้จัดเก็บข้อมูล

1.6.5 Laravel Reverb ใช้รับส่งข้อความแบบเรียลไทม์ผ่าน WebSocket

1.6.6 Git และ GitHub ใช้ควบคุมเวอร์ชันของซอร์สโค้ด

1.7 นิยามศัพท์เฉพาะ

1.7.1 Chatshop หมายถึง ระบบรวมศูนย์ข้อความและการจัดการคำสั่งซื้อสำหรับธุรกิจออนไลน์ที่พัฒนาในโครงงานนี้ ประกอบด้วยระบบเว็บและแอปพลิเคชันบนสมาร์ตโฟน

1.7.2 Webhook หมายถึง กลไกที่แพลตฟอร์มแชทส่งข้อมูลเหตุการณ์ เช่น ข้อความใหม่จากลูกค้า มายัง URL ของระบบทันทีที่เกิดเหตุการณ์

1.7.3 Multi-Tenant หมายถึง สถาปัตยกรรมซอฟต์แวร์ที่ให้บริการร้านค้าหลายแห่งภายใต้ระบบเดียวกัน โดยข้อมูลของแต่ละร้านค้าแยกออกจากกัน

1.7.4 RBAC (Role-Based Access Control) หมายถึง การควบคุมการเข้าถึงระบบโดยกำหนดสิทธิ์ให้แก่บทบาท แล้วมอบบทบาทให้ผู้ใช้

1.7.5 RAG (Retrieval-Augmented Generation) หมายถึง เทคนิคที่ค้นคืนข้อมูลที่เกี่ยวข้องจากฐานความรู้ก่อน แล้วส่งให้แบบจำลองภาษาใช้ประกอบการสร้างคำตอบ

บทที่ 2
ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

การพัฒนาระบบเว็บของ Chatshop ซึ่งรวมแชทจากหลายช่องทาง จัดการทีม แชทบอท สินค้า ลูกค้า คำสั่งซื้อ และรายงาน จำเป็นต้องศึกษาแนวคิดและเทคโนโลยีที่เกี่ยวข้อง เพื่อนำมาออกแบบและพัฒนาระบบให้ทำงานได้ตามวัตถุประสงค์ โดยมีหัวข้อดังนี้

2.1 LINE Messaging API

2.2 Meta Graph API

2.3 Webhook

2.4 Laravel

2.5 Blade และ React

2.6 Queue และ Task Scheduling

2.7 WebSocket และ Laravel Reverb

2.8 MySQL

2.9 MongoDB

2.10 สถาปัตยกรรม Multi-Tenant

2.11 การควบคุมสิทธิ์ตามบทบาท (RBAC)

2.12 OAuth 2.0

2.13 REST API

2.14 แชทบอท

2.15 แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM)

2.16 ผู้ให้บริการแบบจำลองภาษา (AI Provider)

2.17 RAG (Retrieval-Augmented Generation)

2.18 Embedding และความคล้ายโคไซน์

2.19 งานวิจัยที่เกี่ยวข้อง

2.1 LINE Messaging API

LINE Messaging API คือบริการของ LINE สำหรับเชื่อมต่อระบบภายนอกกับบัญชี LINE Official Account ระบบรับข้อความจากผู้ใช้ผ่าน Webhook และส่งข้อความกลับผ่าน API โดยยืนยันตัวตนด้วย Channel Access Token และตรวจสอบ Webhook ด้วย Channel Secret [2]

เมื่อผู้ใช้ส่งข้อความ เพิ่มเพื่อน หรือบล็อกบัญชี แพลตฟอร์ม LINE จะส่งเหตุการณ์ในรูปแบบ JSON ไปยัง Webhook URL ที่ลงทะเบียนไว้ใน LINE Developers Console แต่ละเหตุการณ์ระบุประเภท เช่น message, follow และ unfollow พร้อมรหัสผู้ใช้ (userId) และ replyToken ระบบปลายทางนำข้อมูลเหล่านี้ไปประมวลผลและตอบกลับผู้ใช้ [3]

การส่งข้อความกลับมี 2 แบบ คือ Reply API ซึ่งใช้ replyToken ตอบกลับเหตุการณ์ภายในเวลาที่กำหนดและไม่นับรวมในโควตาข้อความ และ Push API ซึ่งส่งข้อความถึงผู้ใช้เมื่อใดก็ได้โดยอ้างอิง userId แต่นับรวมในโควตาข้อความรายเดือนตามแพ็กเกจของบัญชี นอกจากนี้ยังมี API เสริม เช่น ดึงข้อมูลโปรไฟล์ผู้ใช้ ดาวน์โหลดไฟล์สื่อ และแสดงภาพเคลื่อนไหวกำลังโหลดในหน้าแชท [2]

2.1.1 ข้อดี

2.1.1.1 เข้าถึงผู้ใช้ LINE ซึ่งเป็นแอปพลิเคชันแชทที่คนไทยใช้มากที่สุด

2.1.1.2 ส่งข้อความได้หลายรูปแบบ เช่น ข้อความ รูปภาพ และสติกเกอร์

2.1.2 ข้อเสีย

2.1.2.1 ไฟล์สื่อจากผู้ใช้ไม่มี URL สาธารณะ ต้องดาวน์โหลดผ่าน API

2.1.3 การนำไปใช้ในโครงการ ระบบรับข้อความจาก LINE ผ่าน Webhook ของแต่ละช่องทาง และส่งข้อความกลับด้วย Push API พร้อมแสดงสถานะกำลังโหลดระหว่างรอแชทบอท

2.2 Meta Graph API

Meta Graph API คือ API ของบริษัท Meta สำหรับเข้าถึงข้อมูลและบริการบน Facebook และ Instagram โครงงานนี้ใช้ส่วน Messenger Platform สำหรับรับส่งข้อความกับผู้ใช้ผ่านเพจ Facebook และบัญชี Instagram โดยใช้ Page Access Token ในการยืนยันตัวตน [4]

Graph API มองข้อมูลเป็นกราฟซึ่งประกอบด้วย 3 ส่วน ได้แก่ โหนด (Node) คือวัตถุ เช่น ผู้ใช้ เพจ และข้อความ เส้นเชื่อม (Edge) คือความสัมพันธ์ระหว่างโหนด เช่น บทสนทนาทั้งหมดของเพจ และฟิลด์ (Field) คือคุณสมบัติของโหนด เช่น ชื่อและรูปโปรไฟล์ การเรียกใช้ทำผ่าน HTTPS โดยระบุเวอร์ชันไว้ใน URL ทำให้แอปพลิเคชันไม่ได้รับผลกระทบทันทีเมื่อ Meta ปรับเปลี่ยน API ก่อนใช้งานจริง ผู้พัฒนาต้องสร้างแอปพลิเคชันบน Meta for Developers และขอสิทธิ์ที่จำเป็น เช่น pages_messaging และ instagram_manage_messages ผ่านขั้นตอน App Review เมื่อผู้ใช้ส่งข้อความหาเพจ Meta จะส่งเหตุการณ์ผ่าน Webhook มายังแอปพลิเคชัน ส่วนการส่งข้อความกลับใช้ Send API โดยระบุผู้รับด้วย PSID (Page-Scoped ID) ซึ่งเป็นรหัสผู้ใช้เฉพาะภายในเพจนั้น [4], [5]

2.2.1 ข้อดี

2.2.1.1 ใช้ API ชุดเดียวส่งข้อความได้ทั้ง Messenger และ Instagram

2.2.1.2 ดึงประวัติบทสนทนาเดิมของเพจมาเก็บในระบบได้

2.2.2 ข้อเสีย

2.2.2.1 ข้อความอัตโนมัติส่งได้ภายใน 24 ชั่วโมงหลังข้อความล่าสุดเท่านั้น

2.2.2.2 token ของเพจหมดอายุหรือถูกยกเลิกได้ ระบบต้องตรวจจับข้อผิดพลาดนี้ ปิดการเชื่อมต่อช่องทางนั้น และแจ้งให้ร้านค้าเชื่อมต่อใหม่

2.2.3 การนำไปใช้ในโครงการ ระบบใช้ Graph API เวอร์ชัน 19.0 ส่งข้อความตอบกลับ ดึงรูปโปรไฟล์ลูกค้า และนำเข้าประวัติแชทย้อนหลังได้ไม่เกิน 90 วัน

2.3 Webhook

Webhook คือกลไกที่ระบบต้นทางส่งคำขอ HTTP ไปยัง URL ที่ระบบปลายทางลงทะเบียนไว้ทันทีเมื่อเกิดเหตุการณ์ เช่น มีข้อความใหม่ ระบบปลายทางจึงรับข้อมูลได้ทันทีโดยไม่ต้องสอบถามซ้ำเป็นระยะ ผู้ให้บริการมักแนบลายเซ็นดิจิทัลที่สร้างจาก HMAC มาด้วย เพื่อให้ระบบปลายทางตรวจสอบได้ว่าข้อมูลมาจากผู้ให้บริการจริงและไม่ถูกแก้ไขระหว่างทาง [3], [5] ขั้นตอนการทำงานของ Webhook ตั้งแต่ลงทะเบียน URL จนถึงยืนยันการรับข้อมูล แสดงดังรูปที่ 2.1



รูปที่ 2.1 ขั้นตอนการทำงานของ Webhook

จากรูปที่ 2.1 ระบบปลายทางลงทะเบียน URL กับผู้ให้บริการก่อน ซึ่งมักต้องยืนยันความเป็นเจ้าของ URL เช่น Meta ส่งค่า hub.challenge มาให้ระบบตอบกลับค่าเดิม เมื่อเกิดเหตุการณ์ ผู้ให้บริการจะส่งคำขอ HTTP POST ที่มีข้อมูลรูปแบบ JSON มายัง URL นั้น และระบบปลายทางตอบรหัสสถานะ 200 ทันทีเพื่อยืนยันว่าได้รับข้อมูลแล้ว [5]

วิธีที่ใช้แทน Webhook คือ Polling ซึ่งระบบปลายทางส่งคำขอสอบถามผู้ให้บริการซ้ำตามรอบเวลา คำขอส่วนใหญ่ได้คำตอบว่าไม่มีข้อมูลใหม่ จึงสิ้นเปลืองทรัพยากรและได้ข้อมูลล่าช้าตามรอบเวลา ส่วน Webhook ส่งข้อมูลเฉพาะเมื่อมีเหตุการณ์ แต่ผู้ให้บริการอาจส่งเหตุการณ์เดิมซ้ำเมื่อไม่ได้รับการตอบรับ ระบบปลายทางจึงควรตรวจรหัสเหตุการณ์ก่อนบันทึก เพื่อไม่ให้ข้อมูลซ้ำซ้อน

2.3.1 ข้อดี

2.3.1.1 รับข้อความใหม่ได้ทันที ไม่ต้องสอบถามข้อมูลซ้ำเป็นระยะ

2.3.1.2 ลดภาระของเครื่องแม่ข่ายทั้งฝั่งผู้ให้บริการและฝั่งระบบ

2.3.2 ข้อเสีย

2.3.2.1 ระบบต้องตอบกลับให้เร็ว หากตอบช้าผู้ให้บริการจะส่งซ้ำ

2.3.3 การนำไปใช้ในโครงการ ระบบตรวจสอบลายเซ็น X-Line-Signature ของ LINE และ X-Hub-Signature-256 ของ Meta ด้วย HMAC-SHA256 จากนั้นส่งข้อมูลเข้าคิวงานทันที เพื่อให้ตอบกลับผู้ให้บริการได้รวดเร็ว แล้วจึงบันทึกข้อความและเรียกแชทบอทในเบื้องหลัง

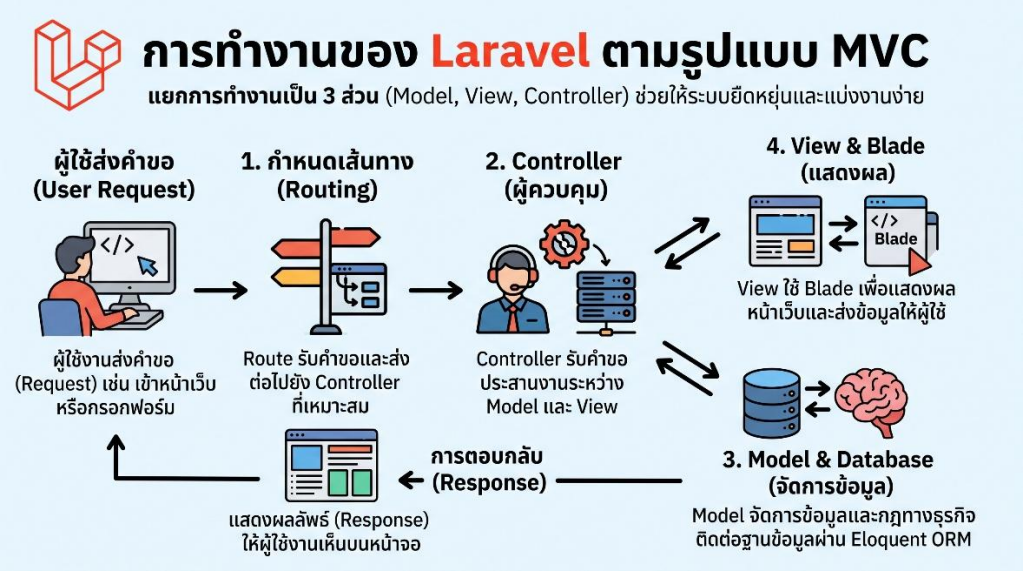
2.4 Laravel

Laravel คือเฟรมเวิร์กภาษา PHP ที่ใช้รูปแบบ MVC (Model-View-Controller) มีเครื่องมือพื้นฐานสำหรับพัฒนาระบบเว็บ ได้แก่ การกำหนดเส้นทาง Eloquent ORM สำหรับติดต่อฐานข้อมูล Blade สำหรับสร้างหน้าเว็บ และระบบคิวงานสำหรับประมวลผลเบื้องหลัง [6]

นอกจากนี้ Laravel มีเครื่องมือ Artisan สำหรับสั่งงานผ่านบรรทัดคำสั่ง เช่น สร้างไฟล์ต้นแบบและรัน Migration ซึ่งเป็นการกำหนดโครงสร้างฐานข้อมูลด้วยโค้ด ทำให้ทุกเครื่องมีโครงสร้างฐานข้อมูลตรงกัน และใช้ Composer จัดการแพ็กเกจที่ระบบต้องใช้

Eloquent ORM แทนแต่ละตารางด้วยคลาส Model ผู้พัฒนาอ่านและบันทึกข้อมูลผ่านเมธอดของคลาสแทนการเขียนคำสั่ง SQL เอง และกำหนดความสัมพันธ์ระหว่างตาราง เช่น หนึ่งต่อหลาย ด้วยเมธอด hasMany และ belongsTo ได้ ทั้งยังผูกค่าพารามิเตอร์ในคำสั่งค้นหาให้อัตโนมัติ ช่วยป้องกันการโจมตีแบบ SQL Injection นอกจากนี้ยังมี Eager Loading สำหรับดึงข้อมูลของตารางที่สัมพันธ์กันมาพร้อมกัน ลดปัญหาการส่งคำสั่งค้นหาซ้ำทีละแถวเมื่อแสดงรายการข้อมูลจำนวนมาก ทำให้หน้าเว็บแสดงผลได้เร็วขึ้น [7]

รูปแบบ MVC แบ่งระบบเป็น 3 ส่วน ได้แก่ Model ทำหน้าที่จัดการข้อมูลและกฎทางธุรกิจ View ทำหน้าที่แสดงผล และ Controller ทำหน้าที่รับคำขอและประสานงานระหว่าง Model กับ View การแยกส่วนนี้ทำให้แก้ไขส่วนหนึ่งได้โดยไม่กระทบส่วนอื่น และแบ่งงานในทีมพัฒนาได้ง่าย [6] การทำงานของ Laravel ตามรูปแบบ MVC แสดงดังรูปที่ 2.2



รูปที่ 2.2 การทำงานของ Laravel ตามรูปแบบ MVC

จากรูปที่ 2.2 เมื่อมีคำขอ HTTP เข้ามา Router ของ Laravel จะจับคู่ URL กับ Controller ที่รับผิดชอบ โดยคำขอต้องผ่าน Middleware ก่อน เช่น ตรวจว่าผู้ใช้เข้าสู่ระบบแล้วหรือมีสิทธิ์เพียงพอ จากนั้น Controller เรียก Model เพื่ออ่านหรือแก้ไขข้อมูลในฐานข้อมูล แล้วส่งข้อมูลที่ได้ให้ View นำไปสร้างเป็นหน้าเว็บ และส่งกลับไปแสดงผลที่เบราว์เซอร์ของผู้ใช้

2.4.1 ข้อดี

2.4.1.1 มีเครื่องมือพื้นฐานครบ ลดเวลาในการพัฒนา

2.4.1.2 โครงสร้างโค้ดเป็นระเบียบตามรูปแบบ MVC

2.4.1.3 มีเอกสารและชุมชนผู้ใช้งานจำนวนมาก

2.4.2 ข้อเสีย

2.4.2.1 ใช้ทรัพยากรเครื่องแม่ข่ายมากกว่าการเขียน PHP แบบไม่ใช้เฟรมเวิร์ก

2.4.2.2 ผู้พัฒนาต้องเรียนรู้ข้อกำหนดของเฟรมเวิร์กก่อนใช้งาน

2.4.3 การนำไปใช้ในโครงการ โครงงานนี้ใช้ Laravel 10 พัฒนาระบบเว็บทั้งหมด ทั้งหน้าเว็บ การรับ Webhook คิวงาน แชทบอท และ API ที่หน้าเว็บเรียกใช้

2.5 Blade และ React

Blade คือระบบ Template ของ Laravel สำหรับสร้างหน้าเว็บจากฝั่งเครื่องแม่ข่าย ส่วน React คือไลบรารี JavaScript สำหรับสร้างส่วนติดต่อผู้ใช้จากคอมโพเนนต์ เมื่อข้อมูลเปลี่ยน React จะปรับเฉพาะส่วนของหน้าจอที่เปลี่ยนแปลง จึงเหมาะกับหน้าจอที่มีข้อมูลใหม่เข้ามาตลอดเวลา [8], [9]

ไฟล์ Blade ใช้นามสกุล .blade.php เขียนผสมระหว่าง HTML กับคำสั่งของ Blade เช่น @if และ @foreach การแสดงค่าตัวแปรด้วยเครื่องหมาย {{ }} จะแปลงอักขระพิเศษให้อัตโนมัติ ช่วยป้องกันการโจมตีแบบ XSS และหน้าเว็บหลายหน้าใช้โครงหน้าเดียวกันได้ด้วย @extends และ @section Blade จะถูกแปลงเป็นโค้ด PHP และเก็บไว้ในแคช จึงไม่ทำให้ระบบทำงานช้าลง [8]

React แบ่งหน้าจอเป็นคอมโพเนนต์ ซึ่งเป็นฟังก์ชันที่รับข้อมูลผ่าน props และคืนค่าเป็น JSX แต่ละคอมโพเนนต์มีสถานะ (state) ของตนเอง เมื่อสถานะเปลี่ยน React จะสร้าง Virtual DOM ชุดใหม่ เปรียบเทียบกับชุดเดิม แล้วปรับ DOM จริงเฉพาะส่วนที่ต่างกัน ส่วน Hooks เช่น useState และ useEffect ใช้จัดการสถานะและงานที่เกิดขึ้นหลังการแสดงผล เช่น การดึงข้อมูลจาก API [9]

2.5.1 การนำไปใช้ในโครงการ หน้าเข้าสู่ระบบและหน้าตั้งค่าทั่วไปสร้างด้วย Blade ส่วนหน้ากล่องข้อความสร้างด้วย React เพื่อแสดงข้อความใหม่ได้ทันทีโดยไม่ต้องโหลดหน้าเว็บใหม่

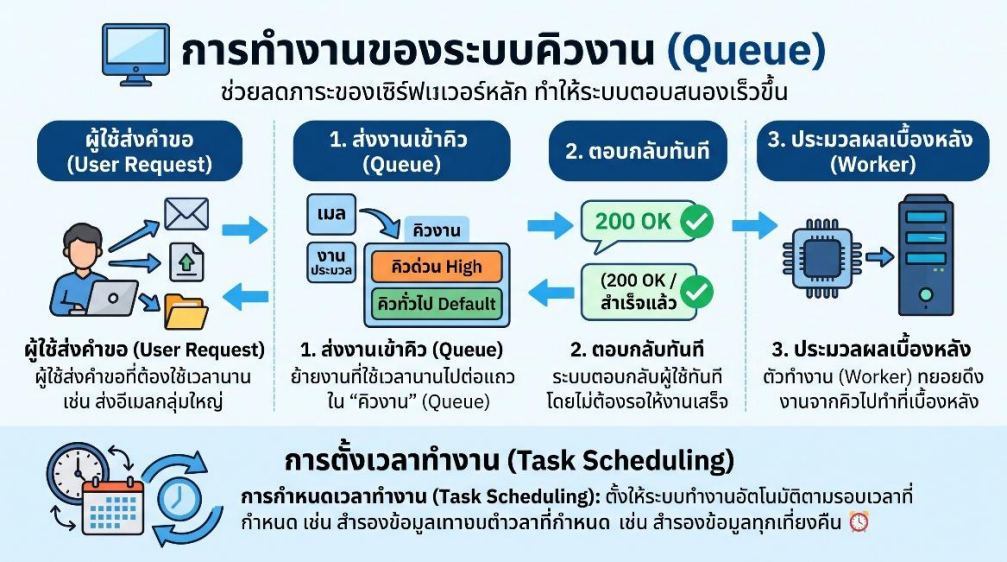
2.6 Queue และ Task Scheduling

Queue คือการย้ายงานที่ใช้เวลานานออกไปทำงานเบื้องหลัง ทำให้ระบบตอบกลับคำขอได้ทันที งานแต่ละประเภทแยกไว้คนละคิวได้เพื่อกำหนดลำดับความสำคัญ ส่วน Task Scheduling คือการกำหนดให้คำสั่งทำงานอัตโนมัติตามรอบเวลา แนวคิดนี้เรียกว่าการประมวลผลแบบไม่ประสานเวลา (Asynchronous Processing) ซึ่งผู้ใช้ไม่ต้องรอให้งานเสร็จก่อนได้รับคำตอบ [10]

Task Scheduling ของ Laravel ช่วยให้ไม่ต้องเพิ่มรายการ cron ทีละคำสั่ง ผู้พัฒนาตั้ง cron เพียงรายการเดียวให้เรียกคำสั่ง schedule:run ทุกนาที แล้วกำหนดรอบเวลาของแต่ละงานในโค้ด เช่น ทุกนาทีหรือทุกวัน และกำหนดไม่ให้งานเดียวกันทำงานซ้อนกันได้เมื่อรอบก่อนหน้ายังทำไม่เสร็จ ตัวอย่างงานที่นิยมตั้งเวลา เช่น ล้างข้อมูลชั่วคราวที่หมดอายุ และสรุปรายงานประจำวัน [11]

ระบบที่รับข้อมูลจาก Webhook ต้องตอบกลับผู้ให้บริการภายในเวลาจำกัด หากรอให้บันทึกข้อมูลและสร้างคำตอบเสร็จก่อน คำขออาจหมดเวลาและถูกส่งซ้ำ คิวงานจึงช่วยแยกการรับข้อมูลออกจากการประมวลผล งานอื่นที่เหมาะกับคิว ได้แก่ การส่งอีเมล การประมวลผลไฟล์ และการเรียก API ภายนอกที่ใช้เวลานาน ซึ่งหากทำระหว่างคำขอ ผู้ใช้ต้องรอจนกว่างานจะเสร็จ การแยก

งานผ่านคิวยังช่วยให้ระบบรับมือกับปริมาณงานที่เพิ่มขึ้นอย่างฉับพลันได้ เช่น ช่วงที่ร้านค้าจัดโปรโมชันและมีลูกค้าส่งข้อความเข้ามาพร้อมกันจำนวนมาก งานจะรอในคิวและถูกทยอยประมวลผลแทนที่ระบบจะหยุดตอบสนอง การทำงานของระบบคิวงานแสดงดังรูปที่ 2.3



รูปที่ 2.3 การทำงานของระบบคิวงาน

จากรูปที่ 2.3 ระบบคิวประกอบด้วย 3 ส่วน ได้แก่ Job คืองานหนึ่งชิ้นพร้อมข้อมูลที่ต้องใช้ Queue คือที่เก็บงานที่รอทำ ซึ่งเก็บได้ทั้งในฐานข้อมูลหรือ Redis และ Worker คือกระบวนการที่ดึงงานจากคิวมาทำตามลำดับเข้าก่อนออกก่อน (FIFO) เมื่อได้รับคำขอ ระบบสร้าง Job ส่งเข้า Queue แล้วตอบกลับผู้ใช้ทันที โดยไม่ต้องรอให้งานเสร็จ เปิด Worker หลายตัวพร้อมกันได้ หากงานล้มเหลว ระบบจะลองใหม่ตามจำนวนครั้งที่กำหนด และบันทึกงานที่ยังล้มเหลวไว้ให้ผู้พัฒนาตรวจสอบภายหลัง [10]

2.6.1 การนำไปใช้ในโครงการ

2.6.1.1 คิวความสำคัญสูง ใช้ประมวลผล Webhook ขาเข้า และการนำเข้าประวัติแชทย้อนหลังจาก Facebook และ Instagram

2.6.1.2 คิวทั่วไป ใช้สร้างคำตอบของแชทบอท สรุปบทสนทนา จัดทำดัชนีเอกสารความรู้ และส่งบรอดแคสต์เป็นชุด ชุดละ 100 บทสนทนา

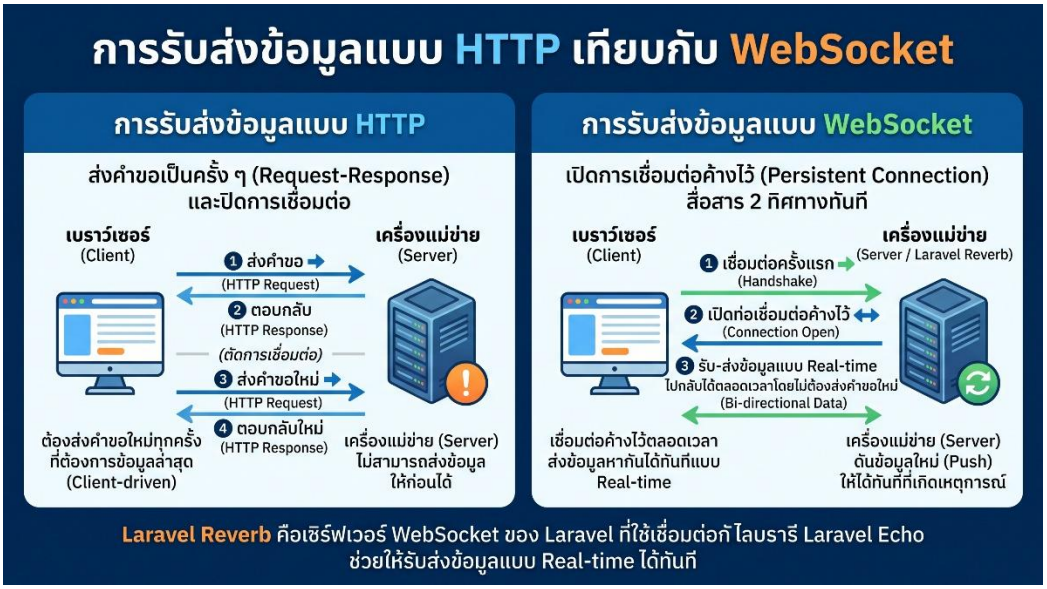
2.6.1.3 คิวสถานะ ใช้ส่งสถานะกำลังพิมพ์ระหว่างที่แชทบอทสร้างคำตอบ

2.6.1.4 ตัวจัดตารางเวลาตรวจหาแชทที่ค้างตอบนานเกินกำหนดทุก 1 นาที

2.6.1.5 ระบบเก็บคิวงานทั้ง 3 คิวไว้ใน Redis ผ่านไลบรารี predis

2.7 WebSocket และ Laravel Reverb

WebSocket คือโพรโทคอลที่เปิดการเชื่อมต่อค้างไว้ระหว่างเบราว์เซอร์กับเครื่องแม่ข่าย ทำให้เครื่องแม่ข่ายส่งข้อมูลไปยังผู้ใช้ได้ทันที ส่วน Laravel Reverb คือเซิร์ฟเวอร์ WebSocket ของ Laravel ที่ใช้โพรโทคอลเดียวกับ Pusher จึงใช้ร่วมกับไลบรารี Laravel Echo ได้ [12] ความแตกต่างระหว่างการรับส่งข้อมูลแบบ HTTP กับ WebSocket แสดงดังรูปที่ 2.4



รูปที่ 2.4 การรับส่งข้อมูลแบบ HTTP เทียบกับ WebSocket

จากรูปที่ 2.4 การเชื่อมต่อ WebSocket เริ่มจากเบราว์เซอร์ส่งคำขอ HTTP ขอเปลี่ยนโพรโทคอล (Upgrade) เมื่อเครื่องแม่ข่ายตอบรับ ทั้งสองฝั่งจะใช้การเชื่อมต่อ TCP เดิมรับส่งข้อมูลได้พร้อมกันทั้งสองทิศทาง [13] ต่างจาก HTTP ที่เครื่องแม่ข่ายตอบได้เฉพาะเมื่อมีคำขอเข้ามา เบราว์เซอร์จึงต้องส่งคำขอสอบถามซ้ำ ทำให้ข้อมูลล่าช้าและสิ้นเปลือง

Laravel ส่งข้อมูลผ่าน WebSocket ด้วยกลไก Broadcasting เมื่อเกิดเหตุการณ์ในระบบ เหตุการณ์จะถูกส่งออกไปยังช่อง (Channel) ซึ่งมี 3 แบบ ได้แก่ ช่องสาธารณะที่ทุกคนรับได้ ช่องส่วนตัวที่ต้องผ่านการตรวจสิทธิ์ก่อนรับ และช่อง Presence ที่บอกได้ว่ามีผู้ใช้คนใดอยู่ในช่องบ้าง ฝั่งเบราว์เซอร์ใช้ Laravel Echo สมัครรับช่องและรอรับเหตุการณ์ [12]

2.7.1 การนำไปใช้ในโครงการ ระบบส่งเหตุการณ์ เช่น ข้อความใหม่ การแจ้งเตือน และคำสั่งซื้อที่เปลี่ยนสถานะ ผ่านช่องส่วนตัวของแต่ละร้านค้า หน้าเว็บจึงแสดงข้อมูลล่าสุดได้ทันที

2.8 MySQL

MySQL คือระบบจัดการฐานข้อมูลเชิงสัมพันธ์แบบโอเพนซอร์ส จัดเก็บข้อมูลในรูปตารางที่เชื่อมกันด้วยคีย์ ใช้ภาษา SQL ในการจัดการข้อมูล และรองรับธุรกรรมตามคุณสมบัติ ACID [14]

ฐานข้อมูลเชิงสัมพันธ์ตั้งอยู่บนแบบจำลองที่เสนอโดย Codd [15] ข้อมูลเก็บเป็นตารางที่มีแถวและคอลัมน์ แต่ละตารางมีคีย์หลัก (Primary Key) ระบุแต่ละแถวไม่ให้ซ้ำกัน และใช้คีย์นอก (Foreign Key) อ้างอิงคีย์หลักของตารางอื่นเพื่อเชื่อมข้อมูล การออกแบบตารางตามหลัก Normalization ช่วยลดข้อมูลซ้ำซ้อน ส่วนดัชนี (Index) ช่วยให้ค้นหาข้อมูลได้รวดเร็ว ธุรกรรมตามคุณสมบัติ ACID ประกอบด้วย Atomicity คือคำสั่งทั้งชุดต้องสำเร็จทั้งหมดหรือไม่เกิดขึ้นเลย Consistency คือข้อมูลต้องถูกต้องตามกฎที่กำหนดทั้งก่อนและหลังธุรกรรม Isolation คือธุรกรรมที่ทำพร้อมกันไม่รบกวนกัน และ Durability คือข้อมูลที่บันทึกสำเร็จแล้วจะไม่สูญหายแม้ระบบขัดข้อง เช่น การบันทึกคำสั่งซื้อพร้อมรายการสินค้า หากล้มเหลวระหว่างทางจะยกเลิกทั้งชุด [16]

2.8.1 ข้อดี

2.8.1.1 รักษาความถูกต้องของข้อมูลที่สัมพันธ์กันด้วยคีย์หลักและคีย์นอก

2.8.1.2 รองรับธุรกรรม เหมาะกับข้อมูลที่ต้องการความถูกต้องสูง

2.8.2 ข้อเสีย

2.8.2.1 ต้องกำหนดโครงสร้างตารางล่วงหน้า ไม่เหมาะกับข้อมูลที่เปลี่ยนบ่อย

2.8.3 การนำไปใช้ในโครงการ โครงงานนี้ใช้ MySQL จัดเก็บข้อมูลร้านค้า สมาชิก บทบาท ช่องทางที่เชื่อมต่อ สินค้า ลูกค้า คำสั่งซื้อ บรอดแคสต์ และการตั้งค่า

2.9 MongoDB

MongoDB คือฐานข้อมูลแบบเอกสาร จัดเก็บข้อมูลเป็นเอกสารรูปแบบ BSON ซึ่งมีโครงสร้างคล้าย JSON เอกสารในคอลเลกชันเดียวกันไม่จำเป็นต้องมีฟิลด์เหมือนกัน [17] การใช้ฐานข้อมูลหลายประเภทในระบบเดียวตามลักษณะของข้อมูลเรียกว่า Polyglot Persistence [18]

MongoDB จัดเก็บข้อมูลเป็นลำดับชั้น ฐานข้อมูลหนึ่งมีหลายคอลเลกชัน (Collection) ซึ่งเทียบได้กับตาราง และคอลเลกชันหนึ่งมีหลายเอกสาร (Document) ซึ่งเทียบได้กับแถว ทุกเอกสารมีฟิลด์ _id เป็นรหัสไม่ซ้ำกัน เอกสารเก็บเอกสารย่อยและอาร์เรย์ซ้อนไว้ภายในได้ ข้อมูลที่ใช้ร่วมกันจึงอ่านได้ในครั้งเดียวโดยไม่ต้องเชื่อมตาราง และยังสร้างดัชนีบนฟิลด์ต่าง ๆ เพื่อเร่งการค้นหาได้ [17], [19]

แนวคิด Polyglot Persistence เสนอว่าระบบควรเลือกฐานข้อมูลให้เหมาะกับลักษณะของข้อมูลแต่ละส่วน ข้อมูลที่ต้องการความถูกต้องและมีความสัมพันธ์ซับซ้อนเหมาะกับฐานข้อมูลเชิง

สัมพันธ์ ส่วนข้อมูลที่มีปริมาณมากและโครงสร้างหลากหลายเหมาะกับฐานข้อมูลแบบเอกสาร ข้อแลกเปลี่ยนคือผู้พัฒนาต้องดูแลฐานข้อมูลมากกว่าหนึ่งระบบ และการเชื่อมข้อมูลข้ามฐานข้อมูลทำในระดับโปรแกรม [18]

เมื่อข้อมูลเพิ่มขึ้น MongoDB ขยายระบบได้ 2 แนวทาง คือ Replica Set ซึ่งสำเนาข้อมูลไว้หลายเครื่อง หากเครื่องหลักขัดข้อง เครื่องสำรองจะทำหน้าที่แทนโดยอัตโนมัติ และ Sharding ซึ่งกระจายข้อมูลของคอลเลกชันเดียวไปเก็บไว้หลายเครื่องตามค่าของคีย์ที่กำหนด ทำให้รองรับข้อมูลและปริมาณการอ่านเขียนที่เพิ่มขึ้นได้โดยไม่ต้องเปลี่ยนเป็นเครื่องที่ใหญ่ขึ้น [20]

2.9.1 ข้อดี

2.9.1.1 โครงสร้างข้อมูลยืดหยุ่น เพิ่มฟิลด์ได้โดยไม่ต้องแก้โครงสร้างตาราง

2.9.1.2 รองรับข้อมูลปริมาณมากและเก็บอาร์เรย์ในเอกสารได้โดยตรง

2.9.2 ข้อเสีย

2.9.2.1 ไม่มีการบังคับความสัมพันธ์ระหว่างข้อมูลเหมือนฐานข้อมูลเชิงสัมพันธ์

2.9.3 การนำไปใช้ในโครงการ โครงงานนี้ใช้ MongoDB จัดเก็บบทสนทนาและข้อความจากทุกช่องทาง ซึ่งมีหลายประเภทและมีปริมาณมาก รวมถึงเอกสารความรู้ของแชทบอท

2.10 สถาปัตยกรรม Multi-Tenant

สถาปัตยกรรม Multi-Tenant คือการออกแบบให้ระบบเดียวให้บริการผู้เช่าหลายรายโดยข้อมูลของผู้เช่าแต่ละรายแยกออกจากกัน วิธีที่ใช้กันทั่วไปคือใช้ฐานข้อมูลร่วมกันและระบุรหัสผู้เช่าในทุกตาราง แล้วกรองข้อมูลตามรหัสผู้เช่าทุกครั้งที่อ่านหรือเขียน การแยกข้อมูลของผู้เช่าทำได้ 3 รูปแบบ ได้แก่ แยกฐานข้อมูลให้ผู้เช่าแต่ละราย ซึ่งแยกข้อมูลได้เด็ดขาดแต่ดูแลยากเมื่อผู้เช่ามาก แยกเป็น Schema ภายในฐานข้อมูลเดียว และใช้ตารางร่วมกันโดยมีรหัสผู้เช่ากำกับทุกแถว ซึ่งประหยัดทรัพยากรและเพิ่มผู้เช่าใหม่ได้ง่าย แต่ต้องระวังการกรองข้อมูล [21]

ความเสี่ยงหลักของการใช้ตารางร่วมกันคือข้อมูลรั่วไหลข้ามผู้เช่า หากผู้พัฒนาลืมกรองรหัสผู้เช่าในคำสั่งค้นหาใดคำสั่งหนึ่ง ระบบจึงนิยมบังคับการกรองไว้ที่ชั้นกลาง เช่น Global Scope ของ ORM ที่เพิ่มเงื่อนไขรหัสผู้เช่าให้ทุกคำสั่งค้นหาอัตโนมัติ แทนการเขียนเงื่อนไขเองในทุกจุด และตรวจสอบรหัสผู้เช่าของข้อมูลทุกครั้งก่อนอนุญาตให้แก้ไข [21]

2.10.1 การนำไปใช้ในโครงการ ทุกตารางและทุกเอกสารมีรหัสร้านค้ากำกับ ร้านค้าหนึ่งแห่งมีได้หลายธุรกิจ แต่ละธุรกิจมีช่องทาง แบรนด์ และการตั้งค่าแชทบอทของตนเอง

2.11 การควบคุมสิทธิ์ตามบทบาท (RBAC)

RBAC (Role-Based Access Control) คือการควบคุมการเข้าถึงระบบโดยกำหนดสิทธิ์ให้แก่บทบาท แล้วมอบบทบาทให้ผู้ใช้ เมื่อต้องการเปลี่ยนสิทธิ์ของผู้ใช้หลายคน ผู้ดูแลแก้ไขเฉพาะบทบาทโดยไม่ต้องแก้ทีละคน ช่วยลดความผิดพลาดในการกำหนดสิทธิ์และตรวจสอบสิทธิ์ของผู้ใช้ได้ง่าย [22]

RBAC มีองค์ประกอบหลัก 3 ส่วน ได้แก่ ผู้ใช้ (User) บทบาท (Role) และสิทธิ์ (Permission) ผู้ใช้หนึ่งคนมีได้หลายบทบาท และบทบาทหนึ่งมีได้หลายสิทธิ์ สิทธิ์ของผู้ใช้จึงเท่ากับสิทธิ์รวมของทุกบทบาทที่ได้รับ [22] ต่างจากการกำหนดสิทธิ์ให้ผู้ใช้โดยตรงแบบ Access Control List ซึ่งจัดการยากเมื่อมีผู้ใช้จำนวนมาก

การออกแบบบทบาทควรยึดหลักสิทธิ์น้อยที่สุด (Least Privilege) คือให้แต่ละบทบาทได้รับเฉพาะสิทธิ์ที่จำเป็นต่อหน้าที่ เพื่อจำกัดความเสียหายหากบัญชีถูกใช้งานผิดวัตถุประสงค์ และการตรวจสิทธิ์ต้องทำที่ฝั่งเครื่องแม่ข่ายทุกครั้งที่มีคำขอ ไม่ใช่เพียงซ่อนปุ่มบนหน้าเว็บ เพราะผู้ใช้สามารถส่งคำขอไปยังระบบได้โดยตรง

2.11.1 การนำไปใช้ในโครงการ ระบบมีบทบาทพื้นฐาน 5 บทบาท ได้แก่ owner, manager, admin, agent และ viewer และเจ้าของร้านสร้างบทบาทใหม่ได้ โดยเลือกจากสิทธิ์ทั้งหมด 13 รายการ เช่น การตอบข้อความ การมอบหมายงาน และการจัดการคำสั่งซื้อ

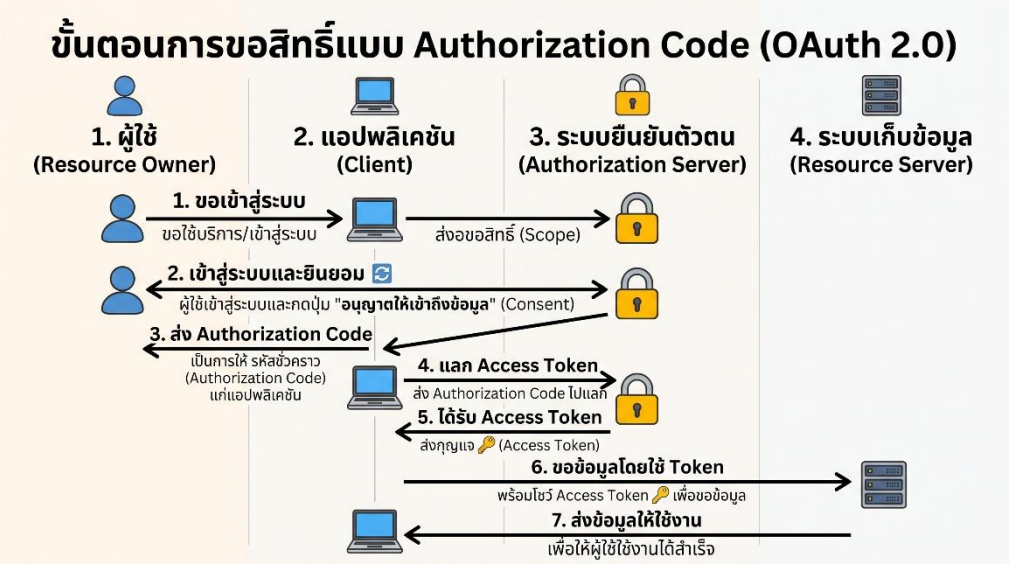
2.12 OAuth 2.0

OAuth 2.0 คือมาตรฐานการอนุญาตให้แอปพลิเคชันเข้าถึงข้อมูลของผู้ใช้บนบริการอื่นโดยผู้ใช้ไม่ต้องให้รหัสผ่านแก่แอปพลิเคชัน ผู้ใช้เข้าสู่ระบบและยินยอมที่ผู้ให้บริการแล้วแอปพลิเคชันจะได้รับ Access Token สำหรับเรียกใช้ API แทนผู้ใช้ [23]

OAuth 2.0 ออกแบบมาเพื่อการมอบสิทธิ์ (Authorization) ให้แอปพลิเคชันเข้าถึงข้อมูล ไม่ได้ออกแบบมาเพื่อยืนยันตัวตนโดยตรง จึงมีมาตรฐาน OpenID Connect ที่ต่อยอดจาก OAuth 2.0 โดยเพิ่ม ID Token สำหรับระบุตัวตนของผู้ใช้ นอกจากนี้ผู้ให้บริการบางรายออก Refresh Token ให้แอปพลิเคชันขอ Access Token ใหม่ได้เมื่อ Token เดิมหมดอายุ โดยผู้ใช้ไม่ต้องกลับไปยินยอมที่หน้าของผู้ให้บริการซ้ำอีกครั้ง

Access Token ที่ผู้ให้บริการออกให้มักเป็นแบบ Bearer Token คือผู้ใดถือ Token ก็เรียกใช้ API ได้ทันที จึงต้องส่งผ่าน HTTPS เท่านั้นและห้ามเปิดเผยในที่สาธารณะ ส่วนผู้ใช้ยกเลิกสิทธิ์ที่เคยให้แก่แอปพลิเคชันได้ทุกเมื่อ จากหน้าการตั้งค่าของผู้ให้บริการ ซึ่งทำให้ Token เดิมใช้งานไม่ได้ทันที และแอปพลิเคชันต้องขอสิทธิ์จากผู้ใช้ใหม่

OAuth 2.0 กำหนดบทบาทไว้ 4 บทบาท ได้แก่ เจ้าของข้อมูล (Resource Owner) คือ ผู้ใช้ แอปพลิเคชันที่ขอเข้าถึงข้อมูล (Client) เครื่องแม่ข่ายที่ยืนยันตัวตนและออก Token (Authorization Server) และเครื่องแม่ข่ายที่เก็บข้อมูล (Resource Server) [23] แอปพลิเคชันระบุขอบเขตสิทธิ์ที่ต้องการด้วย Scope และ Token ที่ได้รับมีอายุจำกัด รูปแบบที่นิยมใช้กับระบบเว็บคือ Authorization Code ซึ่งมีขั้นตอนการขอสิทธิ์แสดงดังรูปที่ 2.5



รูปที่ 2.5 ขั้นตอนการขอสิทธิ์แบบ Authorization Code ของ OAuth 2.0

จากรูปที่ 2.5 ระบบส่งผู้ใช้ไปยังหน้ายินยอมของผู้ให้บริการ เมื่อผู้ใช้ยินยอม ผู้ให้บริการจะส่งรหัส Authorization Code กลับมายัง Callback URL ของระบบ จากนั้นเครื่องแม่ข่ายของระบบนำรหัสนี้พร้อม Client Secret ไปแลกเป็น Access Token โดยตรง Token จึงไม่ผ่านเบราว์เซอร์ของผู้ใช้ การเข้าสู่ระบบด้วยบัญชีภายนอกใช้ Token นี้ดึงข้อมูลโปรไฟล์มายืนยันตัวตนผู้ใช้

2.12.1 การนำไปใช้ในโครงการ ระบบใช้ Laravel Socialite ให้ผู้ใช้เข้าสู่ระบบด้วยบัญชี Facebook หรือ Google และใช้ OAuth ของ Facebook ขอสิทธิ์จัดการข้อความของเพจ

2.13 REST API

REST API คือรูปแบบการออกแบบส่วนต่อประสานสำหรับสื่อสารระหว่างระบบผ่านโพรโทคอล HTTP โดยระบุทรัพยากรด้วย URL และใช้เมธอด GET POST PUT PATCH และ DELETE แทนการอ่าน สร้าง และแก้ไขข้อมูล ข้อมูลที่รับส่งนิยมใช้รูปแบบ JSON [24]

REST ถูกเสนอโดย Fielding [25] โดยมีหลักการสำคัญ ได้แก่ แยกฝั่งผู้ใช้บริการกับฝั่งให้บริการออกจากกัน (Client-Server) ทุกคำขอต้องมีข้อมูลครบในตัวเองโดยเครื่องแม่ข่ายไม่ต้องจำสถานะของคำขอก่อนหน้า (Stateless) คำตอบระบุได้ว่าเก็บแคชได้หรือไม่ (Cacheable) และใช้รูปแบบการเรียกที่เป็นมาตรฐานเดียวกันทั้งระบบ (Uniform Interface)

การออกแบบ REST API นิยมตั้ง URL เป็นคำนามแทนทรัพยากร เช่น /conversations/{id}/messages หมายถึงข้อความของบทสนทนาหนึ่ง แล้วใช้เมธอดของ HTTP ระบุการกระทำ และใช้รหัสสถานะ HTTP แจ้งผลลัพธ์ ได้แก่ กลุ่ม 2xx หมายถึงสำเร็จ กลุ่ม 4xx หมายถึงคำขอผิดพลาด เช่น 401 ยังไม่ได้เข้าสู่ระบบ 403 ไม่มีสิทธิ์ 404 ไม่พบข้อมูล และ 422 ข้อมูลไม่ถูกต้อง ส่วนกลุ่ม 5xx หมายถึงเครื่องแม่ข่ายขัดข้อง [24]

2.13.1 การนำไปใช้ในโครงการ หน้ากล่องข้อความที่พัฒนาด้วย React ดึงและส่งข้อมูลผ่าน API รูปแบบ JSON ของระบบเว็บ เช่น ดูรายการบทสนทนา ตอบข้อความ และเปิดคำสั่งซื้อ โดยยืนยันตัวตนด้วย Session ของผู้ใช้ที่เข้าสู่ระบบ และ WebChat widget ที่ฝังบนเว็บไซต์ของร้านค้ารับส่งข้อความผ่าน API ชุดแยกของ WebChat ซึ่งไม่ต้องเข้าสู่ระบบ

2.14 แชทบอท

แชทบอทคือโปรแกรมที่สนทนากับผู้ใช้ด้วยภาษาธรรมชาติผ่านช่องทางแชท แชทบอทแบบกฎจะตอบตามคำสำคัญหรือเงื่อนไขที่กำหนดไว้ล่วงหน้า ส่วนแชทบอทที่ใช้แบบจำลองภาษาสามารถสร้างคำตอบใหม่ได้ตามบริบทของการสนทนา แต่อาจสร้างข้อมูลที่ไม่มีอยู่จริงได้ ระบบแชทบอทสำหรับร้านค้าจึงต้องมีกลไกส่งต่อบทสนทนาให้เจ้าหน้าที่เมื่อคำถามอยู่นอกเหนือข้อมูลที่แชทบอทมี [26]

แชทบอทมีมาตั้งแต่ ELIZA [27] ซึ่งตอบผู้ใช้ด้วยการจับรูปแบบคำแล้วเติมคำตอบตามแม่แบบ ปัจจุบันแชทบอทโดยทั่วไปทำงานเป็นขั้นตอน ได้แก่ รับข้อความ ทำความเข้าใจเจตนาของผู้ใช้ ค้นหาข้อมูลที่ต้องใช้ สร้างคำตอบ และส่งคำตอบกลับ แชทบอทแบบกฎทำงานแม่นยำและคาดเดาได้แต่ตอบได้เฉพาะคำถามที่เตรียมไว้ ส่วนแชทบอทที่ใช้แบบจำลองภาษาเข้าใจคำถามที่ใช้ถ้อยคำหลากหลายได้ดีกว่า

ในงานบริการลูกค้าของร้านค้าออนไลน์ คำถามจำนวนมากซ้ำกัน เช่น ราคาสินค้า วิธีสั่งซื้อ และค่าจัดส่ง แชทบอทช่วยตอบคำถามเหล่านี้ได้ตลอด 24 ชั่วโมง ลดเวลารอของลูกค้าและลดภาระของเจ้าหน้าที่ ระบบที่ดีจึงให้แชทบอทกับเจ้าหน้าที่ทำงานร่วมกัน โดยให้เจ้าหน้าที่ตรวจหรือรับช่วงบทสนทนาต่อเมื่อแชทบอทไม่มั่นใจในคำตอบ

การพัฒนาแชทบอทภาษาไทยมีความท้าทายเพิ่มเติม เพราะภาษาไทยเขียนต่อกันโดยไม่เว้นวรรคระหว่างคำ ลูกค้ามักพิมพ์ภาษาพูด คำย่อ หรือสะกดผิด และมักถามหลายเรื่องในข้อความ

เดียว แชทบอทแบบกฎที่จับคำสำคัญจึงพลาดได้ง่าย ส่วนแบบจำลองภาษาที่ฝึกด้วยข้อมูลภาษาไทยจำนวนมากรับมือกับข้อความลักษณะนี้ได้ดีกว่า

2.14.1 การนำไปใช้ในโครงการ แชทบอทตอบลูกค้าได้ทุกช่องทาง โดยร้านค้ากำหนดโหมดเริ่มต้นของแต่ละธุรกิจ และเปลี่ยนโหมดแยกตามบทสนทนาได้ 3 โหมด ดังนี้

2.14.1.1 Manual เจ้าหน้าที่ตอบลูกค้าเองทั้งหมด

2.14.1.2 Suggest แชทบอทเสนอคำตอบให้เจ้าหน้าที่เลือกใช้

2.14.1.3 Autopilot แชทบอทตอบลูกค้าเอง ส่งต่อให้เจ้าหน้าที่เมื่อตอบไม่ได้

2.14.2 การตอบอัตโนมัติด้วยคีย์เวิร์ด นอกจากแชทบอทที่ใช้แบบจำลองภาษา ระบบมีการตอบอัตโนมัติแบบกฎ ร้านค้ากำหนดคีย์เวิร์ดและข้อความตอบกลับได้เอง เมื่อข้อความของลูกค้าตรงกับคีย์เวิร์ดทั้งข้อความหรือมีคีย์เวิร์ดอยู่ ระบบจะส่งข้อความที่กำหนดไว้ทันที

2.15 แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM)

แบบจำลองภาษาขนาดใหญ่คือแบบจำลองที่ผ่านการฝึกด้วยข้อความจำนวนมหาศาลเพื่อทำนายคำถัดไปของข้อความ แบบจำลองส่วนใหญ่ใช้สถาปัตยกรรม Transformer ซึ่งใช้กลไก Self-Attention พิจารณาความสัมพันธ์ระหว่างคำทุกคำในข้อความพร้อมกัน [28] เมื่อแบบจำลองมีขนาดใหญ่เพียงพอ จะสามารถทำงานใหม่ได้จากคำสั่งและตัวอย่างที่ให้ไว้ในข้อความนำเข้า โดยไม่ต้องฝึกแบบจำลองเพิ่มเติม [29]

แบบจำลองภาษาสร้างข้อความทีละส่วน เริ่มจากแบ่งข้อความนำเข้าเป็นหน่วยย่อยที่เรียกว่าโทเคน (Token) แปลงแต่ละโทเคนเป็นเวกเตอร์ แล้วส่งผ่านชั้น Transformer หลายชั้นเพื่อคำนวณความน่าจะเป็นของโทเคนถัดไป จากนั้นเลือกโทเคนหนึ่งต่อท้ายข้อความและทำซ้ำจนจบคำตอบ ค่า Temperature กำหนดความหลากหลายของการเลือก ค่าต่ำให้คำตอบคงที่ ค่าสูงให้คำตอบหลากหลาย และจำนวนโทเคนที่รับได้ในครั้งเดียวถูกจำกัดด้วยขนาดหน้าต่างบริบท (Context Window) [30]

การกำหนดพฤติกรรมของแบบจำลองทำผ่านข้อความสั่งงาน (Prompt) ซึ่งมักประกอบด้วยข้อความสั่งงานระบบ (System Prompt) ที่กำหนดบทบาทและข้อห้าม ข้อมูลอ้างอิง และประวัติการสนทนา ข้อจำกัดของแบบจำลองภาษา ได้แก่ การสร้างข้อมูลที่ไม่มีอยู่จริง (Hallucination) และความรู้ที่หยุดอยู่ ณ วันที่ฝึก การใช้งานทำได้ทั้งผ่านบริการคลาวด์ หรือติดตั้งแบบจำลองแบบเปิดบนเครื่องแม่ข่ายของตนเอง ซึ่งข้อมูลไม่ถูกส่งออกนอกองค์กรแต่ต้องใช้หน่วยประมวลผลกราฟิกที่มีประสิทธิภาพ [30]

2.15.1 การนำไปใช้ในโครงการ ระบบประกอบข้อความสั่งงานจากบุคลิกที่ร้านค้ากำหนด ข้อมูลแบรนด์ ผลการค้นคืนจากฐานความรู้ บทสรุปของบทสนทนา และข้อความล่าสุด 20 ข้อความ

แล้วส่งให้แบบจำลองภาษาสร้างคำตอบ ค่าเริ่มต้นของระบบใช้แบบจำลอง qwen3:14b ผ่าน Ollama ที่ติดตั้งบนเครื่องแม่ข่ายของบริษัทเอง และร้านค้าเพิ่มผู้ให้บริการแบบจำลองภาษาที่รองรับรูปแบบ OpenAI API ได้เอง เช่น OpenRouter, OpenAI, Google Gemini และ Typhoon โดยใส่ URL, API key ของร้าน และเลือกแบบจำลองที่ต้องการ

2.16 ผู้ให้บริการแบบจำลองภาษา (AI Provider)

ผู้ให้บริการแบบจำลองภาษาคือบริการที่เปิดให้ระบบอื่นเรียกใช้แบบจำลองภาษาผ่าน API โดยไม่ต้องติดตั้งแบบจำลองเอง ผู้ใช้สมัครบัญชีกับผู้ให้บริการ สร้าง API key แล้วส่งคำขอพร้อมคีย์นั้นทุกครั้งที่เรียกใช้ ค่าบริการส่วนใหญ่คิดตามจำนวนโทเคนของข้อความที่ส่งเข้าและข้อความที่ได้รับกลับ ผู้ให้บริการแบ่งได้เป็นบริษัทที่พัฒนาแบบจำลองเอง เช่น OpenAI, Anthropic และ Google ผู้ให้บริการที่นำแบบจำลองแบบเปิดมาให้บริการ เช่น Groq และ Together AI และบริการตัวกลางอย่าง OpenRouter ที่รวมแบบจำลองจากหลายผู้ให้บริการไว้ภายใต้ API เดียว ส่วนองค์กรที่ต้องการเก็บข้อมูลไว้ภายในสามารถติดตั้งแบบจำลองแบบเปิดบนเครื่องแม่ข่ายของตนเองด้วยซอฟต์แวร์อย่าง Ollama ได้ [31]

รูปแบบ API ที่ผู้ให้บริการจำนวนมากรองรับคือรูปแบบ Chat Completions ของ OpenAI ซึ่งกลายเป็นมาตรฐานโดยพฤตินัย เรียกว่า OpenAI-compatible API การเรียกใช้ทำด้วยคำขอ POST ไปยัง {Base URL}/chat/completions แนบ API key ใน Header Authorization แบบ Bearer และส่งชื่อแบบจำลองพร้อมรายการข้อความ ซึ่งแต่ละข้อความระบุบทบาทเป็น system, user หรือ assistant คำตอบที่ได้อยู่ในฟิลด์ choices นอกจากนี้ยังมี GET {Base URL}/models สำหรับดูรายชื่อแบบจำลองที่ใช้ได้ เมื่อผู้ให้บริการใช้รูปแบบเดียวกัน ระบบจึงเปลี่ยนผู้ให้บริการได้เพียงเปลี่ยน Base URL, API key และชื่อแบบจำลอง โดยไม่ต้องแก้โค้ด [31]

2.16.1 ข้อดี

2.16.1.1 เลือกใช้แบบจำลองได้หลากหลายตามงบประมาณและคุณภาพที่ต้องการ

2.16.1.2 ไม่ต้องลงทุนเครื่องแม่ข่ายและหน่วยประมวลผลกราฟิกเอง

2.16.2 ข้อเสีย

2.16.2.1 มีค่าใช้จ่ายตามปริมาณการใช้งาน และถูกจำกัดจำนวนคำขอต่อช่วงเวลา

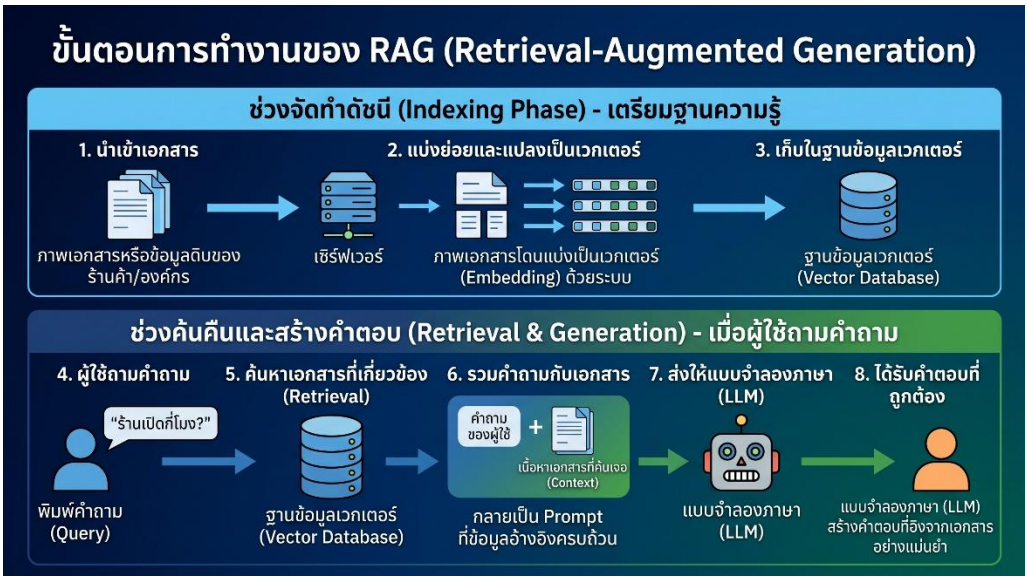
2.16.2.2 ข้อความของลูกค้าถูกส่งออกไปประมวลผลภายนอก และ API key ที่รั่วไหลอาจถูกนำไปใช้จนเกิดค่าใช้จ่าย จึงต้องเก็บคีย์เป็นความลับ

2.16.3 การนำไปใช้ในโครงการ ระบบใช้ Ollama บนเครื่องแม่ข่ายของบริษัทเป็นค่าเริ่มต้น และให้ร้านค้าเพิ่มผู้ให้บริการที่รองรับ OpenAI-compatible API ได้เอง โดยกรอก Base

URL, API key และเลือกแบบจำลอง ระบบเข้ารหัส API key ก่อนจัดเก็บ รายละเอียดอธิบายไว้ในหัวข้อ 3.8

2.17 RAG (Retrieval-Augmented Generation)

RAG คือเทคนิคที่ค้นคืนเอกสารที่เกี่ยวข้องกับคำถามจากฐานความรู้ก่อน แล้วส่งเอกสารเหล่านั้นให้แบบจำลองภาษาใช้ประกอบการสร้างคำตอบ [32] ช่วยให้แบบจำลองตอบจากข้อมูลของร้านค้าที่ไม่ได้อยู่ในข้อมูลที่ใช้ฝึก และลดการสร้างข้อมูลที่ไม่มีอยู่จริง ขั้นตอนการทำงานของ RAG ทั้งช่วงจัดทำดัชนีและช่วงค้นคืนเพื่อสร้างคำตอบ แสดงดังรูปที่ 2.6



รูปที่ 2.6 ขั้นตอนการทำงานของ RAG

จากรูปที่ 2.6 RAG ทำงานเป็น 2 ช่วง ช่วงแรกคือการจัดทำดัชนี ระบบแบ่งเอกสารเป็นส่วนย่อย (Chunk) แปลงแต่ละส่วนเป็นเวกเตอร์ แล้วจัดเก็บไว้ ช่วงที่สองคือการค้นคืนและสร้างคำตอบ ระบบแปลงคำถามเป็นเวกเตอร์ ค้นหาส่วนย่อยที่ใกล้เคียงที่สุดจำนวน k อันดับแรก แล้วนำมาใส่ในข้อความสั่งงานพร้อมคำถาม ให้แบบจำลองภาษาสร้างคำตอบจากข้อมูลนั้น

เมื่อเทียบกับการฝึกแบบจำลองเพิ่มเติม (Fine-tuning) RAG ปรับปรุงความรู้ได้ทันทีเพียงแก้ไขเอกสาร ไม่ต้องฝึกแบบจำลองใหม่ ใช้ต้นทุนต่ำ และระบุได้ว่าคำตอบมาจากเอกสารใด คุณภาพของคำตอบขึ้นกับการแบ่งส่วนและการค้นคืน ส่วนย่อยที่เล็กเกินไปจะขาดบริบท ส่วนย่อยที่ใหญ่เกินไปจะมีข้อมูลที่ไม่เกี่ยวข้องปะปน และหากค้นคืนผิดเอกสาร แบบจำลองก็จะตอบผิดตามไปด้วย [33]

2.17.1 การนำไปใช้ในโครงการ ร้านค้าเพิ่มเอกสารความรู้และนำข้อมูลสินค้าเข้าสู่ฐานความรู้ได้ ระบบแบ่งเอกสารเป็นส่วนย่อยขนาดไม่เกิน 500 ตัวอักษร จัดเก็บพร้อมเวกเตอร์ใน MongoDB และค้นคืนส่วนย่อยที่เกี่ยวข้อง 4 อันดับแรกมาประกอบข้อความสั่งงาน

2.18 Embedding และความคล้ายโคไซน์

Embedding คือการแปลงข้อความเป็นเวกเตอร์ตัวเลขที่มีมิติคงที่ โดยข้อความที่มีความหมายใกล้เคียงกันจะได้เวกเตอร์ที่อยู่ใกล้กันในปริภูมิเวกเตอร์ [34] ความใกล้เคียงวัดได้จากความคล้ายโคไซน์ $\cos(\theta) = (A \cdot B)(\| A \| \| B \|)$ ค่าที่เข้าใกล้ 1 แสดงว่าข้อความทั้งสองมีความหมายใกล้เคียงกัน ส่วนค่าที่เข้าใกล้ 0 แสดงว่าไม่เกี่ยวข้องกัน

แบบจำลอง Embedding ในปัจจุบันพัฒนาบนสถาปัตยกรรม Transformer แปลงข้อความทั้งประโยคหรือทั้งย่อหน้าเป็นเวกเตอร์ขนาดหลายร้อยถึงหลายพันมิติ และหลายแบบจำลองรองรับหลายภาษา การค้นหาด้วย Embedding จึงวัดจากความหมาย ไม่ใช่การตรงกันของตัวอักษร ทำให้ค้นพบข้อความที่เกี่ยวข้องแม้ใช้คำต่างกัน เช่น คำถามเรื่อง “ค่าส่ง” ค้นพบเอกสารที่เขียนว่า “ค่าจัดส่ง” [35]

ในสูตรความคล้ายโคไซน์ A · B คือผลรวมของผลคูณสมาชิกแต่ละมิติของเวกเตอร์ทั้งสอง ส่วน $\| A \|$ และ $\| B \|$ คือขนาดของเวกเตอร์ ผลลัพธ์อยู่ในช่วง -1 ถึง 1 และขึ้นกับทิศทางของเวกเตอร์เท่านั้น ไม่ขึ้นกับความยาวของข้อความ การเทียบคำถามกับทุกส่วนย่อยทำได้ง่ายและแม่นยำ แต่ใช้เวลามากขึ้นตามจำนวนเอกสาร หากข้อมูลมีปริมาณมากจึงนิยมใช้ดัชนีเวกเตอร์ช่วยค้นหาแบบประมาณ

2.18.1 การนำไปใช้ในโครงการ ระบบแปลงเอกสารและคำถามเป็นเวกเตอร์ด้วยแบบจำลอง Embedding bge-m3 ผ่าน Ollama แล้วคำนวณความคล้ายโคไซน์ระหว่างคำถามกับทุกส่วนย่อย และเลือกเฉพาะส่วนย่อยที่มีค่าตั้งแต่ 0.3 ขึ้นไป

2.19 งานวิจัยที่เกี่ยวข้อง

ผู้จัดทำได้ศึกษางานวิจัยและโครงงานที่เกี่ยวข้องกับแชทบอท การบริการลูกค้าหลายช่องทาง และระบบจัดการคำสั่งซื้อที่เชื่อมต่อกับ LINE เพื่อนำแนวคิด จุดเด่น และข้อจำกัดของแต่ละงานมาประยุกต์ใช้และเปรียบเทียบกับระบบที่พัฒนาในโครงงานนี้ ดังนี้

2.19.1 แชทบอทค้นคว้าปริญญานิพนธ์ของห้องสมุด พัชรา วงค์ตาผา [36] วิจัยเรื่องการพัฒนาระบบแชทบอทสำหรับการค้นคว้าปริญญานิพนธ์ ระดับปริญญาตรี ของห้องสมุดคณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี โดยพัฒนาแชทบอทบน LINE Official Account ระบบรับข้อความผ่าน LINE Messaging API แล้วใช้ Google Apps Script ค้นหาข้อมูลจาก Google Sheet ส่งผลลัพธ์กลับให้ผู้ใช้ และมีริชเมนูเชื่อมไปยังฐานข้อมูลออนไลน์และบริการอื่

นของห้องสมุด ผลการประเมินกับผู้ใช้ 100 คน พบว่าความพึงพอใจโดยรวมอยู่ในระดับมากที่สุด (μ = 4.68) งานนี้แสดงว่าแชทบอทบน LINE ช่วยให้ผู้ใช้เข้าถึงข้อมูลได้รวดเร็ว แต่ตอบได้เฉพาะคำค้นที่ตรงกับข้อมูลในตาราง และรองรับเพียงช่องทาง LINE ไม่ครอบคลุมช่องทางแชทอื่นที่ผู้ใช้ติดต่อเข้ามา

2.19.2 ระบบจัดการสินค้าสำหรับผู้ผลิตรายย่อยกรณีศึกษาบ้านฟาร์มขนม ธนพล วุฒิธาดา และวชิรวิทย์ ชื่นจิตร [37] พัฒนาเว็บแอปพลิเคชันจัดการคำสั่งซื้อให้ผู้ผลิตขนมรายย่อย ร้านค้าในชุมชนสั่งซื้อสินค้าผ่าน LINE LIFF ส่วนผู้ผลิตตรวจสถานะคำสั่งซื้อ ค้นหาร้านค้าบนแผนที่ และสรุปยอดขายได้ ระบบพัฒนาด้วย Next.js และ Express.js จัดเก็บข้อมูลด้วย MySQL ผลการประเมินกับผู้ใช้ 20 คน ได้ค่าเฉลี่ยความพึงพอใจ 4.33 (S.D. = 0.65) และช่วยลดความผิดพลาดในการจัดการคำสั่งซื้อเมื่อเทียบกับวิธีเดิม งานนี้เน้นการสั่งซื้อผ่านแบบฟอร์ม ไม่มีส่วนสนทนาตอบคำถามลูกค้า

2.19.3 แชทบอทบน LINE สำหรับงานบริการด้านงานวิจัย มนต์ทิชา รัตนพันธ์ และฉัตรวดี สายใยทอง [38] พัฒนาแชทบอทบน LINE สำหรับงานบริการด้านงานวิจัยของสถาบันวิจัยและพัฒนา มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี โดยใช้ Dialogflow จำแนกเจตนาของคำถาม และใช้ Google Sheets เก็บข้อมูล ผลการประเมินกับบุคลากร 327 คน พบว่าความพึงพอใจโดยรวมอยู่ในระดับมาก (ค่าเฉลี่ย 3.83, S.D. = 0.85) และช่วยลดเวลาที่ผู้ใช้ต้องรอคำตอบจากเจ้าหน้าที่ แต่แชทบอทตอบได้เฉพาะเจตนาที่กำหนดไว้ล่วงหน้า และรองรับเพียงช่องทาง LINE

2.19.4 แชทบอทบน LINE สำหรับแจ้งซ่อมโสตทัศนูปกรณ์ ชรัช แสวงเจริญ และชุติมา พิศาลย์ [39] พัฒนาแชทบอทบน LINE Official Account ให้บุคลากรแจ้งซ่อมโสตทัศนูปกรณ์ ระบบรับข้อความผ่าน LINE Messaging API บันทึกข้อมูลลง Google Sheets ด้วย Google Apps Script และแจ้งสถานะการซ่อมกลับอัตโนมัติ ผลการทดสอบกับบุคลากร 95 คน พบว่าลดเวลาการแจ้งซ่อมได้มากกว่าร้อยละ 50 ลดการสอบถามสถานะซ้ำ และได้ความพึงพอใจเฉลี่ย 4.42 งานนี้แสดงว่าการรับเรื่องผ่านแชทช่วยลดงานซ้ำของเจ้าหน้าที่ได้ แต่ยังจำกัดอยู่ที่ช่องทางเดียว

2.19.5 แชทบอทบน Facebook Messenger สำหรับบริการลูกค้า วิสิฏฐา ชุนสิทธิ์ และรังสรรค์ เกียรติ์ภานนท์ [40] ใช้กระบวนการคิดเชิงออกแบบสัมภาษณ์ลูกค้า 42 คนที่ติดต่อบริษัทผ่าน Facebook เพื่อออกแบบต้นแบบแชทบอทบน Facebook Messenger แล้วนำกลับไปให้ลูกค้าทดสอบ ผลการศึกษาพบว่าต้นแบบตอบคำถามพื้นฐานได้ดี ลูกค้าเข้าถึงข้อมูลได้ตลอดเวลา และลดภาระพนักงานจากคำถามที่ซ้ำกัน ผู้วิจัยเสนอให้ศึกษาความคาดหวังของลูกค้าและออกแบบเส้นทางการสนทนาก่อนพัฒนาแชทบอท เพื่อให้คำตอบตรงกับความต้องการของลูกค้า

2.19.6 แชทบอทบน LINE ที่ใช้แบบจำลองภาษา สิปปกร พงศ์เดโชธนบดี และคณะ [41] พัฒนาแชทบอทบน LINE สำหรับสื่อสารและให้ข้อมูลแก่ลูกค้าของบริษัทกรณีศึกษา โดยใช้ LINE Messaging API ร่วมกับ Gemini AI และการประมวลผลภาษาธรรมชาติเพื่อจำแนกประเภทคำถาม

ของลูกค้า ผลการทดสอบกับผู้ใช้ 30 คน พบว่าความพึงพอใจโดยรวมอยู่ในระดับมากที่สุด (ค่าเฉลี่ย 4.54) งานนี้แสดงว่าแบบจำลองภาษาช่วยให้แชทบอทเข้าใจคำถามที่หลากหลายขึ้น แต่ยังรองรับเพียงช่องทาง LINE และส่งข้อมูลไปประมวลผลที่บริการภายนอก

2.19.7 ระบบซื้อขายสินค้าออนไลน์ของร้านค้า ณัฐภัทร สุวรรณศิลป์ และคณะ [42] พัฒนาระบบบริหารจัดการร้านค้าออนไลน์ของร้านกลับตานี สาขาพิษณุโลก ระบบรับคำสั่งซื้อ แสดงรายการสินค้า จำนวน ราคา วันเวลาที่สั่ง และที่อยู่จัดส่ง ผลการประเมินกับลูกค้า 30 คน พบว่าความพึงพอใจโดยรวมอยู่ในระดับมากที่สุด (ค่าเฉลี่ย 4.59, S.D. = 0.37) ระบบช่วยให้สั่งซื้อสะดวกและจัดการข้อมูลเป็นระเบียบขึ้น แต่ไม่มีส่วนสนทนากับลูกค้า

2.19.8 แชทบอทกับการบริการลูกค้าแบบ Omnichannel Ghosh et al. [43] เขียนบทความเรื่อง The Role of AI Enabled Chatbots in Omnichannel Customer Service เพื่อทบทวนบทบาทของปัญญาประดิษฐ์และแชทบอทในการบริการลูกค้าแบบ Omnichannel ซึ่งต่างจากแบบ Multichannel ตรงที่ลูกค้าเปลี่ยนช่องทางได้โดยบริการยังต่อเนื่อง และได้รับคำตอบที่สอดคล้องกันทุกช่องทาง ผู้วิจัยสรุปว่าแชทบอทที่ใช้การประมวลผลภาษาธรรมชาติตอบคำถามที่ซ้ำกันได้ตลอด 24 ชั่วโมง รับคำถามพร้อมกันได้จำนวนมาก และควรส่งต่อบทสนทนาให้เจ้าหน้าที่เมื่อแก้ปัญหาไม่ได้ ทั้งนี้องค์กรต้องคำนึงถึงการปกป้องข้อมูลส่วนบุคคลและความโปร่งใสในการใช้งานแชทบอท

2.19.9 สรุปงานวิจัยที่เกี่ยวข้อง งานวิจัยในประเทศไทยที่ศึกษา ทั้งงานของพัชรา วงค์ตาผา [36] มนต์ทิชา รัตนพันธ์ และฉัตรวดี สายใยทอง [38] ชรัช แสวงเจริญ และชุติมา พิศาลย์ [39] วิสิฏฐา ชุนสิทธิ์ และรังสรรค์ เกียรติ์ภานนท์ [40] และสิปปกร พงศ์เดโชธนบดี และคณะ [41] ยืนยันว่าแชทบอทช่วยลดเวลารอคำตอบและลดงานซ้ำของเจ้าหน้าที่ แต่ทุกงานรองรับเพียงช่องทางเดียว ส่วนงานของธนพล วุฒิธาดา และวชิรวิทย์ ชื่นจิตร [37] และณัฐภัทร สุวรรณศิลป์ และคณะ [42] จัดการคำสั่งซื้อได้ แต่ไม่มีส่วนสนทนากับลูกค้า ส่วน Ghosh et al. [43] ชี้ว่าการบริการลูกค้าควรรวมทุกช่องทางและให้แชทบอททำงานร่วมกับเจ้าหน้าที่ โครงงานนี้จึงพัฒนาระบบที่รวมแชทจาก LINE Official Account, Facebook Messenger, Instagram และ WebChat ไว้ในกล่องข้อความเดียว ใช้แชทบอทที่อาศัยแบบจำลองภาษาร่วมกับ RAG ตอบจากข้อมูลของร้านค้า ส่งต่อบทสนทนาให้เจ้าหน้าที่เมื่อตอบไม่ได้ และเชื่อมบทสนทนากับการจัดการสินค้าและคำสั่งซื้อในระบบเดียว

บทที่ 3
วิธีดำเนินงาน

ระบบเว็บของ Chatshop พัฒนาขึ้นเพื่อรวมแชทจากหลายช่องทางไว้ในที่เดียว และช่วยให้ร้านค้าจัดการทีม แชทบอท สินค้า ลูกค้า และคำสั่งซื้อได้จากระบบเดียว บทนี้อธิบายขั้นตอนการดำเนินงานตั้งแต่การวิเคราะห์ปัญหาจนถึงการพัฒนาระบบ โดยมีหัวข้อดังนี้

3.1 กำหนดปัญหาที่เกิดขึ้นกับระบบงาน

3.2 การวิเคราะห์ระบบ

3.3 การพัฒนาระบบ

3.4 โครงสร้างระบบเว็บ Chatshop

3.5 การออกแบบ API

3.6 การพัฒนาเว็บแอปพลิเคชัน

3.7 การเชื่อมต่อบริการภายนอก

3.8 การเชื่อมต่อผู้ให้บริการแบบจำลองภาษา

ซึ่งแต่ละหัวข้อมีรายละเอียดดังต่อไปนี้

3.1 กำหนดปัญหาที่เกิดขึ้นกับระบบงาน

ร้านค้าออนไลน์ที่ขายผ่านแชทต้องเปิดแอปพลิเคชันของแต่ละช่องทางแยกกัน เจ้าหน้าที่ต้องสลับหน้าจอไปมา เมื่อมีหลายคนช่วยกันตอบก็ตอบซ้ำกันหรือตกหล่น ไม่มีการแบ่งสิทธิ์ว่าใครเห็นข้อมูลใดได้ คำถามเดิมต้องพิมพ์ตอบซ้ำทุกวัน และการเปิดคำสั่งซื้อยังจดด้วยมือในแชท เมื่อจำนวนลูกค้าเพิ่มขึ้น ปัญหาเหล่านี้ทำให้ลูกค้าต้องรอนาน บางแชทไม่มีผู้ตอบเลย และเจ้าของร้านไม่มีข้อมูลสรุปว่าแต่ละช่องทางมีลูกค้าทักเข้ามามากน้อยเพียงใด หรือเจ้าหน้าที่แต่ละคนตอบได้เร็วเพียงใด ผู้จัดทำจึงออกแบบระบบเว็บที่รับข้อความจากทุกช่องทางเข้ามาในกล่องข้อความเดียว พร้อมระบบทีม แชทบอท และระบบจัดการคำสั่งซื้อที่เชื่อมกับหน้าแชท เพื่อลดขั้นตอนและความผิดพลาด

3.2 การวิเคราะห์ระบบ

ผู้จัดทำวิเคราะห์ระบบจากขั้นตอนการทำงานจริงของร้านค้าที่ขายผ่านแชท แล้วสรุปเป็นผู้ใช้งาน ความต้องการด้านการทำงาน และความต้องการด้านคุณภาพของระบบ ดังนี้

3.2.1 ผู้ใช้งานของระบบ แบ่งออกเป็น 3 กลุ่ม ดังนี้

3.2.1.1 เจ้าของร้าน ผู้สมัครใช้งานระบบ เชื่อมต่อช่องทางแชท เชิญสมาชิกเข้าทีม และกำหนดบทบาทของสมาชิก รวมถึงตั้งค่าร้านค้าและแชทบอท

3.2.1.2 เจ้าหน้าที่ ได้แก่ ผู้จัดการ แอดมิน และพนักงานตอบแชท ใช้งานได้ตามสิทธิ์ของบทบาทที่ได้รับ เช่น ตอบแชท เปิดคำสั่งซื้อ หรือดูรายงาน

3.2.1.3 ลูกค้า ผู้ส่งข้อความหาร้านผ่าน LINE, Facebook Messenger, Instagram หรือ WebChat โดยไม่ต้องเข้าใช้งานระบบโดยตรง

3.2.2 Functional Requirements แบ่งตามส่วนของระบบ ดังนี้

3.2.2.1 ส่วนกล่องข้อความและช่องทาง

1) ระบบต้องเชื่อมต่อ LINE Official Account, Facebook Page, Instagram และ WebChat ได้ โดยร้านค้าหนึ่งแห่งเชื่อมต่อได้หลายช่องทาง

2) ระบบต้องรับข้อความใหม่ผ่าน Webhook และแสดงในกล่องข้อความทันทีโดยไม่ต้องโหลดหน้าใหม่ พร้อมแจ้งจำนวนข้อความที่ยังไม่อ่าน

3) เจ้าหน้าที่ต้องตอบข้อความ สติกเกอร์ รูปภาพ และไฟล์ได้

4) เจ้าหน้าที่ต้องค้นหา ติดแท็ก และมอบหมายบทสนทนาได้

5) ร้านค้าต้องส่งข้อความบรอดแคสต์ตามกลุ่มลูกค้าและตั้งเวลาส่งได้

6) ร้านค้าต้องนำเข้าแชทเก่าจาก Facebook และ Instagram ได้

7) ร้านค้าต้องกำหนดให้เจ้าหน้าที่ยืนยันก่อนตอบแชทที่มอบหมายให้ผู้อื่นได้ และระบบต้องบันทึกการยืนยันไว้ตรวจสอบย้อนหลัง

3.2.2.2 ส่วนทีมและสิทธิ์การใช้งาน

1) ผู้ใช้ต้องเข้าสู่ระบบด้วยอีเมล Facebook หรือ Google ได้

2) เจ้าของร้านต้องเชิญสมาชิกและกำหนดบทบาทของสมาชิกได้

3) เจ้าของร้านต้องสร้างบทบาทใหม่โดยเลือกสิทธิ์การใช้งานเองได้

3.2.2.3 ส่วนแชทบอทและการตอบอัตโนมัติ

1) ร้านค้าต้องเลือกโหมดของแชทบอทได้ 3 โหมด และเจ้าหน้าที่ต้องเปลี่ยนโหมดเฉพาะบทสนทนาที่ต้องการได้ทันทีจากหน้ากล่องข้อความ

2) ร้านค้าต้องกำหนดบุคลิกของแชทบอทผ่านตัวช่วยตั้งค่า และทดลองคุยกับแชทบอทก่อนเปิดใช้งานจริงกับลูกค้าได้

3) ร้านค้าต้องเพิ่มเอกสารความรู้และคำสั่งพิเศษให้แชทบอทใช้ตอบได้

4) ร้านค้าต้องเพิ่มผู้ให้บริการแบบจำลองภาษาเองได้ โดย API key ของแต่ละร้านต้องเข้ารหัสก่อนจัดเก็บลงฐานข้อมูลทุกครั้ง

5) แชทบอทต้องส่งต่อบทสนทนาให้เจ้าหน้าที่เมื่อไม่สามารถตอบได้

6) ร้านค้าต้องตั้งกฎตอบกลับอัตโนมัติตามคีย์เวิร์ดได้

3.2.2.4 ส่วนการขายและรายงาน

1) ร้านค้าต้องเพิ่ม แก้ไข และลบข้อมูลสินค้าได้

2) เจ้าหน้าที่ต้องบันทึกข้อมูลลูกค้า และเปิดคำสั่งซื้อจากหน้าแชทได้

3) ระบบต้องแจ้งเตือนเจ้าหน้าที่เมื่อมีแชทค้างตอบนานเกินกำหนด

4) ระบบต้องแสดงรายงาน และส่งออกเป็นไฟล์ CSV ได้

5) ระบบต้องตรวจสลิปโอนเงินที่ลูกค้าส่งในแชท และเปลี่ยนสถานะคำสั่งซื้อเป็นชำระแล้วเมื่อสลิปถูกต้อง โดยไม่ต้องให้เจ้าหน้าที่ตรวจเอง

3.2.3 Non-Functional Requirements เป็นความต้องการด้านคุณภาพของระบบ ดังนี้

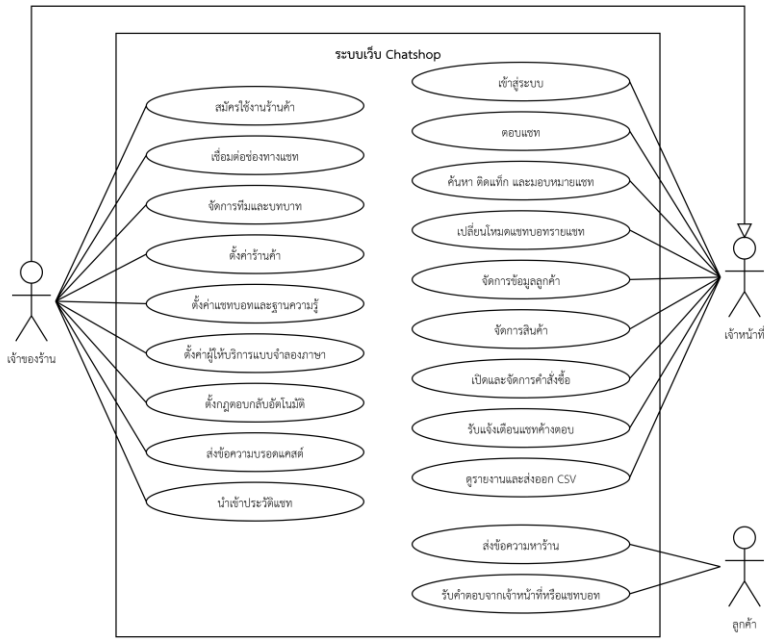
3.2.3.1 ด้านประสิทธิภาพ ระบบต้องตอบกลับ Webhook ได้ทันที แล้วจึงประมวลผลข้อความในคิวงานเบื้องหลัง เพื่อไม่ให้แพลตฟอร์มส่ง Webhook ซ้ำ

3.2.3.2 ด้านความปลอดภัย ระบบต้องตรวจลายเซ็นของ Webhook ทุกครั้ง ยืนยันตัวตนก่อนเรียกใช้งาน และตรวจสิทธิ์ตามบทบาทก่อนทำงาน

3.2.3.3 ด้านการแยกข้อมูล ข้อมูลของแต่ละร้านค้าต้องแยกออกจากกัน ผู้ใช้เห็นได้เฉพาะข้อมูลของร้านตนเอง แม้ทุกร้านจะใช้ฐานข้อมูลชุดเดียวกัน

3.2.3.4 ด้านความน่าเชื่อถือ ระบบต้องลองส่งงานใหม่เมื่อล้มเหลว และต้องไม่ส่งข้อความเดิมซ้ำให้ลูกค้า แม้งานเดียวกันจะถูกประมวลผลซ้ำหลายครั้ง

3.2.4 Use Case Diagram แสดงขอบเขตของระบบและความสัมพันธ์ระหว่างผู้ใช้งานกับฟังก์ชันของระบบเว็บ ซึ่งผู้ใช้งานแต่ละกลุ่มมีสิทธิ์ต่างกัน ดังรูปที่ 3.1



รูปที่ 3.1 Use Case Diagram ของระบบเว็บ Chatshop

จากรูปที่ 3.1 ผู้ใช้งานแต่ละกลุ่มทำงานได้ ดังนี้

3.2.4.1 เจ้าของร้าน จัดการช่องทาง ทีม บทบาท การตั้งค่าร้าน แชทบอท และเข้าถึงทุกฟังก์ชันของเจ้าหน้าที่ได้โดยไม่ต้องกำหนดสิทธิ์เพิ่ม

3.2.4.2 เจ้าหน้าที่ ตอบแชทและเปิดคำสั่งซื้อตามสิทธิ์ของบทบาท

3.2.4.3 ลูกค้า ส่งข้อความและรับคำตอบจากเจ้าหน้าที่หรือแชทบอท

3.3 การพัฒนาระบบ

การพัฒนาระบบเริ่มจากออกแบบโครงสร้างและการจัดเก็บข้อมูล จากนั้นจึงออกแบบขั้นตอนการทำงานหลัก 2 ขั้นตอน คือการรับข้อความจากช่องทางต่าง ๆ และการสร้างคำตอบของแชทบอท ทั้งสองขั้นตอนทำงานเบื้องหลังโดยผู้ใช้มองไม่เห็น แต่ส่งผลต่อความเร็วและความถูกต้องของระบบ

3.3.1 การออกแบบระบบ ระบบแบ่งการทำงานออกเป็น 3 ส่วน ดังนี้

3.3.1.1 Frontend ใช้ Blade สร้างหน้าเว็บทั่วไป และใช้ React สร้างหน้ากล่องข้อความที่ต้องแสดงข้อความใหม่ตลอดเวลาโดยไม่ต้องโหลดหน้าใหม่

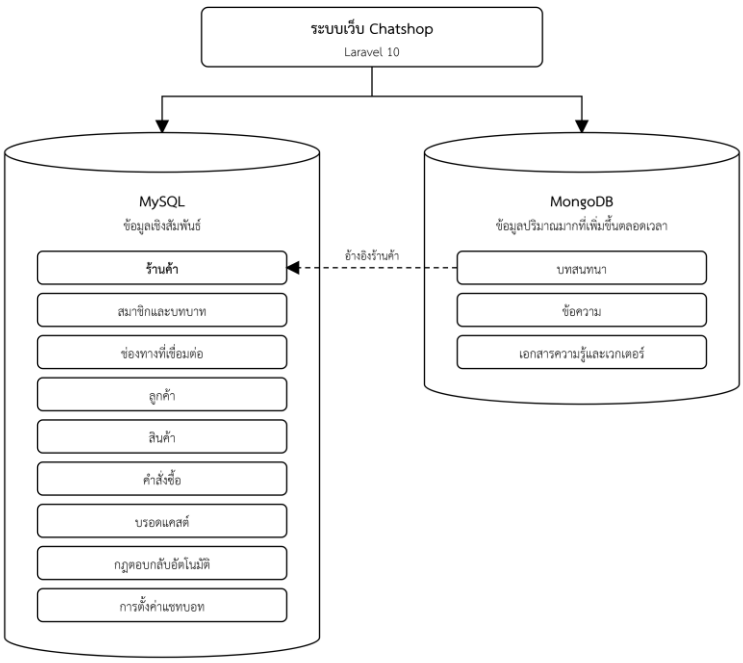
3.3.1.2 Backend ใช้ Laravel 10 รับ Webhook ประมวลผลข้อมูล ทำงานเบื้องหลังผ่านคิวงาน และเปิด API ให้หน้าเว็บเรียกใช้

3.3.1.3 Database ใช้ MySQL เก็บข้อมูลเชิงสัมพันธ์ และใช้ MongoDB เก็บบทสนทนา ข้อความ และฐานความรู้ของแชทบอท

ทั้ง 3 ส่วนทำงานร่วมกัน โดยหน้าเว็บเรียก API ของ Backend ด้วย Session ของผู้ใช้ที่เข้าสู่ระบบ ส่วน Backend แจ้งเหตุการณ์ใหม่กลับไปยังหน้าเว็บผ่าน WebSocket ของ Laravel Reverb ทางช่องส่วนตัวของแต่ละร้านค้า เช่น ข้อความใหม่ การแจ้งเตือน สถานะกำลังพิมพ์ของแชทบอท และคำสั่งซื้อที่เปลี่ยนสถานะ หน้าเว็บจึงแสดงข้อมูลล่าสุดได้โดยไม่ต้องสอบถามเครื่องแม่ข่ายซ้ำเป็นระยะ

งานที่ใช้เวลานานแยกไปทำในคิวงาน 3 คิวตามความเร่งด่วน ได้แก่ คิวความสำคัญสูงสำหรับประมวลผล Webhook การตอบกลับตามคีย์เวิร์ด และการนำเข้าประวัติแชท คิวทั่วไปสำหรับสร้างคำตอบของแชทบอท สรุปบทสนทนา จัดทำดัชนีเอกสารความรู้ และส่งบรอดแคสต์ และคิวสถานะสำหรับส่งสถานะกำลังพิมพ์ระหว่างรอแชทบอท ส่วนตัวจัดตารางเวลาทำงานทุก 1 นาที เพื่อตรวจหาแชทที่ค้างตอบนานเกินกำหนด และส่งบรอดแคสต์ที่ถึงเวลาที่ตั้งไว้

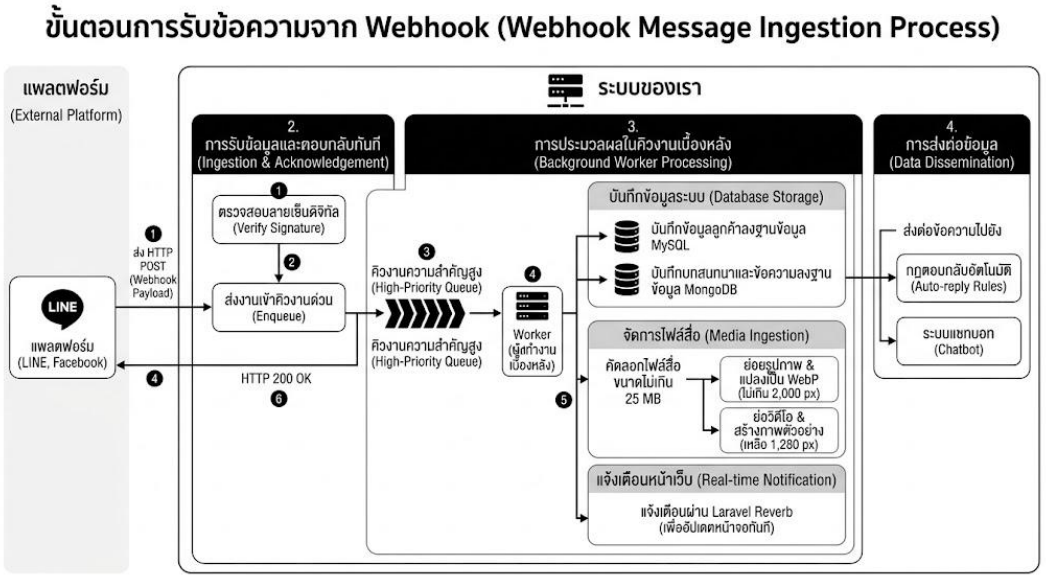
ข้อมูลของระบบจัดเป็นลำดับชั้น ร้านค้าหนึ่งแห่งมีสมาชิกในทีมได้หลายคนและมีได้หลายธุรกิจ แต่ละธุรกิจมีช่องทางที่เชื่อมต่อ ข้อมูลแบรนด์ ข้อมูลการชำระเงิน การตั้งค่าแชทบอทและกฎตอบกลับอัตโนมัติของตนเอง ทุกตารางและทุกเอกสารเก็บรหัสร้านค้าไว้ และระบบกรองข้อมูลตามร้านค้าของผู้ใช้ทุกครั้งที่อ่านหรือเขียน ภาพรวมการจัดเก็บข้อมูลแสดงดังรูปที่ 3.2



รูปที่ 3.2 ภาพรวมการจัดเก็บข้อมูลของระบบเว็บ Chatshop

จากรูปที่ 3.2 ระบบแบ่งการจัดเก็บข้อมูลออกเป็น 2 ส่วน ส่วนแรกคือ MySQL ซึ่งเก็บข้อมูลที่มีความสัมพันธ์กัน ได้แก่ ร้านค้า สมาชิกและบทบาท ช่องทางที่เชื่อมต่อ ลูกค้า สินค้า คำสั่งซื้อ บรอดแคสต์ กฎตอบกลับอัตโนมัติ และการตั้งค่าแชทบอท ส่วนที่สองคือ MongoDB ซึ่งเก็บข้อมูลที่มีปริมาณมากและเพิ่มขึ้นตลอดเวลา ได้แก่ บทสนทนา ข้อความ และเอกสารความรู้ของแชทบอทพร้อมเวกเตอร์ ข้อมูลทุกรายการผูกกับร้านค้าเจ้าของข้อมูล ระบบจึงแยกข้อมูลของแต่ละร้านออกจากกันได้อย่างชัดเจน แม้ทุกร้านจะใช้ฐานข้อมูลชุดเดียวกัน

3.3.2 ขั้นตอนการรับข้อความ ข้อความของลูกค้าจาก LINE, Facebook และ Instagram เข้าสู่ระบบผ่าน Webhook โดย LINE แนบลายเซ็น X-Line-Signature ที่สร้างจาก Channel Secret ส่วน Facebook และ Instagram แนบลายเซ็น X-Hub-Signature-256 ที่สร้างจาก App Secret ระบบตรวจลายเซ็นด้วย HMAC-SHA256 ทุกครั้ง ขั้นตอนการรับข้อความแสดงดังรูปที่ 3.3



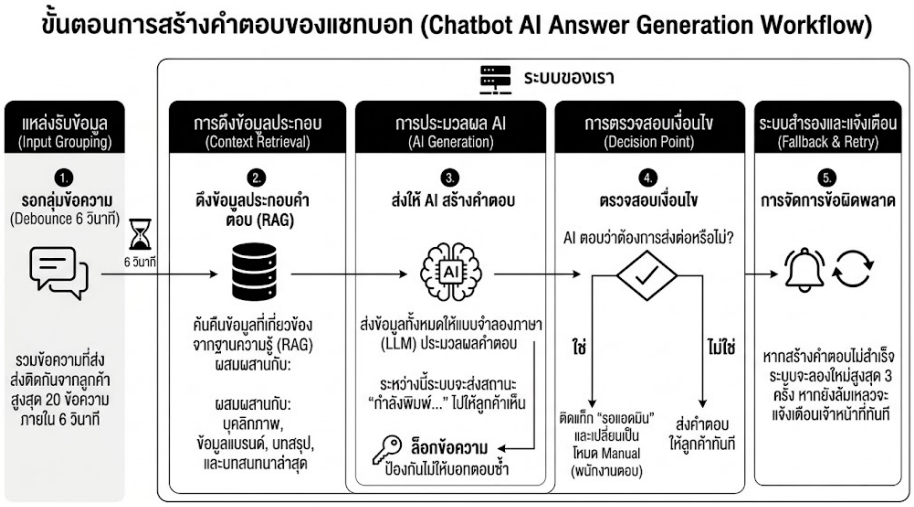
รูปที่ 3.3 ขั้นตอนการรับข้อความจาก Webhook

จากรูปที่ 3.3 แพลตฟอร์มส่ง Webhook มายังระบบ ระบบตรวจลายเซ็นแล้วส่งงานเข้าคิวความสำคัญสูง และตอบกลับแพลตฟอร์มทันที จากนั้นงานในคิวจะบันทึกข้อมูลลูกค้าลง MySQL บันทึกบทสนทนาและข้อความลง MongoDB คัดลอกไฟล์สื่อมาเก็บในระบบ แจ้งหน้าเว็บผ่าน Laravel Reverb และส่งข้อความต่อให้กฎตอบกลับอัตโนมัติและแชทบอท

ข้อความขาเข้ารองรับประเภทข้อความ รูปภาพ วิดีโอ เสียง ไฟล์ และสติกเกอร์ ไฟล์สื่อที่มีขนาดไม่เกิน 25 MB จะถูกคัดลอกมาเก็บในระบบ เพื่อให้เปิดดูได้แม้ลิงก์เดิม

ของแพลตฟอร์มหมดอายุ รูปภาพถูกย่อให้ด้านยาวไม่เกิน 2,000 พิกเซลและแปลงเป็น WebP ส่วนวิดีโอถูกย่อให้ด้านยาวเหลือ 1,280 พิกเซลพร้อมสร้างภาพตัวอย่าง สำหรับรูปภาพจาก LINE ซึ่งไม่มี URL สาธารณะ ระบบส่งรูปผ่านเครื่องแม่ข่ายด้วย URL ที่มีลายเซ็นกำกับ

3.3.3 ขั้นตอนการทำงานของแชทบอท ระบบส่งงานของแชทบอทเข้าคิวทุกครั้งที่มีข้อความใหม่ งานนี้ใช้โหมดที่เจ้าหน้าที่กำหนดให้บทสนทนา หรือโหมดเริ่มต้นของธุรกิจ และสร้างคำตอบส่งให้ลูกค้าเองเฉพาะโหมด Autopilot ส่วนโหมด Suggest เจ้าหน้าที่กดปุ่ม AI เพื่อขอคำตอบแนะนำเอง ขั้นตอนการสร้างคำตอบแสดงดังรูปที่ 3.4



รูปที่ 3.4 ขั้นตอนการสร้างคำตอบของแชทบอท

จากรูปที่ 3.4 ระบบรอ 6 วินาทีเพื่อรวมข้อความที่ลูกค้าส่งติดกันได้สูงสุด 20 ข้อความ แล้วค้นคืนส่วนย่อยของเอกสารความรู้ที่เกี่ยวข้อง ประกอบข้อความสั่งงานจากบุคลิกของร้าน ข้อมูลแบรนด์ บทสรุป และข้อความล่าสุดของบทสนทนา ส่งให้แบบจำลองภาษาสร้างคำตอบ หากแบบจำลองตอบว่าต้องส่งต่อ ระบบจะติดแท็กรอแอดมินและเปลี่ยนเป็นโหมด Manual ระหว่างที่แชทบอทสร้างคำตอบ ระบบส่งสถานะกำลังพิมพ์ให้ลูกค้าเห็น หากสร้างคำตอบไม่สำเร็จ ระบบจะลองใหม่ไม่เกิน 3 ครั้งแล้วแจ้งเตือนเจ้าหน้าที่ และใช้การล็อกป้องกันไม่ให้แชทบอทตอบข้อความเดียวกันซ้ำ

การทำงานของแชทบอทมีรายละเอียด ดังนี้

3.3.3.1 โหมดการทำงาน ร้านค้ากำหนดโหมดเริ่มต้นของธุรกิจ และเจ้าหน้าที่เปลี่ยนโหมดเฉพาะบทสนทนาได้จากสวิตช์ในแผงข้อมูลลูกค้า โหมดของธุรกิจทำหน้าที่เป็นสวิตช์หลัก

หากธุรกิจปิดแชทบอท ทุกบทสนทนาจะทำงานแบบ Manual แม้จะตั้งโหมดอื่นไว้รายบทสนทนา รายละเอียดของแต่ละโหมดแสดงดังตารางที่ 3.1

ตารางที่ 3.1 โหมดการทำงานของแชทบอท

โหมด	ค่าในระบบ	การทำงาน
Manual	manual	แชทบอทไม่ทำงาน เจ้าหน้าที่ตอบเองทั้งหมด
Suggest	suggest	แชทบอทร่างคำตอบให้เจ้าหน้าที่ตรวจและกดส่งเอง
Autopilot	autopilot	แชทบอทส่งคำตอบให้ลูกค้าทันที และส่งต่อเจ้าหน้าที่เมื่อตอบไม่ได้

3.3.3.2 การค้นคืนเอกสารความรู้ ระบบตัดเอกสารความรู้ของร้านเป็นส่วนย่อย แล้วสร้างเวกเตอร์เก็บไว้ เมื่อลูกค้าถาม ระบบสร้างเวกเตอร์ของคำถาม แล้วเลือกส่วนย่อยที่ใกล้เคียงกับคำถามมากที่สุดไปใส่ในข้อความสั่งงาน เพื่อให้แชทบอทตอบจากข้อมูลของร้านค้าเอง

3.3.3.3 คำสั่งพิเศษ ร้านค้าเพิ่มกฎที่แชทบอทต้องทำตามเสมอ หรือประกาศชั่วคราวที่มีวันหมดอายุ เช่น วันหยุดร้านหรือโปรโมชันประจำเดือน ระบบใส่คำสั่งที่ยังไม่หมดอายุไว้ในข้อความสั่งงานทุกครั้ง โดยเรียงกฎไว้ก่อนประกาศชั่วคราว

3.3.3.4 ข้อความที่แชทบอทไม่ตอบ ระบบข้ามข้อความที่เป็นรหัสตัวเลือกหรือตรงกับรูปแบบที่ร้านค้ากำหนด เพื่อไม่ให้แชทบอทตอบข้อความที่ลูกค้าส่งตอบเมนูของร้าน

3.3.3.5 ไฟล์สื่อจากลูกค้า เมื่อลูกค้าส่งรูปภาพ เสียง วิดีโอ หรือไฟล์ แชทบอทจะส่งข้อความรับทราบอัตโนมัติแล้วส่งต่อให้เจ้าหน้าที่ หากเป็นรูปภาพที่ส่งมาภายใน 24 ชั่วโมงหลังมีข้อความเกี่ยวกับการชำระเงิน ระบบจะถือว่าเป็นสลิปโอนเงินเพื่อให้เจ้าหน้าที่ตรวจสอบ

3.3.3.6 การสรุปบทสนทนา เมื่อบทสนทนายาวขึ้น ระบบสรุปข้อความเก่าเป็นบทสรุปแบบสะสม และเก็บข้อความล่าสุด 10 ข้อความไว้แบบเต็ม แชทบอทจึงเข้าใจบริบทของบทสนทนาทั้งหมดโดยไม่ต้องส่งข้อความทุกข้อความให้แบบจำลองภาษา ช่วยลดขนาดของข้อความสั่งงานและเวลาที่ใช้สร้างคำตอบ

3.3.4 การล็อกแชท ระบบป้องกันไม่ให้เจ้าหน้าที่หลายคนตอบลูกค้ารายเดียวกันซ้อนกัน โดยให้ผู้ที่ไม่ได้รับมอบหมายต้องกดยินยอมก่อนตอบ แทนการห้ามเปิดอ่านแชท ร้านค้าเปิดหรือปิดการล็อกแชทได้แยกตามธุรกิจ และค่าเริ่มต้นคือปิด การทำงานมีรายละเอียด ดังนี้

3.3.4.1 เงื่อนไขการล็อก แชทจะถูกล็อกสำหรับเจ้าหน้าที่คนหนึ่งเมื่อครบ 4 เงื่อนไข ได้แก่ ธุรกิจของแชทเปิดการล็อกแชทไว้ แชทมีผู้รับผิดชอบแล้ว ผู้รับผิดชอบไม่ใช่เจ้าหน้าที่คนนั้น และบทบาทของเจ้าหน้าที่ไม่มีสิทธิ์ตอบแชทของคนอื่นโดยไม่ต้องยินยอม บทบาท Owner มีสิทธิ์นี้เสมอ และเจ้าของร้านเพิ่มสิทธิ์นี้ให้บทบาทอื่นได้ แชทที่ยังไม่มีผู้รับผิดชอบจึงไม่ถูกล็อก

3.3.4.2 การมอบหมายผู้รับผิดชอบ แชทมีผู้รับผิดชอบได้ 2 วิธี คือ ผู้ที่มีสิทธิ์มอบหมายแชทเลือกเจ้าหน้าที่จากแผงข้อมูลลูกค้า หรือระบบมอบหมายให้อัตโนมัติเมื่อเจ้าหน้าที่คนแรกตอบแชทที่ยังไม่มีผู้รับผิดชอบ ระบบไม่บันทึกสถานะล็อกไว้ในฐานข้อมูล แต่คำนวณใหม่ทุกครั้งจากผู้รับผิดชอบ เวลาที่มอบหมาย และบันทึกกิจกรรม

3.3.4.3 การยินยอมก่อนตอบ เมื่อเจ้าหน้าที่เปิดแชทที่ถูกล็อก หน้าเว็บแสดงหน้าต่างแจ้งว่าแชทนี้มอบหมายให้คนอื่นแล้ว พร้อมปุ่ม “ยินยอมและตอบแชท” หากปิดหน้าต่างช่องพิมพ์จะถูกแทนด้วยแถบแจ้งเตือนและปุ่ม “ยินยอมเพื่อตอบ” การกดยินยอมจะบันทึกลงบันทึกกิจกรรมของร้านในชื่อ “ขอเข้าตอบ” โดยไม่เปลี่ยนผู้รับผิดชอบเดิม

3.3.4.4 การตรวจสอบฝั่งเซิร์ฟเวอร์ ทุกครั้งที่ส่งข้อความตอบกลับ ระบบตรวจว่าเจ้าหน้าที่ยินยอมหลังการมอบหมายครั้งล่าสุดแล้วหรือไม่ หากยังไม่ยินยอม ระบบจะปฏิเสธการส่งและหน้าเว็บแสดงหน้าต่างยินยอมอีกครั้งโดยเก็บข้อความที่พิมพ์ไว้ การยินยอมจึงใช้ได้เฉพาะเจ้าหน้าที่คนนั้นและการมอบหมายครั้งปัจจุบัน เมื่อแชทถูกมอบหมายใหม่ต้องยินยอมอีกครั้ง

3.3.4.5 การปลดล็อก แชทจะไม่ถูกล็อกเมื่อยกเลิกการมอบหมาย หรือเมื่อร้านค้าปิดการล็อกแชท เมื่อเปลี่ยนการตั้งค่า ระบบแจ้งหน้าเว็บที่เปิดอยู่ผ่าน Laravel Reverb ให้ตรวจสถานะการล็อกใหม่ทันที ส่วนการเก็บถาวรแชทไม่ได้ยกเลิกการมอบหมาย

3.3.5 การตรวจสลิปอัตโนมัติ เมื่อลูกค้าส่งรูปสลิปโอนเงินในแชท ระบบตรวจสลิปกับบริการ check-slip และเปลี่ยนสถานะคำสั่งซื้อเป็นชำระแล้วเมื่อสลิปถูกต้อง ร้านค้าเปิดใช้ได้แยกตามธุรกิจ โดยต้องตั้งค่าบัญชี check-slip ก่อน ขั้นตอนการทำงานมีดังนี้

3.3.5.1 การคัดแยกรูปภาพ เมื่อได้รับรูปภาพจาก LINE, Facebook หรือ Instagram ระบบส่งงานตรวจสลิปเข้าคิวงาน งานนี้ดาวน์โหลดรูปแล้วอ่าน QR บนเซิร์ฟเวอร์ของระบบเอง โดยลองอ่านทั้งรูปและส่วนย่อยของรูป ระบบถือว่าเป็นสลิปเฉพาะเมื่อ QR อยู่ในรูปแบบ QR ของสลิปตามมาตรฐานธนาคารแห่งประเทศไทย รูปทั่วไปและ QR พร้อมเพย์สำหรับรับเงินจึงไม่ถูกส่งตรวจและไม่เสียเครดิตของบริการตรวจสลิป

3.3.5.2 การตรวจสลิปซ้ำ ก่อนส่งตรวจ ระบบค้นว่า QR นี้เคยผ่านการตรวจในร้านเดียวกันแล้วหรือไม่ หากเคยผ่านแล้ว ระบบบันทึกว่าเป็นสลิปซ้ำโดยไม่เรียกบริการภายนอก

3.3.5.3 การจับคู่คำสั่งซื้อ ระบบค้นคำสั่งซื้อที่รอชำระของแชทนั้น หากพบเพียงรายการเดียวจึงส่งตรวจ หากไม่พบหรือพบหลายรายการ ระบบแจ้งเตือนเจ้าหน้าที่ให้ตรวจสอบเอง

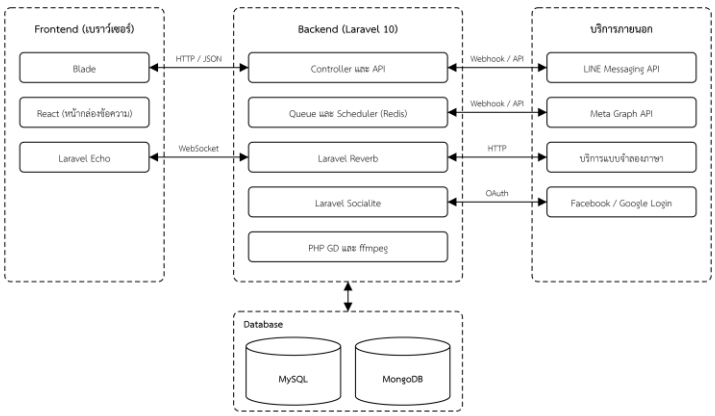
3.3.5.4 การส่งตรวจ ระบบส่งข้อมูล QR รหัสร้านค้า เลขอ้างอิงของการตรวจ ยอดรวมของคำสั่งซื้อ และวันที่ของคำสั่งซื้อไปยัง check-slip โดยไม่ส่งรูปภาพ check-slip ตรวจยอดเงิน วันที่โอน บัญชีผู้รับ และการใช้สลิปซ้ำ แล้วส่งผลกลับมาภายหลังผ่าน Webhook วันที่ที่ส่งไปจะย้อนหลังจากวันที่สร้างคำสั่งซื้อ 7 วัน เพราะลูกค้ามักโอนเงินก่อนเจ้าหน้าที่เปิดคำสั่งซื้อ งานนี้ทำเพียงครั้งเดียวโดยไม่ลองซ้ำ เพื่อไม่ให้เสียเครดิตซ้ำ

3.3.5.5 การรับผลตรวจ Webhook ของ check-slip แนบลายเซ็นที่สร้างด้วย HMAC-SHA256 จาก Webhook secret ของร้าน ระบบตรวจลายเซ็นก่อนรับผล หากไม่ถูกต้องจะปฏิเสธ เมื่อสลิปผ่าน ระบบบันทึกยอดที่โอนและเวลาโอน เปลี่ยนคำสั่งซื้อเป็นชำระแล้ว และส่งการแจ้งเตือนให้ร้านค้า หากไม่ผ่าน คำสั่งซื้อยังคงรอชำระ และระบบบันทึกส่วนต่างระหว่างยอดที่โอนกับยอดคำสั่งซื้อไว้ให้เจ้าหน้าที่ใช้ตรวจสอบกับลูกค้าต่อ

3.3.5.6 การแจ้งผล ระบบแจ้งผลให้เจ้าหน้าที่ผ่าน Laravel Reverb เป็นข้อความแจ้งเตือนในหน้ากล่องแชท ได้แก่ สลิปถูกต้อง สลิปไม่ผ่านพร้อมยอดต่าง สลิปซ้ำ และตรวจไม่สำเร็จ ซึ่งคลิกเพื่อเปิดแชทได้ ระบบไม่แจ้งผลการตรวจให้ลูกค้าโดยอัตโนมัติ เจ้าหน้าที่ต้องแจ้งลูกค้าเอง

3.4 โครงสร้างระบบเว็บ Chatshop

ระบบเว็บ Chatshop ประกอบด้วยหน้าเว็บที่ทำงานบนเบราว์เซอร์ของผู้ใช้ เครื่องแม่ข่ายที่รัน Laravel พร้อมคิวงานและเซิร์ฟเวอร์ WebSocket ฐานข้อมูล 2 ระบบ และบริการภายนอกที่ระบบเชื่อมต่อ หน้าเว็บสื่อสารกับเครื่องแม่ข่ายทั้งผ่านคำขอ HTTP และการเชื่อมต่อ WebSocket ที่เปิดค้างไว้ ส่วนบริการภายนอกสื่อสารกับระบบผ่าน API และ Webhook ความสัมพันธ์ของทุกส่วนแสดงดังรูปที่ 3.5



รูปที่ 3.5 สถาปัตยกรรมของระบบเว็บ Chatshop

จากรูปที่ 3.5 โครงสร้างของระบบเว็บ Chatshop แบ่งออกเป็นส่วนต่าง ๆ ดังนี้

3.4.1 ส่วนหน้าบ้านหรือ Frontend ประกอบด้วยเครื่องมือ ดังนี้

3.4.1.1 Blade ใช้สร้างหน้าเข้าสู่ระบบ หน้าตั้งค่า และหน้าจัดการข้อมูลทั่วไป

3.4.1.2 React ใช้สร้างหน้ากล่องข้อความ โหลดจาก CDN และแปลงไฟล์ JSX ด้วย Babel บนเบราว์เซอร์ จึงแก้ไขหน้ากล่องข้อความได้โดยไม่ต้องผ่านขั้นตอน build

3.4.1.3 Laravel Echo ใช้รับข้อความใหม่ผ่าน WebSocket โดยสมัครรับช่องส่วนตัวของร้านค้าซึ่งต้องผ่านการตรวจสิทธิ์ก่อน

3.4.2 ส่วนหลังบ้านหรือ Backend พัฒนาด้วย Laravel 10 ประกอบด้วยเครื่องมือ ดังนี้

3.4.2.1 Queue และ Scheduler ใช้ประมวลผลงานเบื้องหลัง ได้แก่ Webhook แชทบอท และบรอดแคสต์ โดยเก็บคิวไว้ใน Redis และตรวจแชทที่ค้างตอบทุก 1 นาที

3.4.2.2 Laravel Reverb ใช้เป็นเซิร์ฟเวอร์ WebSocket

3.4.2.3 Laravel Socialite ใช้เข้าสู่ระบบด้วยบัญชี Facebook และ Google

3.4.2.4 PHP GD และ ffmpeg ใช้ย่อขนาดรูปภาพและวิดีโอก่อนจัดเก็บ

3.4.3 ส่วนฐานข้อมูลหรือ Database ใช้ MySQL เก็บข้อมูลเชิงสัมพันธ์ และใช้ MongoDB ผ่านไลบรารี laravel-mongodb เก็บบทสนทนา ข้อความ และฐานความรู้

3.4.4 ส่วนการเชื่อมต่อบริการภายนอก ประกอบด้วยบริการ ดังนี้

3.4.4.1 LINE Messaging API ใช้รับส่งข้อความกับบัญชี LINE ของร้าน

3.4.4.2 Meta Graph API ใช้รับและส่งข้อความกับ Facebook Messenger และ Instagram และนำเข้าประวัติแชทย้อนหลังของเพจได้ไม่เกิน 90 วัน

3.4.4.3 บริการแบบจำลองภาษา ได้แก่ Ollama บนเครื่องแม่ข่ายของบริษัท ซึ่งมีแบบจำลองสำหรับสร้างคำตอบและแบบจำลองสำหรับสร้าง Embedding รวมทั้งผู้ให้บริการที่รองรับรูปแบบ OpenAI API ซึ่งร้านค้าเพิ่มเองได้จากหน้าตั้งค่าแชทบอท

3.4.4.4 บริการตรวจสลิป check-slip ใช้ตรวจความถูกต้องของสลิปโอนเงินจาก QR บนสลิป และส่งผลการตรวจกลับมาผ่าน Webhook

3.5 การออกแบบ API

ระบบเว็บแบ่ง API ออกเป็น 3 กลุ่ม ได้แก่ Webhook ที่แพลตฟอร์มแชทเรียกเข้ามา API ที่หน้าเว็บและหน้ากล่องข้อความเรียกใช้ และ API ของ WebChat widget ทุกกลุ่มรับส่งข้อมูลในรูปแบบ JSON โดย Webhook ยืนยันแหล่งที่มาด้วยลายเซ็นของแพลตฟอร์ม ส่วน API ของหน้าเว็บยืนยันตัวตนด้วย Session ของผู้ใช้ที่เข้าสู่ระบบ ป้องกันการปลอมคำขอด้วย CSRF token และตรวจสิทธิ์ตามบทบาทก่อนทำงาน แต่ละกลุ่มมีรายละเอียด ดังนี้

3.5.1 กลุ่ม Webhook ใช้รับเหตุการณ์จาก LINE, Facebook และ Instagram เช่น ข้อความใหม่จากลูกค้า ระบบตรวจลายเซ็นของแพลตฟอร์มก่อนรับข้อมูลทุกครั้ง แล้วส่งงานเข้าคิวเพื่อประมวลผลในเบื้องหลัง และตอบกลับแพลตฟอร์มทันที LINE ใช้ URL แยกตามช่องทางที่เชื่อมต่อ ส่วน Facebook และ Instagram ใช้เส้นทางเดียวกัน ซึ่งรับคำขอ GET สำหรับยืนยัน verify token ตอนลงทะเบียน และคำขอ POST สำหรับรับเหตุการณ์

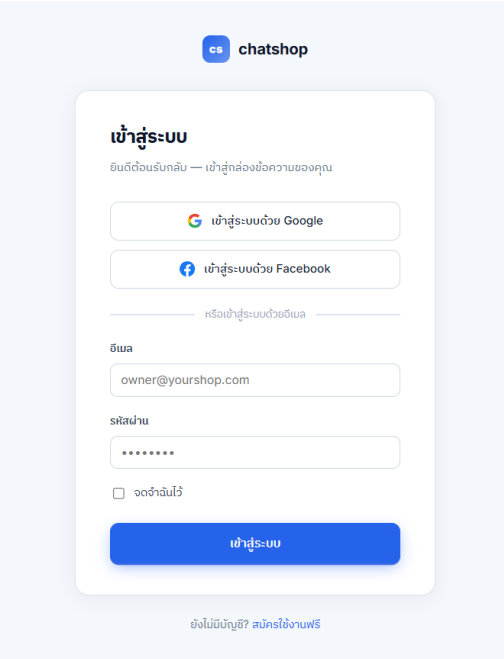
3.5.2 กลุ่ม API ของหน้าเว็บ ใช้กับหน้าเว็บและหน้ากล่องข้อความ ครอบคลุมการยืนยันตัวตน การดูและตอบบทสนทนา การจัดการลูกค้า สินค้า และคำสั่งซื้อ การจัดการทีม และการตั้งค่าแชทบอท ผู้ใช้ต้องเข้าสู่ระบบและมีสิทธิ์ตามบทบาทจึงจะเรียกใช้ได้ ตัวอย่างเช่น การโหลดรายการบทสนทนาคืนข้อมูลสูงสุด 200 รายการ โดยเรียงบทสนทนาที่ปักหมุดไว้ก่อน แล้วตามด้วยบทสนทนาที่มีข้อความล่าสุด ส่วนข้อความในบทสนทนาโหลดครั้งละ 30 ข้อความ เมื่อเลื่อนขึ้นไปดูข้อความเก่า ระบบจึงโหลดชุดถัดไป หน้ากล่องข้อความจึงเปิดได้เร็วแม้บทสนทนาจะมีข้อความจำนวนมาก

3.5.3 กลุ่ม API ของ WebChat widget ใช้กับหน้าต่างแชทที่ร้านค้านำไปติดบนเว็บไซต์ของร้าน ให้ผู้เยี่ยมชมเริ่มบทสนทนา ส่งข้อความ และดูประวัติข้อความได้โดยไม่ต้องเข้าสู่ระบบ ส่วนข้อความใหม่จากร้านค้าส่งไปยังหน้าต่างแชทผ่าน WebSocket ของ Laravel Reverb

3.6 การพัฒนาเว็บแอปพลิเคชัน

ผู้จัดทำพัฒนาหน้าเว็บตามความต้องการในหัวข้อ 3.2 เมื่อเข้าสู่ระบบแล้ว ทุกหน้ามีแถบเมนูด้านซ้ายสำหรับสลับหน้าจอ ได้แก่ กล่องแชท ออเดอร์ ภาพรวม ทีมงาน และตั้งค่า ตามด้วยสินค้า ลูกค้า และบรอดแคสต์ เมนูแต่ละรายการแสดงเฉพาะเมื่อบทบาทของผู้ใช้มีสิทธิ์ ส่วนแถบด้านบนมีชื่อร้าน ช่องค้นหาแชท ลูกค้า หรือเลขออเดอร์ และกระดิ่งแจ้งเตือน หน้าจอหลักของระบบมีดังนี้

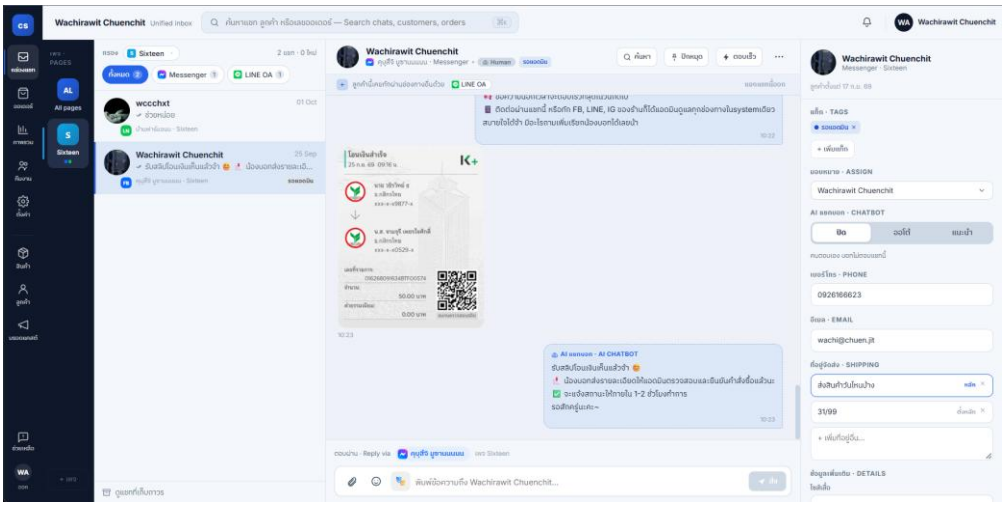
3.6.1 หน้าเข้าสู่ระบบ ผู้ใช้เข้าสู่ระบบด้วยบัญชี Google, Facebook หรืออีเมล ร้านค้าที่ยังไม่มีบัญชีสมัครใช้งานได้เองโดยกรอกชื่อ อีเมล และรหัสผ่านอย่างน้อย 8 ตัวอักษร อีเมลต้องไม่ซ้ำกับบัญชีที่มีอยู่ และระบบเก็บรหัสผ่านแบบแฮชแทนรหัสผ่านจริง เมื่อสมัครสำเร็จ ระบบสร้างร้านค้าใหม่ กำหนดให้ผู้สมัครเป็นเจ้าของร้าน แล้วพาเข้าสู่ระบบทันที ส่วนการเข้าสู่ระบบด้วย Google หรือ Facebook ใช้อีเมลของบัญชีนั้นค้นหาผู้ใช้เดิม หากไม่พบจะสร้างบัญชีและร้านค้าใหม่ให้เช่นเดียวกัน สมาชิกที่ได้รับคำเชิญทางอีเมลจะเข้าร่วมร้านค้าเดิมตามบทบาทที่เจ้าของร้านกำหนด



รูปที่ 3.6 หน้าเข้าสู่ระบบ

จากรูปที่ 3.6 กลางหน้าจอเป็นการ์ดเข้าสู่ระบบ ด้านบนมีปุ่ม “เข้าสู่ระบบด้วย Google” และ “เข้าสู่ระบบด้วย Facebook” ถัดลงมาเป็นช่องกรอกอีเมลและรหัสผ่าน ช่องเลือก “จดจำฉันไว้” และปุ่ม “เข้าสู่ระบบ” ใต้การ์ดมีลิงก์ “สมัครใช้งานฟรี” สำหรับร้านค้าที่ยังไม่มีบัญชี

3.6.2 หน้ากล่องแชท เป็นหน้าหลักที่เจ้าหน้าที่ใช้ตอบลูกค้าจากทุกช่องทาง

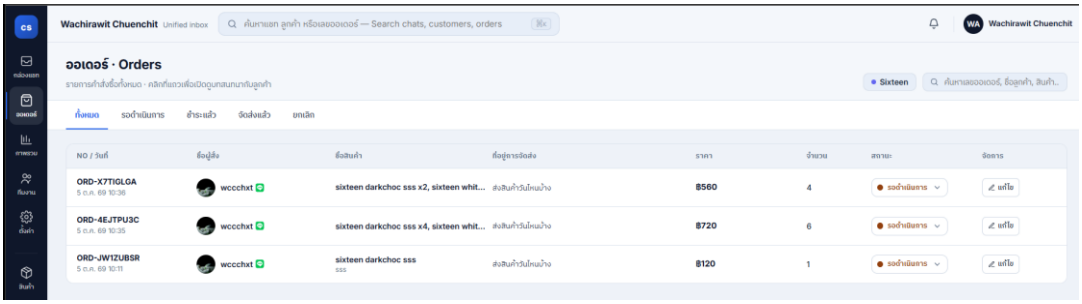


รูปที่ 3.7 หน้ากล่องแชท

จากรูปที่ 3.7 หน้าจอแบ่งเป็น 4 ส่วนจากซ้ายไปขวา ส่วนแรกคือรายการเพจที่เชื่อมต่อ แสดงรูปและชื่อเพจพร้อมจุดสีบอกแพลตฟอร์ม และปุ่ม “+ เพจ” สำหรับเชื่อมต่อเพจเพิ่ม ส่วนที่สองคือรายการบทสนทนา ด้านบนมีปุ่มกรองตามแพลตฟอร์มพร้อมจำนวนแชท แต่ละแถวแสดงรูปและชื่อลูกค้า เวลา ข้อความล่าสุด จำนวนข้อความที่ยังไม่อ่าน และแท็ก ด้านล่างมีปุ่ม “ดูแชทที่เก็บถาวร”

ส่วนที่สามคือหน้าแชท หัวแชทแสดงชื่อลูกค้า ช่องทาง โหมดของแชทบอท และแท็ก พร้อมปุ่ม “ค้นหา” “ปักหมุด” และ “ตอบเร็ว” สำหรับเลือกข้อความตอบสำเร็จรูป เมื่อนำเมาส์ไปวางบนข้อความ เจ้าหน้าที่ตอบกลับอ้างอิง ปักหมุด หรือใช้ข้อความนั้นเป็นที่อยู่จัดส่งได้ ด้านล่างเป็นช่องพิมพ์ข้อความพร้อมปุ่มแนบไฟล์ อีโมจิ สติกเกอร์ ปุ่ม “AI” สำหรับขอคำตอบแนะนำจากแชทบอท และปุ่ม “ส่ง” ส่วนที่สี่คือแผงข้อมูลลูกค้า ประกอบด้วยแท็ก ผู้รับผิดชอบแชท สวิตช์โหมดแชทบอท “ปิด” “ออโต้” และ “แนะนำ” เบอร์โทร อีเมล ที่อยู่จัดส่ง โน้ตภายใน และออเดอร์ล่าสุดของลูกค้าพร้อมปุ่ม “+ สร้างออเดอร์” ซึ่งเปิดหน้าต่างให้ค้นหาสินค้าจากแค็ตตาล็อก กรอกราคา จำนวน และที่อยู่จัดส่ง แล้วแสดงยอดรวมก่อนบันทึก

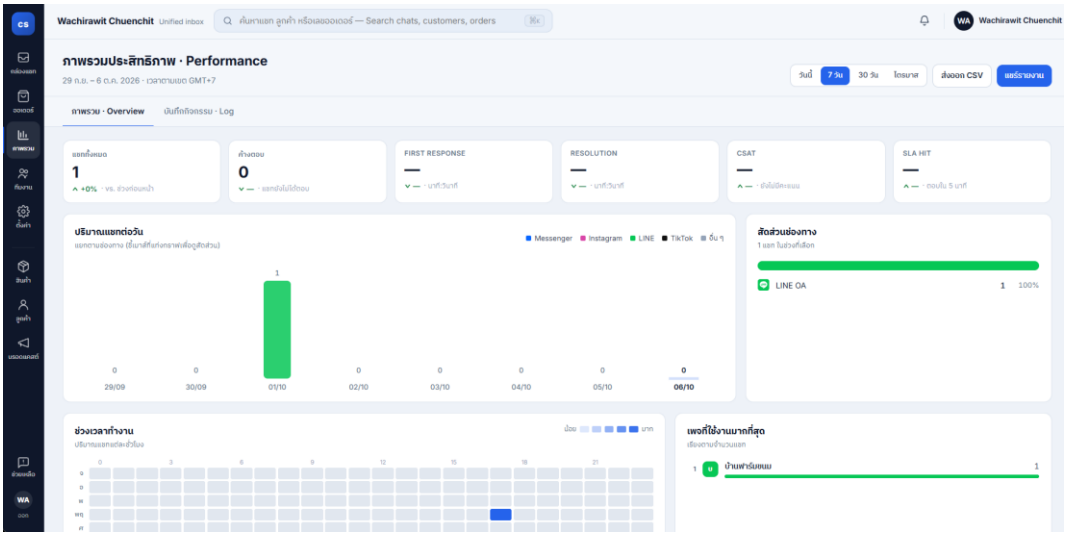
3.6.3 หน้าออเดอร์ แสดงคำสั่งซื้อทั้งหมดของร้าน



รูปที่ 3.8 หน้าออเดอร์

จากรูปที่ 3.8 ด้านบนมีช่องค้นหาตามเลขออเดอร์ ชื่อลูกค้า หรือสินค้า และแท็บสถานะ 5 แท็บ ได้แก่ ทั้งหมด รอดำเนินการ ชำระแล้ว จัดส่งแล้ว และยกเลิก ตารางแสดงเลขและวันที่ ชื่อผู้สั่ง ชื่อสินค้า ที่อยู่การจัดส่ง ราคา จำนวน และสถานะ คอลัมน์สถานะเป็นรายการเลือกที่มีสีต่างกัน เจ้าหน้าที่เปลี่ยนสถานะได้ทันทีจากตาราง และเมื่อคลิกที่แถวระบบจะเปิดบทสนทนากับลูกค้ารายนั้น

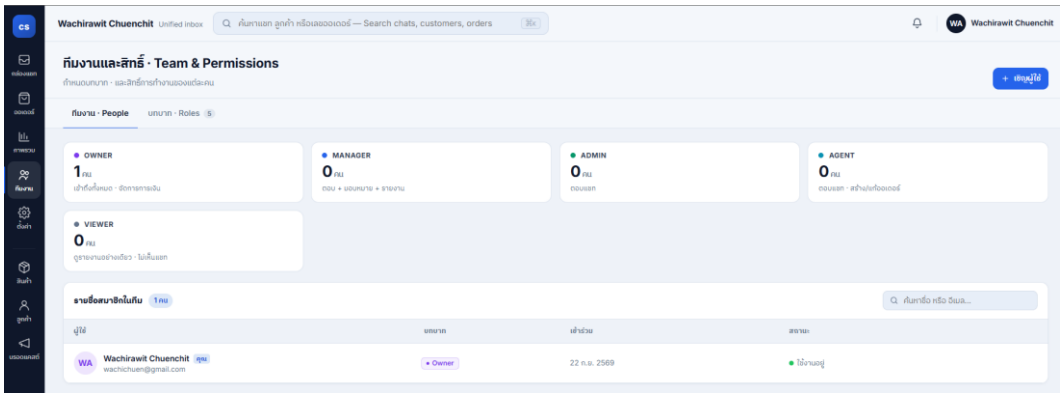
3.6.4 หน้าภาพรวม แสดงรายงานสรุปผลการให้บริการ



รูปที่ 3.9 หน้าภาพรวม

จากรูปที่ 3.9 มุมขวาบนเลือกช่วงเวลาได้ 4 แบบ ได้แก่ วันนี้ 7 วัน 30 วัน และไตรมาส พร้อมปุ่ม “ส่งออก CSV” ซึ่งได้ไฟล์ที่เข้ารหัส UTF-8 พร้อม BOM ให้ Excel แสดงภาษาไทยได้ ถัดลงมาเป็นการ์ดตัวชี้วัด เช่น จำนวนแชททั้งหมด แชทที่ค้างตอบ เวลาตอบกลับครั้งแรก เวลาที่ใช้ปิดบทสนทนา และสัดส่วนแชทที่ตอบทันเวลาที่กำหนด โดยแต่ละการ์ดเทียบกับช่วงก่อนหน้า จากนั้นเป็นกราฟแท่งปริมาณแชทแยกตามช่องทาง สัดส่วนช่องทาง Heatmap แสดงปริมาณแชทแต่ละชั่วโมง อันดับเพจที่มีแชทมากที่สุด และตารางผลงานของเจ้าหน้าที่แต่ละคน ส่วนแท็บ “บันทึกกิจกรรม” แสดงประวัติการตอบกลับและการมอบหมายแชทของทีม กรองตามสมาชิก การกระทำ และวันที่ได้

3.6.5 หน้าทีมงาน ใช้จัดการสมาชิกและบทบาทของร้าน

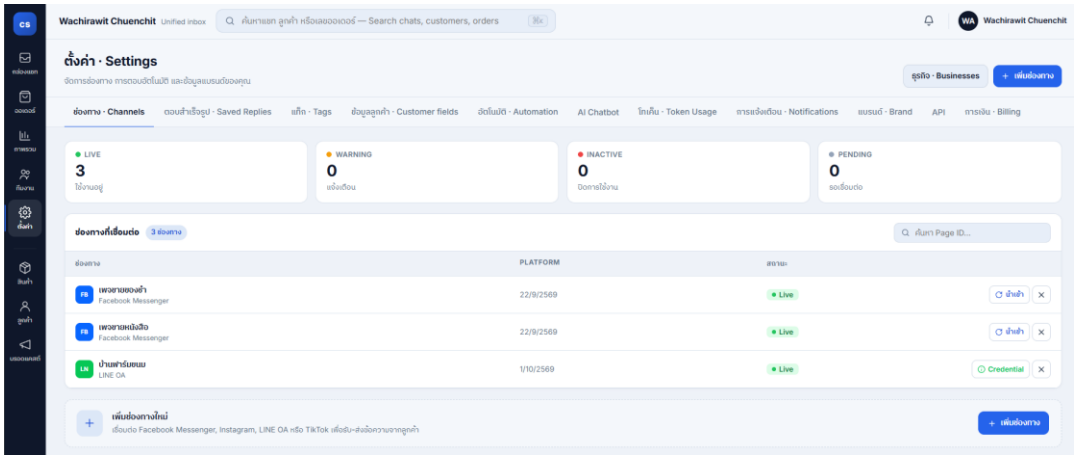


รูปที่ 3.10 หน้าทีมงาน

จากรูปที่ 3.10 หน้าจอมี 2 แท็บ คือ “ทีมงาน” และ “บทบาท” แท็บทีมงานแสดงการ์ดบทบาทพร้อมจำนวนสมาชิก ซึ่งคลิกเพื่อกรองรายชื่อได้ และตารางสมาชิกที่แสดงชื่อ อีเมล บทบาท วันที่เข้าร่วม และสถานะ เมนู “•••” ของแต่ละแถวใช้เปลี่ยนบทบาท โอนสิทธิ์เจ้าของ หรือลบสมาชิกออกจากทีม ปุ่ม “+ เชิญผู้ใช้” เปิดหน้าต่างกรอกอีเมลและเลือกบทบาท โดยคำเชิญหมดอายุใน 7 วัน ส่วนแท็บบทบาทแสดงการ์ดของแต่ละบทบาทและปุ่ม “สร้างบทบาทเอง” ซึ่งเปิดหน้าต่างให้กำหนดชื่อ สี และเลือกสิทธิ์จาก 13 รายการ แบ่งเป็นกลุ่มกล่องแชท ออเดอร์ สินค้า รายงาน และตั้งค่า

3.6.6 หน้าตั้งค่า รวมการตั้งค่าของร้านไว้เป็นแท็บ ด้านบนมีตัวเลือกธุรกิจและปุ่ม “+ เพิ่มช่องทาง” แท็บที่สำคัญมีดังนี้

3.6.6.1 แท็บช่องทาง ใช้เชื่อมต่อและดูสถานะของช่องทางแชท



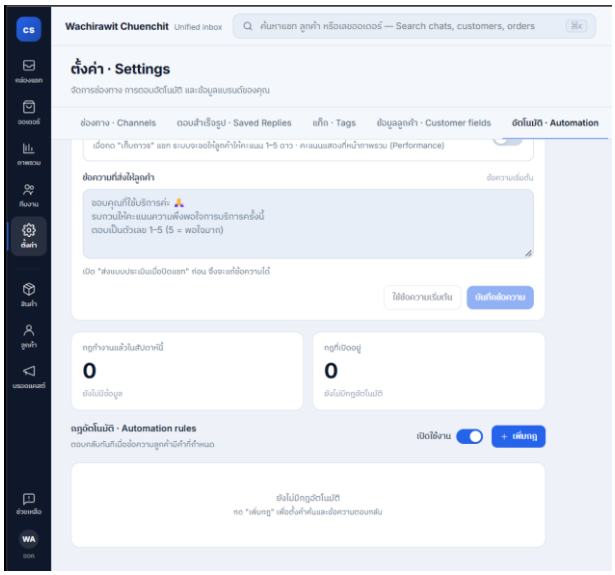
รูปที่ 3.11 แท็บช่องทางในหน้าตั้งค่า

จากรูปที่ 3.11 ด้านบนมีการ์ดสถานะ 4 ใบ ได้แก่ ใช้งานอยู่ แจ้งเตือน ปิดการใช้งาน และรอเชื่อมต่อ ซึ่งคลิกเพื่อกรองตารางได้ ตารางแสดงชื่อเพจหรือบัญชี แพลตฟอร์ม วันที่เชื่อมต่อ และสถานะ แต่ละแถวมีปุ่มตามแพลตฟอร์ม ได้แก่ “นำเข้า” สำหรับ Facebook และ Instagram “Credential” สำหรับดู Webhook URL ของ LINE และ “Embed” สำหรับคัดลอกโค้ดของ WebChat เมื่อกดปุ่ม “+ เพิ่มช่องทาง” ระบบจะแสดงหน้าต่างให้เลือกแพลตฟอร์มที่ต้องการเชื่อมต่อ

3.6.6.2 แท็บอัตโนมัติ ใช้ตั้งกฎตอบกลับอัตโนมัติด้วยคีย์เวิร์ดและการล็อกแชท กฎแต่ละข้อเป็นของธุรกิจหนึ่งธุรกิจ กำหนดคีย์เวิร์ดได้หลายคำ และเลือกวิธีจับคู่ได้ 2 แบบ คือ ตรง

ทั้งข้อความ และมีคำนี้อยู่ในข้อความ เมื่อข้อความของลูกค้าตรงกับกฎ ระบบส่งข้อความตอบกลับผ่าน LINE, Facebook หรือ Instagram ตามช่องทางของบทสนทนา และบันทึกข้อความที่ส่งพร้อมชื่อกฎไว้ในบทสนทนา เจ้าหน้าที่จึงรู้ว่าข้อความใดมาจากกฎใด กฎตอบกลับทำงานในคิวความสำคัญสูงแยกจากแชทบอท จึงตอบลูกค้าได้ทันทีโดยไม่ต้องรอแบบจำลองภาษา และร้านค้าเปิดหรือปิดกฎทั้งหมดของธุรกิจได้ในครั้งเดียว

การล็อกแชทอัตโนมัติช่วยป้องกันไม่ให้เจ้าหน้าที่หลายคนตอบลูกค้ารายเดียวกันซ้อนกัน เมื่อเปิดสวิตช์ “ยินยอมก่อนตอบแชทของคนอื่น” เจ้าหน้าที่ที่ไม่ได้รับมอบหมายต้องกดยินยอมก่อนตอบแชทที่มีผู้รับผิดชอบแล้ว ยกเว้นบทบาทที่มีสิทธิ์ตอบแชทของคนอื่นโดยไม่ต้องยินยอม และการยินยอมทุกครั้งถูกบันทึกในบันทึกกิจกรรมของร้าน ตามที่อธิบายในหัวข้อ 3.3.4 ผู้ใช้ต้องมีสิทธิ์จัดการการตั้งค่าจึงจะเพิ่มหรือแก้ไขกฎได้ แท็บอัตโนมัติแสดงดังรูปที่ 3.12



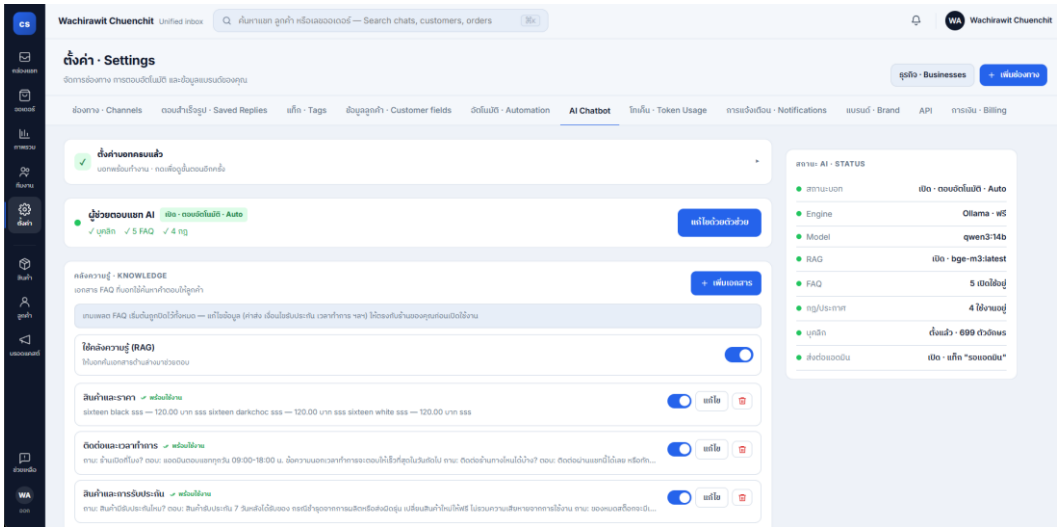
รูปที่ 3.12 แท็บอัตโนมัติในหน้าตั้งค่า

จากรูปที่ 3.12 ด้านบนสุดเป็นสวิตช์ล็อกแชทอัตโนมัติ ถัดมาเป็นการ์ดแสดงจำนวนครั้งที่กฎทำงานในสัปดาห์นี้และจำนวนกฎที่เปิดอยู่ ปุ่ม “+ เพิ่มกฎ” เปิดฟอร์มให้กรอกชื่อกฎ คีย์เวิร์ด เลือกวิธีจับคู่ “ตรงทั้งข้อความ” หรือ “มีคำนี้อยู่ในข้อความ” และข้อความตอบกลับ กฎที่สร้างแล้วแสดงในตารางพร้อมสวิตช์เปิดปิดรายกฎ

ส่วนการตรวจสลิปอัตโนมัติในแท็บเดียวกันมีสวิตช์ “ตรวจสลิปที่ลูกค้าส่งมา” ซึ่งเปิดได้เมื่อตั้งค่าบัญชี check-slip แล้ว เจ้าของร้านกรอก API Token, Merchant code และ Webhook secret แล้วกด “บันทึก API key” ระบบเข้ารหัส API Token และ Webhook secret

ก่อนจัดเก็บ และไม่ส่งค่าเหล่านี้กลับไปแสดงบนหน้าเว็บ การ์ดนี้แสดงสถานะการเชื่อมต่อ Webhook URL สำหรับนำไปกรอกใน check-slip และเครดิตคงเหลือ

3.6.6.3 แท็บ AI Chatbot ใช้ตั้งค่าแชทบอทของธุรกิจ



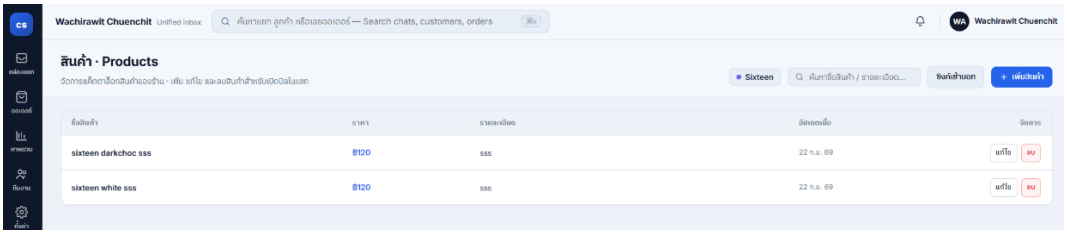
รูปที่ 3.13 แท็บ AI Chatbot ในหน้าตั้งค่า

จากรูปที่ 3.13 ด้านซ้ายบนเป็นการ์ดขั้นตอนเริ่มต้น 3 ขั้น ได้แก่ เปิดโหมดบอท ตั้งบุคลิกและ System Prompt และเพิ่มคลังความรู้ ถัดมาเป็นการ์ดสถานะบอทพร้อมปุ่ม “ตั้งค่าด้วยตัวช่วย” และส่วนคลังความรู้สำหรับเพิ่ม แก้ไข และเปิดปิดเอกสาร ซึ่งแสดงสถานะว่าเอกสารพร้อมใช้งานหรือกำลังประมวลผล ด้านขวาเป็นแผงสรุปสถานะของบอท

ส่วนตั้งค่าขั้นสูงใช้เลือกโหมดของบอท แก้ไข System Prompt ตั้งค่าการส่งต่อแอดมิน และเพิ่มคำสั่งพิเศษแบบกฎถาวรหรือประกาศชั่วคราว รวมทั้งเลือกผู้ให้บริการ AI ระหว่าง AI ของระบบซึ่งใช้ Ollama กับผู้ให้บริการของร้านเอง ฟอร์มผู้ให้บริการมีตัวเลือกผู้ให้บริการสำเร็จรูป ช่อง Base URL และ API key ปุ่ม “ดึงรายการโมเดล” และปุ่ม “ทดสอบการเชื่อมต่อ” ส่วนปุ่ม “ตั้งค่าด้วยตัวช่วย” เปิดหน้าต่าง 4 ขั้นตอน คือ ข้อมูลร้านและบุคลิก นโยบายของร้าน กฎของบอท และตรวจสอบ ซึ่งขั้นสุดท้ายให้ร้านค้าทดลองคุยกับบอทก่อนบันทึกได้

3.6.6.4 แท็บอื่น ได้แก่ ตอบสำเร็จรูป สำหรับเพิ่มข้อความตอบกลับที่ใช้บ่อยหรือดึงจาก Facebook แท็ก สำหรับกำหนดชื่อและสีของแท็ก การแจ้งเตือน สำหรับเปิดปิดการแจ้งเตือนบนเบราว์เซอร์และเสียง รวมทั้งเลือกช่องทางที่ต้องการรับแจ้งเตือน และแบรนด์ สำหรับกรอกชื่อ ที่อยู่ เบอร์โทร และอีเมลของร้าน

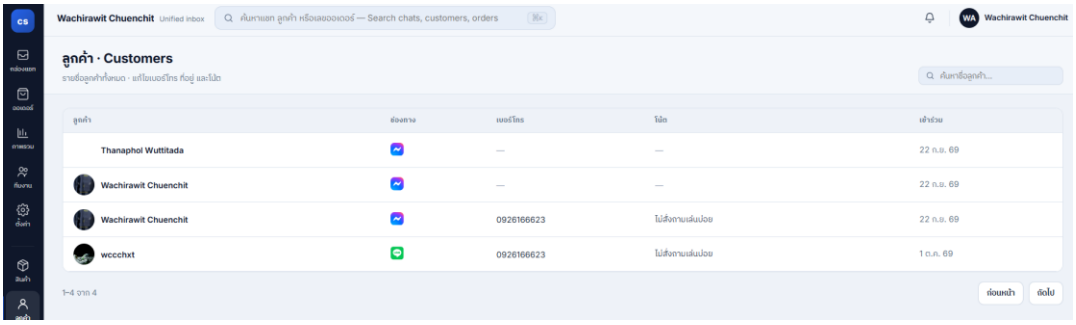
3.6.7 หน้าสินค้า ใช้จัดการแค็ตตาล็อกสินค้าสำหรับเปิดคำสั่งซื้อในแชท



รูปที่ 3.14 หน้าสินค้า

จากรูปที่ 3.14 ตารางแสดงชื่อสินค้า ราคา รายละเอียด และวันที่อัปเดต แต่ละแถวมีปุ่ม “แก้ไข” และ “ลบ” ปุ่ม “+ เพิ่มสินค้า” เปิดหน้าต่างกรอกชื่อ ราคา และรายละเอียด ส่วนปุ่ม “ซิงค์เข้าบอท” นำข้อมูลสินค้าเข้าคลังความรู้ของแชทบอท

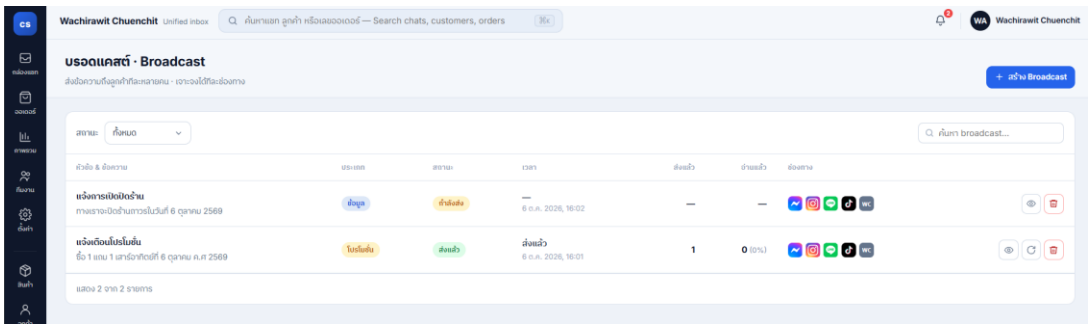
3.6.8 หน้าลูกค้า แสดงรายชื่อลูกค้าจากทุกช่องทาง ระบบสร้างข้อมูลลูกค้าให้อัตโนมัติเมื่อลูกค้าส่งข้อความครั้งแรก โดยเก็บแพลตฟอร์ม รหัสผู้ใช้บนแพลตฟอร์ม ชื่อ และรูปโปรไฟล์ ส่วนเบอร์โทร ที่อยู่ และโน้ตซึ่งยาวได้ไม่เกิน 2,000 ตัวอักษร เจ้าหน้าที่กรอกเพิ่มเองได้ และเมื่อบันทึกที่อยู่จัดส่งในคำสั่งซื้อ ระบบจะปรับที่อยู่ของลูกค้าให้ตรงกันด้วย



รูปที่ 3.15 หน้าลูกค้า

จากรูปที่ 3.15 ตารางแสดงชื่อลูกค้า ช่องทาง เบอร์โทร โน้ต และวันที่เข้าร่วม แบ่งเป็นหน้า เมื่อคลิกที่แถว แผงด้านขวาจะเปิดให้แก้ไขเบอร์โทร ที่อยู่ และโน้ตของลูกค้า

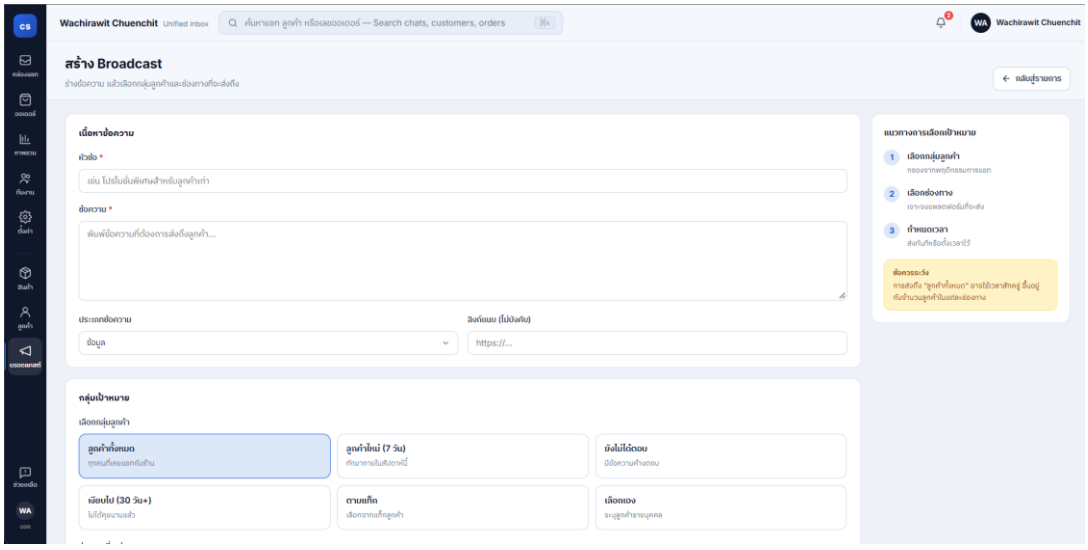
3.6.9 หน้าบรอดแคสต์ ใช้ส่งข้อความถึงลูกค้าหลายคนพร้อมกัน



รูปที่ 3.16 หน้ารายการบรอดแคสต์

จากรูปที่ 3.16 ตารางแสดงหัวข้อและข้อความ ประเภท สถานะ เวลา จำนวนที่ส่งแล้ว จำนวนที่อ่านแล้ว และช่องทาง สถานะมี 5 แบบ ได้แก่ ฉบับร่าง ตั้งเวลาไว้ กำลังส่ง ส่งแล้ว และล้มเหลว แต่ละแถวมีปุ่มดูรายละเอียด แก้ไข ส่งทันที และลบ

ผู้ใช้ต้องมีสิทธิ์ส่งบรอดแคสต์จึงจะสร้างหรือส่งบรอดแคสต์ได้ บรอดแคสต์ส่งได้เฉพาะข้อความ และสำหรับ Facebook และ Instagram ระบบไม่ส่งถึงลูกค้าที่ข้อความล่าสุดเกิน 24 ชั่วโมงตามข้อกำหนดของ Meta เพราะเป็นการส่งอัตโนมัติ บรอดแคสต์ที่ตั้งเวลาไว้ถูกตรวจโดยตัวจัดตารางเวลาทุก 1 นาที เมื่อถึงเวลา ระบบจะเปลี่ยนสถานะเป็นกำลังส่งและส่งงานเข้าคิว หากเลยเวลาที่ตั้งไว้เกิน 1 วันโดยยังไม่ได้ส่ง เช่น เครื่องแม่ข่ายหยุดทำงานในช่วงนั้น ระบบจะถือว่าบ รอดแคสต์ล้มเหลวแทนการส่งข้อความที่ล่าช้าเกินไป เมื่อส่งเสร็จ ตารางจะแสดงจำนวนที่ส่งแล้วและจำนวนที่ลูกค้าอ่านแล้ว หน้าสร้างบรอดแคสต์แสดงดังรูปที่ 3.17



รูปที่ 3.17 หน้าสร้างบรอดแคสต์

จากรูปที่ 3.17 ฟอร์มแบ่งเป็น 3 การ์ด การ์ดแรกกรอกหัวข้อ ข้อความ ประเภทข้อความ และลิงก์แนบ การ์ดที่สองเลือกกลุ่มลูกค้า 6 แบบ ได้แก่ ลูกค้าทั้งหมด ลูกค้าใหม่ 7 วัน ยังไม่ได้ตอบ เงียบเกิน 30 วัน ตามแท็ก และเลือกเอง พร้อมเลือกช่องทางที่จะส่งได้มากกว่าหนึ่งช่องทาง การ์ดที่สามกำหนดวันและเวลาส่ง หากเว้นว่างระบบจะส่งทันที ปุ่ม “ดูจำนวนผู้รับ” แสดงจำนวนลูกค้าที่จะได้รับข้อความก่อนบันทึก ระบบส่งเป็นชุด ชุดละ 100 บทสนทนา และไม่ส่งซ้ำผู้รับที่ส่งสำเร็จแล้ว

3.7 การเชื่อมต่อบริการภายนอก

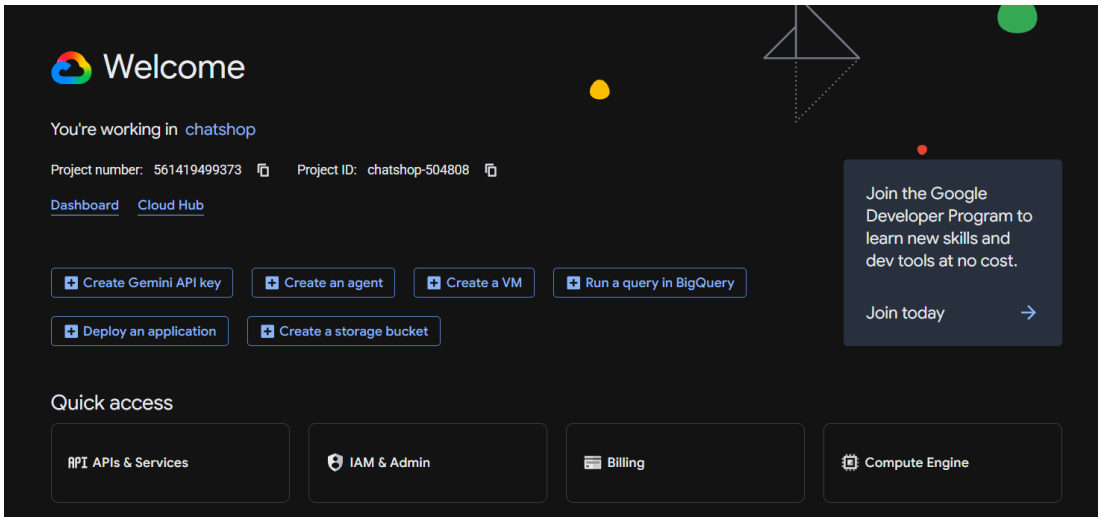
ระบบเชื่อมต่อบริการภายนอก 2 กลุ่ม คือ การเข้าสู่ระบบด้วยบัญชี Google และ Facebook และการเชื่อมต่อช่องทางแชท ได้แก่ Facebook Page, Instagram, LINE Official Account และ WebChat ทุก URL สำหรับ Redirect และ Webhook ใช้โดเมนของระบบ และต้องเป็น HTTPS ที่เข้าถึงได้จากอินเทอร์เน็ต ระหว่างพัฒนา ผู้จัดทำใช้ Cloudflare Tunnel เปิดเครื่องที่รันระบบให้รับคำขอจากภายนอกได้ แต่ละบริการมีขั้นตอนการตั้งค่าดังนี้

3.7.1 การเข้าสู่ระบบด้วย Google ตั้งค่าที่ Google Cloud Console ดังนี้

3.7.1.1 สร้างโปรเจกต์ แล้วตั้งค่าหน้าขอความยินยอม (OAuth consent screen) แบบ External กรอกชื่อแอปพลิเคชันและขอบเขต email และ profile

3.7.1.2 สร้าง OAuth client ID ประเภท Web application และกำหนด Authorized redirect URI เป็น URL ของระบบที่ใช้รับผลการเข้าสู่ระบบด้วย Google

3.7.1.3 ระบบต้องใช้ค่า Client ID และ Client secret ที่ได้ โดยกำหนดไว้ในไฟล์ตั้งค่าสภาพแวดล้อมของระบบ เพื่อไม่ให้ค่าลับปะปนอยู่ในซอร์สโค้ด



รูปที่ 3.18 การสร้าง OAuth client ID ใน Google Cloud Console

จากรูปที่ 3.18 หน้าสร้าง OAuth client ID มีช่องเลือกประเภทแอปพลิเคชัน ชื่อ และรายการ Authorized redirect URIs เมื่อบันทึกแล้ว Google แสดง Client ID และ Client secret ให้คัดลอก เมื่อผู้ใช้กดปุ่มเข้าสู่ระบบด้วย Google ระบบใช้ Laravel Socialite ส่งผู้ใช้ไปยังหน้ายินยอมของ Google แล้ว Google ส่งผู้ใช้กลับมายัง Redirect URI พร้อมรหัสสำหรับแลกข้อมูลบัญชี ระบบใช้อีเมลค้นหาผู้ใช้เดิม หากไม่พบจะสร้างบัญชีและร้านค้าใหม่ให้

3.7.2 การเข้าสู่ระบบด้วย Facebook ใช้แอปพลิเคชันบน Meta for Developers แยกจากแอปที่ใช้เชื่อมต่อเพจ ผู้จัดทำแยกแอปของ Meta เป็น 2 แอป คือ แอปประเภท Consumer สำหรับเข้าสู่ระบบเพียงอย่างเดียว และแอปประเภท Business สำหรับเชื่อมต่อเพจ Instagram และรับ Webhook ในช่วงแรกระบบใช้แอปประเภท Business แอปเดียว แต่แอปประเภทนี้ไม่มี Facebook Login แบบดั้งเดิม การเข้าสู่ระบบจึงต้องขอสิทธิ์ business_management ไปด้วย ทำให้บัญชีที่ไม่มีเพจหรือ Business Portfolio ขึ้นข้อความว่าไม่พบเครื่องมือที่ต้องการใช้และเข้าสู่ระบบไม่ได้ เมื่อแยกแอปแล้ว การเข้าสู่ระบบขอเพียงสิทธิ์ public_profile และ email ซึ่งเป็นสิทธิ์ระดับ Standard Access ไม่ต้องผ่าน App Review และบัญชี Facebook ทั่วไปใช้ได้ ขั้นตอนการตั้งค่ามีดังนี้

3.7.2.1 สร้างแอปประเภท Consumer โดยเลือกกรณีการใช้งานยืนยันตัวตนด้วย Facebook Login แล้วเพิ่มผลิตภัณฑ์ Facebook Login

3.7.2.2 กำหนด App Domains เป็นโดเมนของระบบ และกำหนด Valid OAuth Redirect URIs เป็น URL ของระบบที่ใช้รับผลการเข้าสู่ระบบด้วย Facebook

3.7.2.3 เพิ่มสิทธิ์ email หากไม่เพิ่ม ระบบจะไม่ได้รับอีเมลของผู้ใช้และสร้างร้านค้าใหม่ทุกครั้งที่เข้าสู่ระบบ ซึ่งเป็นปัญหาที่พบจริงระหว่างพัฒนา

3.7.2.4 ระบบต้องใช้ค่า App ID และ App Secret ของแอปนี้ โดยกำหนดไว้ในไฟล์ตั้งค่าสภาพแวดล้อมของระบบ แยกจากค่าของแอปที่ใช้เชื่อมต่อเพจ

3.7.2.5 แอปที่สร้างใหม่อยู่ในโหมดพัฒนา ซึ่งเข้าสู่ระบบได้เฉพาะบัญชีที่มีบทบาทในแอป จึงต้องเพิ่มบัญชีที่ใช้ทดสอบเป็น Tester หรือเปลี่ยนแอปเป็นโหมดใช้งานจริง (Live) ซึ่งต้องกรอก URL นโยบายความเป็นส่วนตัวก่อน โดยระบบมีหน้านโยบายความเป็นส่วนตัวไว้สำหรับขั้นนี้



รูปที่ 3.19 การตั้งค่า Facebook Login ของแอปประเภท Consumer

จากรูปที่ 3.19 หน้าตั้งค่า Facebook Login มีช่อง Valid OAuth Redirect URIs สำหรับกรอก URL ที่ Meta อนุญาตให้ส่งผู้ใช้กลับมา หากกรอก URL ไม่ตรงกับที่ระบบส่งไป Meta จะแสดงข้อผิดพลาดว่า URL ถูกบล็อก

3.7.3 การเชื่อมต่อ Facebook Page และ Instagram ใช้แอปประเภท Business เพราะเป็นประเภทเดียวที่มีผลิตภัณฑ์ Messenger ขั้นตอนการตั้งค่าฝั่ง Meta มีดังนี้

3.7.3.1 สร้างแอปประเภท Business แล้วเพิ่มผลิตภัณฑ์ Messenger สำหรับรับส่งข้อความ และผลิตภัณฑ์ Facebook Login for Business สำหรับขอสิทธิ์เข้าถึงเพจของร้านค้า

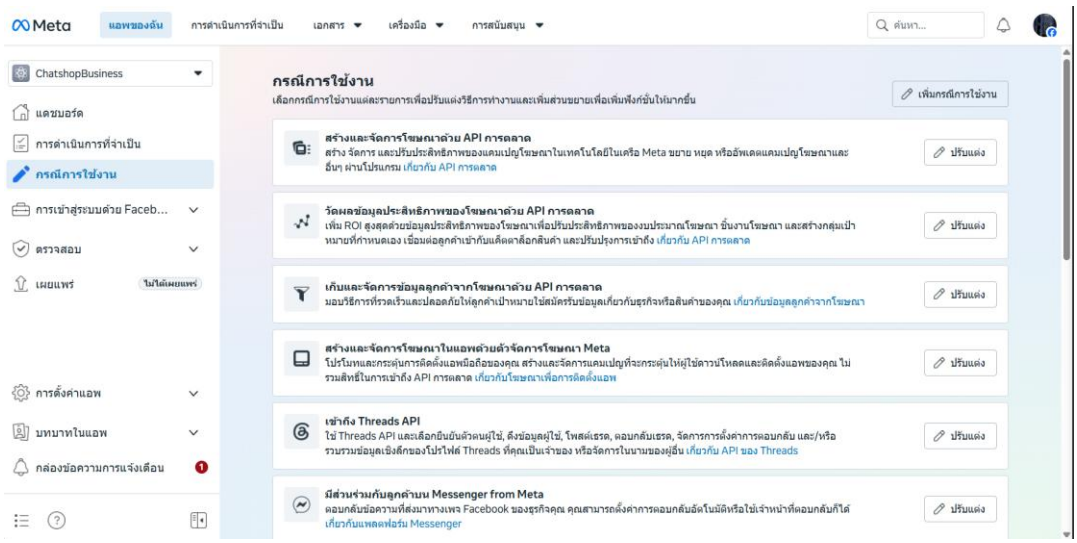
3.7.3.2 กำหนด Valid OAuth Redirect URIs เป็น URL ของระบบที่ใช้รับผลการเชื่อมต่อ 2 รายการ คือ สำหรับเพจ Facebook และสำหรับบัญชี Instagram

3.7.3.3 สร้าง Configuration ใน Facebook Login for Business เลือกชนิดโทเคนเป็น User access token เลือกสินทรัพย์เป็นเพจและบัญชี Instagram และเลือกสิทธิ์ดังนี้

1) pages_show_list ดูรายการเพจที่ผู้ใช้ดูแล
2) pages_messaging รับและส่งข้อความในนามของเพจ
3) pages_manage_metadata สมัครรับ Webhook ของเพจ
4) pages_read_engagement อ่านข้อมูลและเนื้อหาของเพจ
5) instagram_basic อ่านข้อมูลโปรไฟล์ของบัญชี Instagram
6) instagram_manage_messages รับส่งข้อความของ Instagram
7) business_management เข้าถึงเพจใน Business Portfolio

3.7.3.4 ระบบต้องใช้ค่า Configuration ID ที่ได้จากข้อก่อนหน้า รวมทั้ง App ID และ App Secret ของแอปนี้ โดยกำหนดไว้ในไฟล์ตั้งค่าสภาพแวดล้อมของระบบ

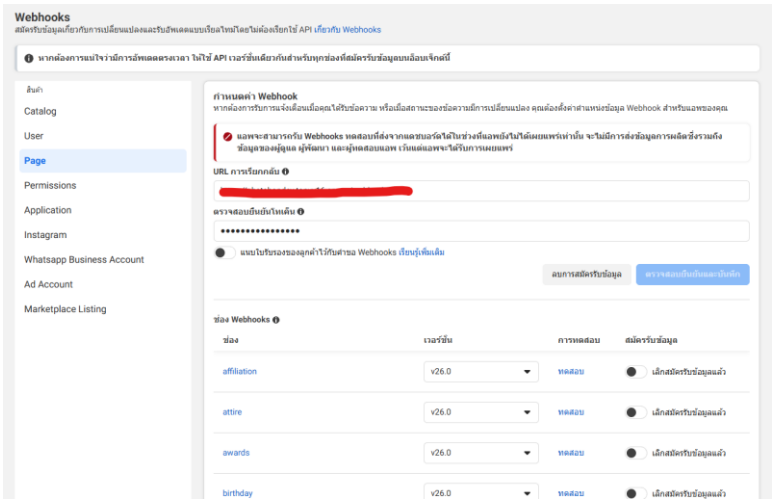
3.7.3.5 ตั้งค่า Webhooks ของผลิตภัณฑ์ Messenger โดยกำหนด Callback URL เป็น URL ของระบบที่ใช้รับ Webhook จาก Meta และกำหนด Verify Token เป็นค่าสุ่มที่ตรงกับค่าที่ตั้งไว้ในระบบ จากนั้นสมัครรับฟิลด์ messages และ messaging_postbacks ของ Messenger และฟิลด์ messages ของ Instagram เพื่อรับข้อความใหม่ของลูกค้า



รูปที่ 3.20 การสร้าง Configuration ของ Facebook Login for Business

จากรูปที่ 3.20 หน้าสร้าง Configuration ประกอบด้วยชื่อ ชนิดของโทเคน ประเภทสินทรัพย์ที่ขอเข้าถึง และรายการสิทธิ์ สิทธิ์ business_management จำเป็นสำหรับเพจที่อยู่ภายใต้ Business Portfolio เพราะหากไม่มีสิทธิ์นี้ Meta จะไม่ส่งเพจเหล่านั้นกลับมาในรายการเพจ ร้านค้าจึงเชื่อมต่อไม่ได้แม้เป็นผู้ดูแลเพจ

ระบบส่งเพียงรหัส Configuration ไปกับคำขอเชื่อมต่อเพจ เมื่อต้องการเพิ่มหรือลดสิทธิ์ ผู้ดูแลแก้ไขที่ Configuration ได้โดยไม่ต้องแก้โค้ด ส่วนการตั้งค่า Webhooks ของ Messenger แสดงดังรูปที่ 3.21

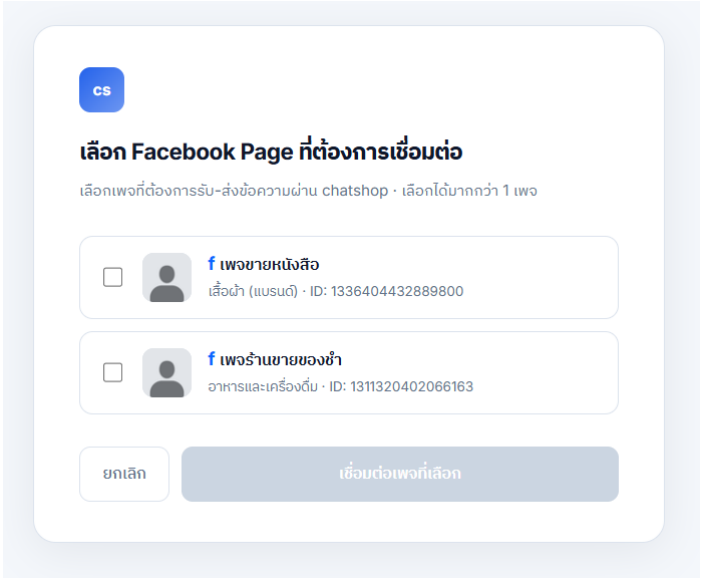


รูปที่ 3.21 การตั้งค่า Webhooks ของ Messenger

จากรูปที่ 3.21 เมื่อกดยืนยัน Meta จะส่งคำขอ GET พร้อมค่า hub.verify_token มายังระบบ ระบบเทียบกับ Verify Token ที่ตั้งไว้ และตอบค่า hub.challenge กลับเมื่อค่าตรงกัน การยืนยัน Callback URL ผ่านแล้วไม่ได้หมายความว่าสมัครรับฟิลด์แล้ว ผู้จัดทำต้องกดสมัครฟิลด์ messages แยกอีกครั้ง ไม่เช่นนั้น Meta จะไม่ส่งข้อความเข้ามาเลย

การตั้งค่าทั้งหมดนี้ทำเพียงครั้งเดียวโดยผู้ดูแลระบบ ร้านค้าที่ใช้งานระบบไม่ต้องสร้างแอปบน Meta for Developers เอง เพราะทุกร้านเชื่อมต่อเพจผ่านแอปเดียวกันของระบบ และแต่ละร้านให้สิทธิ์แก่แอปเฉพาะเพจที่ตนเลือกในหน้ายินยอม Meta ส่ง Webhook ของทุกเพจที่สมัครไว้มายัง Callback URL เดียวกัน ระบบจึงใช้รหัสเพจหรือรหัสบัญชี Instagram ในข้อมูลที่ได้รับ ค้นหาว่าข้อความนั้นเป็นของช่องทางใดและร้านค้าใด ก่อนบันทึกลงบทสนทนาของร้านค้านั้น ด้วยเหตุนี้แอปของระบบจึงต้องผ่าน App Review ก่อนเปิดให้ร้านค้าทั่วไปใช้งาน ตามรายละเอียดในหัวข้อ 3.7.6

เมื่อตั้งค่าฝั่ง Meta เสร็จ ร้านค้าเชื่อมต่อเพจได้จากหน้าตั้งค่าช่องทาง ระบบส่งร้านค้าไปยังหน้ายินยอมของ Meta ซึ่งร้านค้าต้องกด “แก้ไขการตั้งค่า” แล้วเลือกเพจ หากกด “ดำเนินการต่อ” Meta จะใช้การอนุญาตเดิมที่ไม่มีสิทธิ์ของเพจ เมื่อกลับมา ระบบตรวจค่า state เพื่อป้องกันการปลอมคำขอ แลกรหัสที่ได้เป็นโทเคนของผู้ใช้ แลกต่อเป็นโทเคนอายุยาวประมาณ 60 วัน แล้วดึงรายการเพจที่ผู้ใช้เป็นผู้ดูแลมาให้เลือก ดังรูปที่ 3.22



รูปที่ 3.22 หน้าเลือก Facebook Page ที่ต้องการเชื่อมต่อ

จากรูปที่ 3.22 หน้าจอแสดงรายการเพจพร้อมรูป ชื่อ หมวดหมู่ และรหัสเพจ ร้านค้าเลือกได้มากกว่า 1 เพจแล้วกด “เชื่อมต่อเพจที่เลือก” ระบบบันทึก Page Access Token

ของแต่ละเพจแบบเข้ารหัส และสมัครเพจเข้ากับ Webhook ของแอปผ่าน Graph API ให้อัตโนมัติ ร้านค้าจึงไม่ต้องเข้าไปตั้งค่า Webhook ของเพจเองใน Meta for Developers

การเชื่อมต่อ Instagram ใช้แอปเดียวกัน แต่บัญชี Instagram ต้องเป็นบัญชีแบบ Professional และผูกกับเพจที่ร้านค้าเป็นผู้ดูแลก่อน ระบบค้นหาบัญชี Instagram ที่ผูกไว้จากข้อมูลของแต่ละเพจ และใช้ Page Access Token ของเพจที่ผูกไว้ในการรับส่งข้อความ เพราะข้อความจาก Instagram เข้ามาทาง Webhook ของเพจนั้น ระบบจึงบันทึกรหัสเพจที่ผูกไว้คู่กับบัญชี Instagram ด้วย โทเคนของเพจไม่มีการต่ออายุอัตโนมัติ หากหมดอายุหรือถูกยกเลิก ระบบจะปิดการเชื่อมต่อช่องทางนั้น และร้านค้าต้องกดเชื่อมต่อใหม่อีกครั้งจากหน้าตั้งค่าช่องทาง

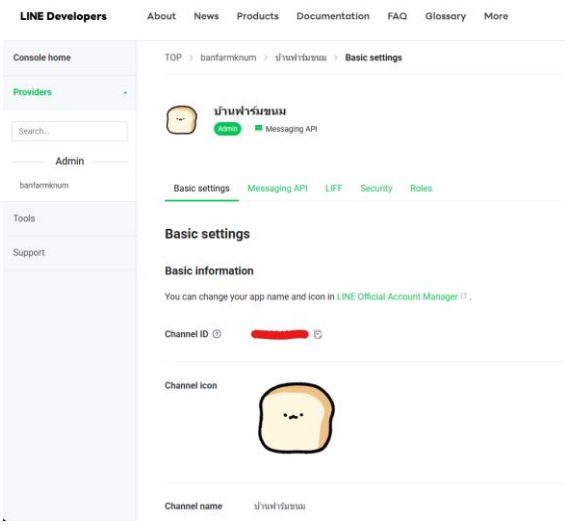
หลังเชื่อมต่อแล้ว ร้านค้านำเข้าประวัติแชทย้อนหลังของเพจได้ โดยเลือกช่วงวันที่ได้ไม่เกิน 90 วัน การนำเข้าทำงานในคิวความสำคัญสูง และติดตามความคืบหน้าได้ หากชนข้อจำกัดจำนวนคำขอของ Meta ระบบจะรอ 60 วินาทีแล้วลองใหม่ไม่เกิน 3 ครั้ง ส่วน LINE ไม่รองรับการนำเข้าประวัติแชท เพราะ LINE ไม่เปิดให้ดึงข้อความย้อนหลังผ่าน API

3.7.4 การเชื่อมต่อ LINE Official Account ไม่ต้องตั้งค่าตัวแปรในระบบ เพราะร้านค้าแต่ละร้านกรอกข้อมูลของบัญชีตนเอง ขั้นตอนฝั่ง LINE มีดังนี้

3.7.4.1 ที่ LINE Developers Console สร้าง Provider สำหรับจัดกลุ่มบัญชีของร้าน แล้วสร้าง Channel แบบ Messaging API ภายใต้ Provider นั้น

3.7.4.2 ที่แท็บ Basic settings คัดลอก Channel ID และ Channel secret

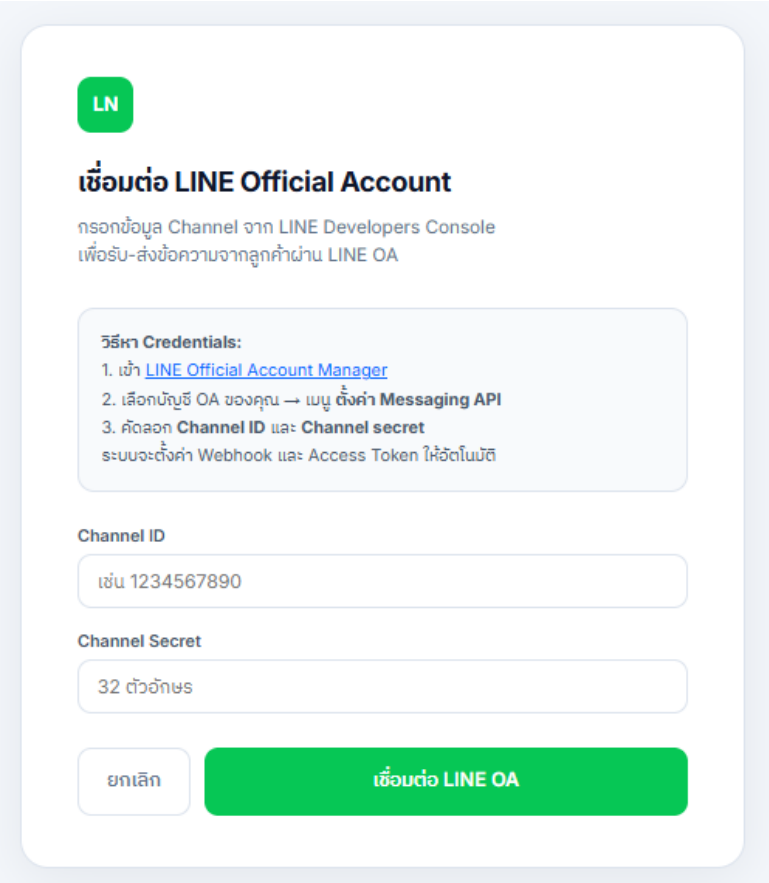
3.7.4.3 ที่แท็บ Messaging API เปิด Use webhook และปิดข้อความตอบกลับอัตโนมัติและข้อความทักทายของ LINE เพื่อไม่ให้ซ้ำกับการตอบของระบบ



รูปที่ 3.23 การตั้งค่า Messaging API ใน LINE Developers Console

จากรูปที่ 3.23 แท็บ Messaging API แสดง Webhook URL สวิตช์ Use webhook และการตั้งค่าข้อความตอบกลับอัตโนมัติ ช่อง Webhook URL ไม่ต้องกรอกเอง เพราะระบบตั้งให้

การส่งข้อความกลับไปยัง LINE ระบบใช้ Push API ซึ่งส่งถึงผู้ใช้ได้ทุกเวลาโดยอ้างอิงรหัสผู้ใช้ แทน Reply API ที่ต้องใช้ replyToken ซึ่งมีอายุสั้น เพราะคำตอบของเจ้าหน้าที่และแชทบอทมักส่งหลังจากได้รับข้อความไปแล้วระยะหนึ่ง ข้อแลกเปลี่ยนคือข้อความที่ส่งด้วย Push API นับรวมในโควตาข้อความรายเดือนของบัญชี ระหว่างที่แชทบอทสร้างคำตอบ ระบบแสดงภาพเคลื่อนไหวกำลังโหลดในหน้าแชทของลูกค้า และแนบปุ่มตอบกลับด่วน (Quick Reply) ไปกับข้อความได้ ส่วนประวัติแชทเดิมของบัญชี LINE นำเข้าระบบไม่ได้ เพราะ LINE ไม่เปิดให้ดึงข้อความย้อนหลังผ่าน API หน้าเชื่อมต่อ LINE Official Account ในระบบแสดงดังรูปที่ 3.24

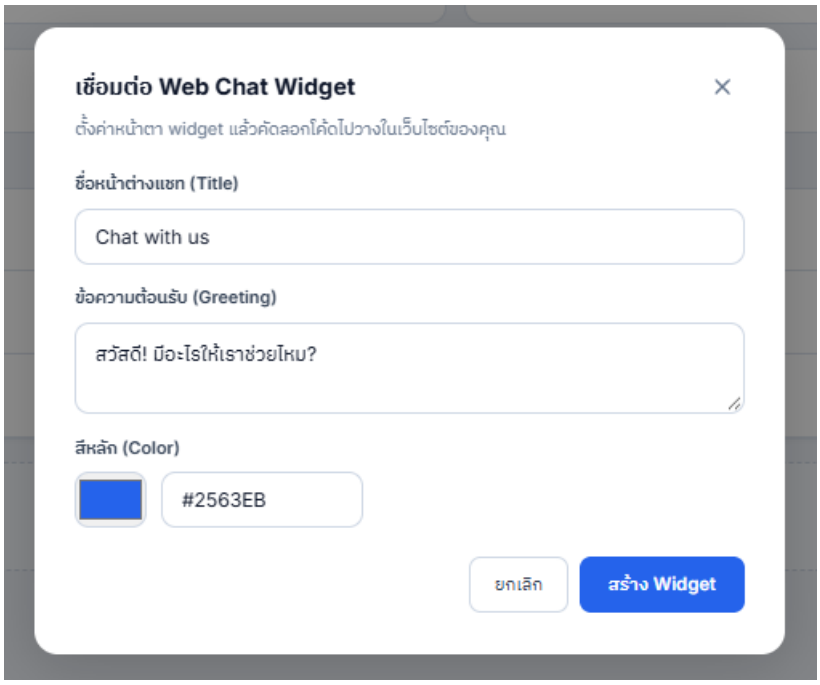


รูปที่ 3.24 หน้าเชื่อมต่อ LINE Official Account ในระบบ

จากรูปที่ 3.24 ร้านค้ากรอกเพียง Channel ID และ Channel Secret แล้วกด “เชื่อมต่อ LINE OA” ระบบนำค่าทั้งสองไปขอ Channel Access Token จาก LINE ด้วยวิธี Client

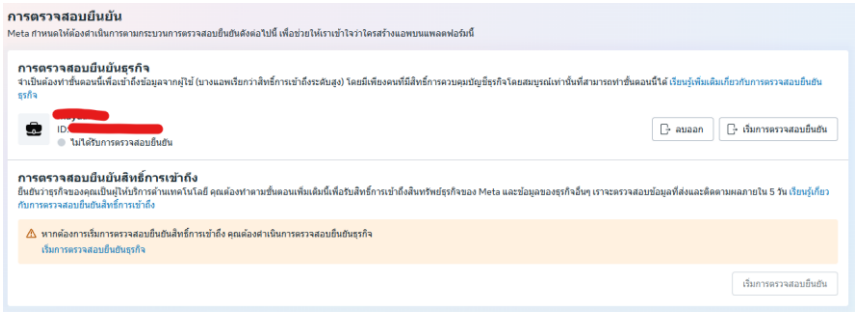
Credentials ดึงชื่อและรูปของบัญชีมาแสดง และตั้ง Webhook URL ของช่องทางนั้นผ่าน API ของ LINE ให้อัตโนมัติ ร้านค้าจึงไม่ต้องกรอก URL เอง ค่า Channel Secret ใช้ตรวจลายเซ็นของทุก Webhook ที่ได้รับ ข้อจำกัดคือโทเคนประเภทนี้มีอายุจำกัดและระบบยังไม่มีขั้นตอนต่ออายุอัตโนมัติ และบัญชี LINE มีโควตาข้อความรายเดือนตามแพ็กเกจของบัญชี

3.7.5 การเชื่อมต่อ WebChat ไม่ต้องตั้งค่ากับบริการภายนอก ร้านค้ากำหนดชื่อหน้าต่างแชท ข้อความต้อนรับ และสีหลัก แล้วกด “สร้าง Widget” ระบบสร้างรหัส Widget แบบสุ่ม และโค้ดสำหรับฝัง ซึ่งเป็นแท็ก script ที่โหลดสคริปต์หน้าต่างแชทจากระบบพร้อมรหัส Widget ร้านค้านำโค้ดนี้ไปวางบนเว็บไซต์ของร้าน เมื่อผู้เยี่ยมชมเริ่มแชท ระบบออกโทเคนของการสนทนาให้หน้าต่างแชทใช้ส่งข้อความ และหน้าต่างแชทรับข้อความใหม่ผ่าน WebSocket ของ Laravel Reverb



รูปที่ 3.25 หน้าต่างโค้ดฝัง WebChat Widget

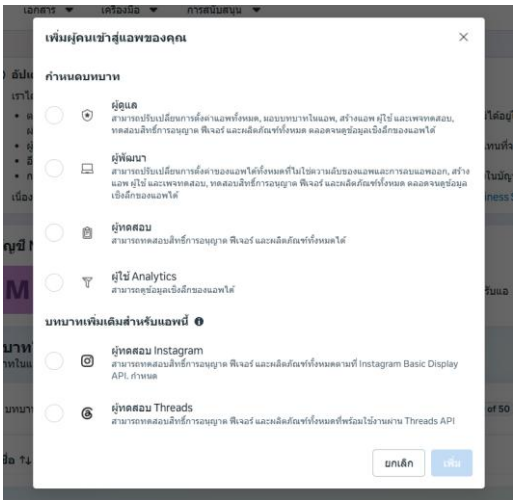
3.7.6 ปัญหาการขออนุมัติแอปจาก Meta สิทธิ์ที่ใช้เชื่อมต่อเพจและ Instagram ทั้งหมดเป็นสิทธิ์ระดับ Advanced Access ขณะที่แอปอยู่ในโหมดพัฒนา สิทธิ์เหล่านี้ใช้ได้เฉพาะบัญชีที่มีบทบาทในแอป ได้แก่ ผู้ดูแล ผู้พัฒนา และผู้ทดสอบ และ Meta ส่ง Webhook มาเฉพาะข้อความจากบัญชีเหล่านี้ การเปิดให้ร้านค้าทั่วไปใช้งานจริงต้องผ่าน 2 ขั้นตอน คือ App Review ซึ่งต้องอธิบายและสาธิตการใช้สิทธิ์แต่ละรายการให้ Meta ตรวจ และ Business Verification ซึ่งต้องยืนยันตัวตนของธุรกิจด้วยเอกสารนิติบุคคล เช่น หนังสือรับรองบริษัท หรือ ภ.พ.20



รูปที่ 3.26 หน้า App Review ของแอปประเภท Business

จากรูปที่ 3.26 หน้าจอแสดงรายการสิทธิ์ที่แอปขอใช้และสถานะของแต่ละสิทธิ์ สิทธิ์ระดับ Advanced Access ต้องยื่นคำขอพร้อมคำอธิบายและวิดีโอสาธิตการใช้งาน

ขณะจัดทำโครงงาน แอปประเภท Business ยังไม่ผ่าน App Review และ Business Verification จึงยังอยู่ในโหมดพัฒนา ผลที่ตามมาคือ บุคคลภายนอกยังเชื่อมต่อเพจหรือ Instagram ไม่ได้ ผู้จัดทำทดสอบระบบได้โดยเพิ่มบัญชีของทีมงานเป็นผู้ทดสอบในแอป ดังรูปที่ 3.27 และทักแชทเข้าเพจจากบัญชีเหล่านั้น นอกจากนี้แท็ก HUMAN_AGENT ซึ่งช่วยขยายเวลาตอบลูกค้าจาก 24 ชั่วโมงเป็น 7 วัน ต้องได้รับสิทธิ์ผ่าน App Review ก่อน แม้ระบบจะส่งข้อความซ้ำพร้อมแท็กนี้เมื่อเกินกรอบ 24 ชั่วโมง แต่ปัจจุบันยังส่งไม่สำเร็จ การตอบลูกค้าทาง Facebook และ Instagram จึงทำได้ภายใน 24 ชั่วโมงนับจากข้อความล่าสุดของลูกค้าเท่านั้น



รูปที่ 3.27 การเพิ่มบัญชีผู้ทดสอบในบทบาทของแอป

จากรูปที่ 3.27 หน้า App Roles ของแอปใน Meta for Developers แสดงรายชื่อบัญชีที่มีบทบาทในแอป แบ่งเป็นผู้ดูแล ผู้พัฒนา และผู้ทดสอบ ผู้จัดทำกด “Add People”

เลือกบทบาท Tester แล้วกรอกชื่อหรือรหัสบัญชี Facebook ของทีมงาน บัญชีที่ได้รับเชิญต้องกดยอมรับคำเชิญที่ Meta for Developers ก่อน บทบาทจึงมีผล หลังจากนั้นบัญชีนั้นเชื่อมต่อเพจที่ตนดูแลเข้ากับระบบ และ Meta ส่ง Webhook ของข้อความระหว่างบัญชีเหล่านี้กับเพจมายังระบบได้ แม้แอปยังอยู่ในโหมดพัฒนา

3.8 การเชื่อมต่อผู้ให้บริการแบบจำลองภาษา

ค่าเริ่มต้นของระบบใช้แบบจำลอง qwen3:14b ผ่าน Ollama บนเครื่องแม่ข่ายของบริษัท ซึ่งกำหนดที่อยู่ของเครื่องไว้ในไฟล์ตั้งค่าสภาพแวดล้อมของระบบ และเรียกใช้ผ่าน API สำหรับสนทนาของ Ollama ร้านค้าที่ต้องการใช้แบบจำลองอื่นเพิ่มผู้ให้บริการของตนเองได้จากแท็บ AI Chatbot ในหน้าตั้งค่า โดยเลือก “API provider ของฉัน” ในส่วน AI Engine ขั้นตอนมีดังนี้

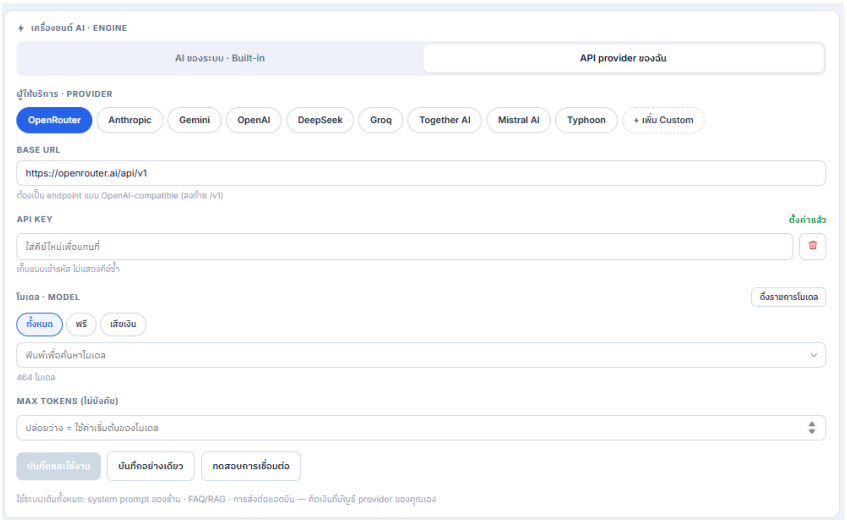
3.8.1 เลือกผู้ให้บริการสำเร็จรูป 9 ราย ได้แก่ OpenRouter, Anthropic, Gemini, OpenAI, DeepSeek, Groq, Together AI, Mistral AI และ Typhoon ระบบจะเติม Base URL ให้อัตโนมัติ หรือเลือก “+ เพิ่ม Custom” เพื่อกรอก Base URL ของผู้ให้บริการอื่นที่รองรับรูปแบบ OpenAI API

3.8.2 กรอก API key ที่สร้างจากเว็บไซต์ของผู้ให้บริการ

3.8.3 กด “ดึงรายการโมเดล” แล้วเลือกแบบจำลองที่ต้องการ

3.8.4 กด “ทดสอบการเชื่อมต่อ” แล้วกด “บันทึกและใช้งาน”

ร้านค้าที่ไม่ได้เพิ่มผู้ให้บริการของตนเองยังใช้ Ollama ของระบบได้ตามปกติ ผู้ให้บริการสำเร็จรูปแต่ละรายต่างกันที่ Base URL และรายชื่อแบบจำลอง เช่น OpenRouter รวมแบบจำลองจากหลายบริษัทไว้ภายใต้ API key เดียว ส่วน Typhoon เป็นแบบจำลองที่พัฒนาสำหรับภาษาไทยโดยเฉพาะ ฟอร์มเพิ่มผู้ให้บริการแสดงดังรูปที่ 3.28



รูปที่ 3.28 ฟอร์มเพิ่มผู้ให้บริการแบบจำลองภาษา

จากรูปที่ 3.28 ฟอร์มประกอบด้วยปุ่มเลือกผู้ให้บริการ ช่อง Base URL ช่อง API key พร้อมสถานะว่าตั้งค่าแล้วหรือไม่ ช่องเลือกโมเดลที่ค้นหาได้และกรองแบบจำลองฟรีหรือเสียเงินได้ ช่อง Max tokens ซึ่งไม่บังคับกรอก และปุ่มบันทึกและทดสอบ การดึงรายการโมเดลและการทดสอบทำผ่านเครื่องแม่ข่ายของระบบทั้งหมด API key จึงไม่ถูกส่งไปยังเบราว์เซอร์ ระบบดึงรายชื่อแบบจำลองจาก API ของผู้ให้บริการและเก็บไว้ในแคช 10 นาที เพื่อลดจำนวนคำขอ ส่วนการทดสอบส่งข้อความ “ping” ไปยัง API สร้างคำตอบ โดยจำกัดคำตอบไว้ 5 โทเคน หากผู้ให้บริการจำกัดจำนวนคำขอ ระบบจะแสดงข้อความให้เลือกแบบจำลองอื่นหรือรอสักครู่

API key ถูกเข้ารหัสด้วยกุญแจของระบบก่อนบันทึกลงฐานข้อมูล แยกตามร้านค้าธุรกิจ และชุดการตั้งค่า แต่ละธุรกิจเก็บผู้ให้บริการได้หลายชุด และเลือกชุดที่ใช้งานได้ครั้งละ 1 ชุด ก่อนเรียกใช้ทุกครั้ง ระบบตรวจว่า Base URL ไม่ได้ชี้ไปยังเครือข่ายภายในของเครื่องแม่ข่าย เพื่อป้องกันการใช้ระบบเป็นทางผ่านโจมตีเครือข่ายภายใน

เมื่อแชทบอทสร้างคำตอบ ระบบส่งคำขอไปยัง API สร้างคำตอบของผู้ให้บริการพร้อม API key ใน Header Authorization แบบ Bearer เนื้อหาคำขอประกอบด้วยชื่อแบบจำลอง รายการข้อความซึ่งข้อความแรกเป็นข้อความสั่งงานระบบ ตามด้วยข้อความของลูกค้าและคำตอบก่อนหน้าของร้าน และจำนวนโทเคนสูงสุดหากร้านค้ากำหนดไว้ ระบบอ่านคำตอบจากผลลัพธ์ที่ได้ และรอคำตอบได้นานสุด 120 วินาที หากตั้งค่าผู้ให้บริการไม่ครบ ระบบจะกลับไปใช้ Ollama แต่หากเรียกผู้ให้บริการไม่สำเร็จ ระบบจะลองใหม่ไม่เกิน 3 ครั้ง แล้วส่งต่อบทสนทนาให้เจ้าหน้าที่ ส่วนการสร้าง Embedding สำหรับฐานความรู้ยังใช้แบบจำลอง bge-m3 ผ่าน Ollama เสมอ ไม่ว่าร้านค้าจะเลือกผู้ให้บริการรายใด เพื่อให้เวกเตอร์ของเอกสารทุกชิ้นเทียบกันได้

บทที่ 4
ผลการดำเนินงานและการทดสอบ

หลังจากพัฒนาระบบเว็บของ Chatshop ตามขั้นตอนในบทที่ 3 แล้ว ผู้จัดทำได้ทดสอบการทำงานของแต่ละฟังก์ชันตามสถานการณ์ใช้งานจริงของร้านค้า บันทึกผลเทียบกับผลลัพธ์ที่คาดหวัง และประเมินความพึงพอใจของผู้ใช้งาน โดยมีหัวข้อดังนี้

4.1 รูปแบบการทดสอบ

4.2 ผลการทดสอบการสมัครและเข้าสู่ระบบด้วยอีเมล

4.3 ผลการทดสอบการเข้าสู่ระบบด้วย Google และ Facebook

4.4 ผลการทดสอบการเชื่อมต่อ LINE Official Account

4.5 ผลการทดสอบการเชื่อมต่อ Facebook Page และ Instagram

4.6 ผลการทดสอบการรับข้อความแบบทันที

4.7 ผลการทดสอบการปฏิเสธ Webhook ที่ลายเซ็นไม่ถูกต้อง

4.8 ผลการทดสอบการตอบข้อความจากกล่องแชท

4.9 ผลการทดสอบการจัดการแท็ก

4.10 ผลการทดสอบการเชิญสมาชิกเข้าทีม

4.11 ผลการทดสอบสิทธิ์ตามบทบาท

4.12 ผลการทดสอบการตอบอัตโนมัติด้วยคีย์เวิร์ด

4.13 ผลการทดสอบการตอบของแชทบอทจากฐานความรู้

4.14 ผลการทดสอบการส่งต่อบทสนทนาให้เจ้าหน้าที่

4.15 ผลการทดสอบการเปิดคำสั่งซื้อจากแชท

4.16 ผลการทดสอบการส่งบรอดแคสต์แบบตั้งเวลา

4.17 ผลการทดสอบการนำเข้าประวัติแชท

4.18 ผลการทดสอบการแจ้งเตือนแชทที่ค้างตอบ

4.19 ผลการทดสอบ WebChat Widget

4.20 แบบประเมินความพึงพอใจในการใช้งานระบบเว็บ

ซึ่งแต่ละหัวข้อมีรายละเอียดดังต่อไปนี้

4.1 รูปแบบการทดสอบ

ผู้จัดทำทดสอบระบบแบบ Black Box โดยเขียนกรณีทดสอบ (Test case) ให้ครอบคลุมทุกส่วนของระบบเว็บ ได้แก่ การเข้าสู่ระบบ การเชื่อมต่อช่องทาง กล่องข้อความ ทีมและสิทธิ์ แชทบอท คำสั่งซื้อ บรอดแคสต์ และการแจ้งเตือน ทุกกรณีทดสอบบนบัญชี LINE Official Account เพจ Facebook และบัญชี Instagram สำหรับทดสอบ แต่ละกรณีบันทึกข้อมูลตามหัวข้อในตารางที่ 4.1

ตารางที่ 4.1 หัวข้อที่ใช้บันทึกกรณีทดสอบ

หัวข้อ	คำอธิบาย
Test Case ID	รหัสสำหรับระบุกรณีทดสอบ
Test Scenario	สถานการณ์ที่ต้องการทดสอบ
Prerequisites	เงื่อนไขที่ต้องเตรียมก่อนทดสอบ
Test Data	ข้อมูลที่ใช้ในการทดสอบ
Test Steps	ขั้นตอนการทดสอบ
Expected Results	ผลลัพธ์ที่คาดหวัง
Actual Results	ผลลัพธ์ที่เกิดขึ้นจริง
Test Status	สถานะของการทดสอบว่าผ่านหรือไม่ผ่าน

จากตารางที่ 4.1 กรณีทดสอบแต่ละกรณีมีรหัส TC ตามด้วยลำดับ และบันทึกผลลัพธ์ที่เกิดขึ้นจริงเทียบกับผลลัพธ์ที่คาดหวัง หากผลตรงกันจึงถือว่าผ่าน

4.2 ผลการทดสอบการสมัครและเข้าสู่ระบบด้วยอีเมล

การทดสอบนี้ตรวจการสมัครร้านค้าและการเข้าสู่ระบบด้วยอีเมล ดังตารางที่ 4.2

ตารางที่ 4.2 การทดสอบการสมัครและเข้าสู่ระบบด้วยอีเมล

หัวข้อ	คำอธิบาย
Test Case ID	TC 001
Test Scenario	สมัครร้านค้าใหม่ด้วยอีเมล แล้วเข้าสู่ระบบด้วยรหัสผ่านที่ถูกต้องและไม่ถูกต้อง
Prerequisites	อีเมลที่ใช้ทดสอบยังไม่เคยสมัครในระบบ
Test Data	ชื่อร้าน อีเมล และรหัสผ่านสำหรับทดสอบ

ตารางที่ 4.2 การทดสอบการสมัครและเข้าสู่ระบบด้วยอีเมล (ต่อ)

หัวข้อ	คำอธิบาย
Test Steps	1. เปิดหน้าสมัครใช้งาน กรอกชื่อร้าน อีเมล และรหัสผ่าน แล้วกดสมัคร 2. ออกจากระบบ 3. เข้าสู่ระบบด้วยรหัสผ่านที่ผิด 4. เข้าสู่ระบบด้วยรหัสผ่านที่ถูกต้อง
Expected Results	ขั้นที่ 1 ระบบสร้างร้านค้าใหม่และพาเข้าสู่หน้ากล่องแชท ขั้นที่ 3 ระบบแจ้งว่าข้อมูลเข้าสู่ระบบไม่ถูกต้อง ขั้นที่ 4 เข้าสู่ระบบสำเร็จ
Actual Results	ระบบสร้างร้านค้าใหม่และพาเข้าสู่หน้ากล่องแชท แจ้งข้อมูลเข้าสู่ระบบไม่ถูกต้องเมื่อใช้รหัสผ่านที่ผิด และเข้าสู่ระบบสำเร็จเมื่อใช้รหัสผ่านที่ถูกต้อง
Test Status	ผ่าน

จากตารางที่ 4.2 ระบบสร้างร้านค้าใหม่จากการสมัครด้วยอีเมลได้ถูกต้อง และไม่ยอมให้เข้าสู่ระบบเมื่อรหัสผ่านผิด ผลที่ได้ตรงกับผลลัพธ์ที่คาดหวัง จึงผ่านการทดสอบ

4.3 ผลการทดสอบการเข้าสู่ระบบด้วย Google และ Facebook

การทดสอบนี้ตรวจการเข้าสู่ระบบผ่านบัญชี Google และ Facebook ดังตารางที่ 4.3

ตารางที่ 4.3 การทดสอบการเข้าสู่ระบบด้วย Google และ Facebook

หัวข้อ	คำอธิบาย
Test Case ID	TC 002
Test Scenario	เข้าสู่ระบบด้วยบัญชี Google และ Facebook ครั้งแรกและครั้งถัดไป
Prerequisites	ตั้งค่า OAuth client ของ Google และแอป Facebook Login ตามหัวข้อ 3.7.1 และ 3.7.2 แล้ว
Test Data	บัญชี Google และบัญชี Facebook ที่เพิ่มเป็นผู้ทดสอบของแอป
Test Steps	1. กดปุ่มเข้าสู่ระบบด้วย Google แล้วยินยอมให้สิทธิ์ 2. ออกจากระบบแล้วเข้าสู่ระบบด้วย Google อีกครั้ง 3. ทำซ้ำขั้นที่ 1 และ 2 ด้วยบัญชี Facebook

ตารางที่ 4.3 การทดสอบการเข้าสู่ระบบด้วย Google และ Facebook (ต่อ)

หัวข้อ	คำอธิบาย
Expected Results	ครั้งแรกระบบสร้างร้านค้าใหม่ ครั้งถัดไประบบพาเข้าร้านค้าเดิมโดยไม่สร้างร้านค้าซ้ำ
Actual Results	ครั้งแรกระบบสร้างร้านค้าใหม่ และครั้งถัดไปพาเข้าร้านค้าเดิมโดยไม่สร้างร้านค้าซ้ำ ทั้งบัญชี Google และ Facebook
Test Status	ผ่าน

จากตารางที่ 4.3 การเข้าสู่ระบบด้วยบัญชี Google และ Facebook ได้อีเมลของผู้ใช้กลับมาครบ ระบบจึงพาผู้ใช้เข้าร้านค้าเดิมได้ถูกต้อง และผ่านการทดสอบ

4.4 ผลการทดสอบการเชื่อมต่อ LINE Official Account

การทดสอบนี้ตรวจการเชื่อมต่อและรับข้อความจากบัญชี LINE ดังตารางที่ 4.4

ตารางที่ 4.4 การทดสอบการเชื่อมต่อ LINE Official Account

หัวข้อ	คำอธิบาย
Test Case ID	TC 003
Test Scenario	เชื่อมต่อบัญชี LINE Official Account เข้ากับร้านค้า
Prerequisites	สร้าง Channel แบบ Messaging API และเปิด Use webhook ตามหัวข้อ 3.7.4 แล้ว
Test Data	Channel ID, Channel secret และ Channel access token ของบัญชีทดสอบ
Test Steps	1. เปิดหน้าตั้งค่า แท็บช่องทาง แล้วเลือกเพิ่มช่องทาง LINE 2. กรอกข้อมูลของ Channel แล้วกดบันทึก 3. คัดลอก Webhook URL ที่ระบบสร้างไปใส่ใน LINE Developers Console แล้วกด Verify 4. ส่งข้อความจากบัญชี LINE ส่วนตัวหาบัญชีร้าน
Expected Results	ช่องทาง LINE แสดงสถานะเชื่อมต่อแล้ว การกด Verify สำเร็จ และข้อความจากขั้นที่ 4 ปรากฏในกล่องแชท
Actual Results	ช่องทาง LINE แสดงสถานะเชื่อมต่อแล้ว การกด Verify สำเร็จ และข้อความจากบัญชีส่วนตัวปรากฏในกล่องแชท
Test Status	ผ่าน

จากตารางที่ 4.4 ร้านค้าเชื่อมต่อบัญชี LINE Official Account ของตนเองได้ และระบบรับข้อความจากบัญชีนั้นได้ทันที จึงผ่านการทดสอบ

4.5 ผลการทดสอบการเชื่อมต่อ Facebook Page และ Instagram

การทดสอบนี้ตรวจการเชื่อมต่อเพจ Facebook และบัญชี Instagram ดังตารางที่ 4.5

ตารางที่ 4.5 การทดสอบการเชื่อมต่อ Facebook Page และ Instagram

หัวข้อ	คำอธิบาย
Test Case ID	TC 004
Test Scenario	เชื่อมต่อเพจ Facebook และบัญชี Instagram ที่ผูกกับเพจ
Prerequisites	ผู้ทดสอบมีบทบาทในแอป Meta และเป็นผู้ดูแลเพจทดสอบ
Test Data	เพจ Facebook และบัญชี Instagram แบบมืออาชีพสำหรับทดสอบ
Test Steps	1. เลือกเพิ่มช่องทาง Facebook แล้วเข้าสู่ระบบ Facebook 2. เลือกเพจที่ต้องการแล้วยืนยัน 3. เลือกเพิ่มช่องทาง Instagram แล้วเลือกบัญชีที่ผูกกับเพจ 4. ส่งข้อความหาเพจและบัญชี Instagram
Expected Results	เพจและบัญชี Instagram แสดงในแท็บช่องทาง และข้อความจากขั้นที่ 4 ปรากฏในกล่องแชทพร้อมไอคอนของช่องทาง
Actual Results	เพจและบัญชี Instagram แสดงในแท็บช่องทาง และข้อความที่ส่งหาเพจและบัญชี Instagram ปรากฏในกล่องแชทพร้อมไอคอนของช่องทาง
Test Status	ผ่าน

จากตารางที่ 4.5 ระบบขอสิทธิ์เข้าถึงเพจและบัญชี Instagram ได้ครบ และรับข้อความจากทั้ง 2 ช่องทางเข้ามาในกล่องแชทเดียวกันได้ จึงผ่านการทดสอบ

4.6 ผลการทดสอบการรับข้อความแบบทันที

การทดสอบนี้ตรวจว่าข้อความใหม่แสดงโดยไม่ต้องโหลดหน้าใหม่ ดังตารางที่ 4.6

ตารางที่ 4.6 การทดสอบการรับข้อความแบบทันที

หัวข้อ	คำอธิบาย
Test Case ID	TC 005

ตารางที่ 4.6 การทดสอบการรับข้อความแบบทันที (ต่อ)

หัวข้อ	คำอธิบาย
Test Scenario	ลูกค้าส่งข้อความ รูปภาพ และสติกเกอร์ ขณะที่เจ้าหน้าที่เปิดกล่องแชทค้างไว้
Prerequisites	เชื่อมต่อช่องทางแล้ว และ Queue worker กับ Laravel Reverb ทำงานอยู่
Test Data	ข้อความ รูปภาพ และสติกเกอร์จากบัญชีลูกค้าทดสอบ
Test Steps	1. เจ้าหน้าที่เปิดหน้ากล่องแชทค้างไว้ 2. ลูกค้าส่งข้อความ รูปภาพ และสติกเกอร์ตามลำดับ 3. สังเกตรายการบทสนทนาและหน้าต่างข้อความ
Expected Results	ข้อความทั้ง 3 รายการแสดงตามลำดับโดยไม่ต้องโหลดหน้าใหม่ บทสนทนาเลื่อนขึ้นไปบนสุดของรายการ และแสดงจำนวนข้อความที่ยังไม่อ่าน
Actual Results	ข้อความ รูปภาพ และสติกเกอร์แสดงตามลำดับโดยไม่ต้องโหลดหน้าใหม่ บทสนทนาเลื่อนขึ้นไปบนสุดของรายการ และแสดงจำนวนข้อความที่ยังไม่อ่าน
Test Status	ผ่าน

จากตารางที่ 4.6 ข้อความใหม่จากลูกค้าแสดงในกล่องแชทได้ทันทีโดยไม่ต้องโหลดหน้าใหม่ แสดงว่า Webhook คิวงาน และ WebSocket ทำงานร่วมกันได้ถูกต้อง

4.7 ผลการทดสอบการปฏิเสธ Webhook ที่ลายเซ็นไม่ถูกต้อง

การทดสอบนี้ตรวจว่าระบบไม่รับข้อมูลที่ไม่ได้มาจากแพลตฟอร์มจริง ดังตารางที่ 4.7

ตารางที่ 4.7 การทดสอบการปฏิเสธ Webhook ที่ลายเซ็นไม่ถูกต้อง

หัวข้อ	คำอธิบาย
Test Case ID	TC 006
Test Scenario	ส่งคำขอไปยัง URL ของ Webhook โดยใส่ลายเซ็นที่ไม่ถูกต้อง
Prerequisites	ระบบเปิดใช้งานและเข้าถึง URL ของ Webhook ได้
Test Data	ข้อมูลเหตุการณ์ข้อความจำลอง พร้อมลายเซ็นที่สร้างจากค่าลับที่ผิด

ตารางที่ 4.7 การทดสอบการปฏิเสธ Webhook ที่ลายเซ็นไม่ถูกต้อง (ต่อ)

หัวข้อ	คำอธิบาย
Test Steps	1. ส่งคำขอ POST ไปยัง Webhook ของ Meta ด้วยลายเซ็นที่ผิด 2. ส่งคำขอ POST ไปยัง Webhook ของ LINE ด้วยลายเซ็นที่ผิด 3. ตรวจกล่องแชทว่ามีข้อความใหม่หรือไม่
Expected Results	ระบบตอบกลับด้วยรหัสสถานะปฏิเสธคำขอ และไม่มีข้อความจำลองปรากฏในกล่องแชท
Actual Results	ระบบตอบกลับด้วยรหัสสถานะปฏิเสธคำขอทั้ง 2 แพลตฟอร์ม และไม่มีข้อความจำลองปรากฏในกล่องแชท
Test Status	ผ่าน

จากตารางที่ 4.7 ระบบปฏิเสธคำขอที่ลายเซ็นไม่ถูกต้องและไม่บันทึกข้อมูลนั้น จึงป้องกันข้อมูลปลอมที่ไม่ได้ส่งมาจาก LINE หรือ Meta ได้ และผ่านการทดสอบ

4.8 ผลการทดสอบการตอบข้อความจากกล่องแชท

การทดสอบนี้ตรวจการส่งข้อความออกไปยังแต่ละช่องทาง ดังตารางที่ 4.8

ตารางที่ 4.8 การทดสอบการตอบข้อความจากกล่องแชท

หัวข้อ	คำอธิบาย
Test Case ID	TC 007
Test Scenario	เจ้าหน้าที่ตอบลูกค้าด้วยข้อความ สติกเกอร์ และไฟล์ บนช่องทาง LINE, Facebook และ Instagram
Prerequisites	มีบทสนทนาจากลูกค้าในแต่ละช่องทาง และผู้ทดสอบมีสิทธิ์ตอบแชท
Test Data	ข้อความสั้น ข้อความยาวเกิน 2,000 ตัวอักษร สติกเกอร์ รูปภาพขนาดไม่เกิน 10 MB และรูปภาพขนาดเกิน 10 MB
Test Steps	1. เปิดบทสนทนาแล้วส่งข้อความสั้น 2. ส่งสติกเกอร์และรูปภาพ 3. ส่งข้อความยาวเกิน 2,000 ตัวอักษร 4. แนบรูปภาพขนาดเกิน 10 MB 5. ทำซ้ำขั้นที่ 1 ถึง 4 ในทุกช่องทาง
Expected Results	ขั้นที่ 1 และ 2 ลูกค้าได้รับข้อความและไฟล์ในแอปพลิเคชันของช่องทางนั้น ขั้นที่ 3 และ 4 ระบบไม่ส่งและแจ้งเหตุผลให้ผู้ใช้ทราบ

ตารางที่ 4.8 การทดสอบการตอบข้อความจากกล่องแชท (ต่อ)

หัวข้อ	คำอธิบาย
Actual Results	ลูกค้าได้รับข้อความ สติกเกอร์ และรูปภาพในทุกช่องทาง ส่วนข้อความที่ยาวเกิน 2,000 ตัวอักษรและรูปภาพที่เกิน 10 MB ระบบไม่ส่งและแจ้งเหตุผลให้ผู้ใช้ทราบ
Test Status	ผ่าน

จากตารางที่ 4.8 เจ้าหน้าที่ตอบลูกค้าได้ครบทุกช่องทางจากหน้าเดียว และระบบตรวจขีดจำกัดของข้อความและไฟล์ก่อนส่งได้ถูกต้อง จึงผ่านการทดสอบ

4.9 ผลการทดสอบการจัดการแท็ก

การทดสอบนี้ตรวจการสร้าง การติด และการลบแท็ก ดังตารางที่ 4.9

ตารางที่ 4.9 การทดสอบการจัดการแท็ก

หัวข้อ	คำอธิบาย
Test Case ID	TC 008
Test Scenario	สร้างแท็ก ติดแท็กบนบทสนทนา ค้นหาด้วยแท็ก แล้วลบแท็ก
Prerequisites	ผู้ทดสอบมีสิทธิ์จัดการแท็ก และมีบทสนทนาอย่างน้อย 2 รายการ
Test Data	แท็กชื่อ “ลูกค้าประจำ” สีเขียว
Test Steps	1. สร้างแท็กในแท็บแท็กของหน้าตั้งค่า 2. ติดแท็กบนบทสนทนา 2 รายการ 3. ค้นหาบทสนทนาด้วยชื่อแท็ก 4. ลบแท็กในหน้าตั้งค่า
Expected Results	ขั้นที่ 3 แสดงเฉพาะ 2 บทสนทนาที่ติดแท็ก ขั้นที่ 4 แท็กหายไปจากทุกบทสนทนา
Actual Results	การค้นหาด้วยชื่อแท็กแสดงเฉพาะ 2 บทสนทนาที่ติดแท็ก และเมื่อลบแท็กแล้ว แท็กหายไปจากทุกบทสนทนา
Test Status	ผ่าน

จากตารางที่ 4.9 ระบบสร้าง ติด ค้นหา และลบแท็กได้ถูกต้อง และไม่เหลือแท็กที่ถูกลบค้างอยู่บนบทสนทนาใดเลย ผลตรงกับที่คาดหวัง จึงผ่านการทดสอบ

4.10 ผลการทดสอบการเชิญสมาชิกเข้าทีม

การทดสอบนี้ตรวจการเชิญสมาชิกและการป้องกันการเชิญซ้ำ ดังตารางที่ 4.10

ตารางที่ 4.10 การทดสอบการเชิญสมาชิกเข้าทีม

หัวข้อ	คำอธิบาย
Test Case ID	TC 009
Test Scenario	เจ้าของร้านเชิญสมาชิกใหม่ในบทบาทพนักงานตอบแชท แล้วเชิญอีเมลเดิมซ้ำ
Prerequisites	เข้าสู่ระบบในบทบาทเจ้าของร้าน
Test Data	อีเมลของผู้ทดสอบคนที่สอง
Test Steps	1. เปิดหน้าทีมงาน กรอกอีเมล เลือกบทบาท แล้วกดเชิญ 2. เปิดลิงก์คำเชิญจากอีเมลแล้วตั้งรหัสผ่าน 3. เจ้าของร้านเชิญอีเมลเดิมซ้ำ
Expected Results	ขั้นที่ 2 ผู้ได้รับเชิญเข้าร้านค้าในบทบาทที่กำหนด ขั้นที่ 3 ระบบแจ้งว่าอีเมลนี้เป็นสมาชิกอยู่แล้ว
Actual Results	ผู้ได้รับเชิญเข้าร้านค้าในบทบาทที่กำหนด และเมื่อเชิญอีเมลเดิมซ้ำ ระบบแจ้งว่าอีเมลนี้เป็นสมาชิกอยู่แล้ว
Test Status	ผ่าน

จากตารางที่ 4.10 เจ้าของร้านเชิญสมาชิกเข้าทีมทางอีเมลได้ และระบบป้องกันการเชิญอีเมลที่เป็นสมาชิกของร้านอยู่แล้วซ้ำได้ จึงผ่านการทดสอบ

4.11 ผลการทดสอบสิทธิ์ตามบทบาท

การทดสอบนี้ตรวจว่าสมาชิกทำได้เฉพาะงานที่บทบาทมีสิทธิ์ ดังตารางที่ 4.11

ตารางที่ 4.11 การทดสอบสิทธิ์ตามบทบาท

หัวข้อ	คำอธิบาย
Test Case ID	TC 010
Test Scenario	สมาชิกบทบาทผู้ชมซึ่งดูได้เฉพาะรายงานเข้าหน้าอื่น และพนักงานตอบแชทมอบหมายบทสนทนา
Prerequisites	มีสมาชิกบทบาทผู้ชม และบทบาทพนักงานตอบแชทที่ไม่มีสิทธิ์มอบหมาย
Test Data	บัญชีสมาชิกทั้ง 2 บทบาท

ตารางที่ 4.11 การทดสอบสิทธิ์ตามบทบาท (ต่อ)

หัวข้อ	คำอธิบาย
Test Steps	1. เข้าสู่ระบบด้วยบทบาทผู้ชมแล้วพยายามเปิดหน้ากล่องแชทและหน้าตั้งค่า 2. เข้าสู่ระบบด้วยบทบาทพนักงานตอบแชทแล้วพยายามมอบหมายบทสนทนาให้ผู้อื่น 3. เรียก API ของการกระทำในขั้นที่ 1 และ 2 โดยตรง
Expected Results	บทบาทผู้ชมเห็นเฉพาะหน้าภาพรวม หน้าเว็บไม่แสดงปุ่มหรือเมนูที่ไม่มีสิทธิ์ และการเรียก API โดยตรงถูกปฏิเสธ
Actual Results	บทบาทผู้ชมเห็นเฉพาะหน้าภาพรวม หน้าเว็บไม่แสดงปุ่มหรือเมนูที่ไม่มีสิทธิ์ และการเรียก API โดยตรงถูกปฏิเสธ
Test Status	ผ่าน

จากตารางที่ 4.11 ระบบตรวจสิทธิ์ทั้งที่หน้าเว็บและที่ API สมาชิกจึงทำได้เฉพาะงานที่บทบาทของตนมีสิทธิ์ แม้จะเรียก API โดยตรงโดยไม่ผ่านหน้าเว็บ จึงผ่านการทดสอบ

4.12 ผลการทดสอบการตอบอัตโนมัติด้วยคีย์เวิร์ด

การทดสอบนี้ตรวจการจับคู่คีย์เวิร์ดทั้ง 2 แบบ ดังตารางที่ 4.12

ตารางที่ 4.12 การทดสอบการตอบอัตโนมัติด้วยคีย์เวิร์ด

หัวข้อ	คำอธิบาย
Test Case ID	TC 011
Test Scenario	ตั้งกฎคีย์เวิร์ด 2 กฎ แล้วให้ลูกค้าส่งข้อความที่ตรงและไม่ตรงกับกฎ
Prerequisites	เปิดใช้งานการตอบอัตโนมัติในแท็บอัตโนมัติของหน้าตั้งค่า
Test Data	กฎที่ 1 คำว่า “ราคา” แบบตรงทั้งข้อความ กฎที่ 2 คำว่า “ส่งฟรี” แบบมีคำนี้
Test Steps	1. ลูกค้าส่งข้อความ “ราคา” 2. ลูกค้าส่งข้อความ “ราคาเท่าไหร่” 3. ลูกค้าส่งข้อความ “มีส่งฟรีไหม” 4. ปิดกฎที่ 2 แล้วส่งข้อความในขั้นที่ 3 ซ้ำ
Expected Results	ขั้นที่ 1 และ 3 ระบบตอบกลับตามกฎ ขั้นที่ 2 และ 4 ระบบไม่ตอบ และหน้าตั้งค่าแสดงจำนวนครั้งที่กฎทำงานเพิ่มขึ้น

ตารางที่ 4.12 การทดสอบการตอบอัตโนมัติด้วยคีย์เวิร์ด (ต่อ)

หัวข้อ	คำอธิบาย
Actual Results	ระบบตอบกลับข้อความ “ราคา” และ “มีส่งฟรีไหม” ตามกฎ ไม่ตอบข้อความ “ราคาเท่าไหร่” และไม่ตอบเมื่อปิดกฎที่ 2 และหน้าตั้งค่าแสดงจำนวนครั้งที่กฎทำงานเพิ่มขึ้น
Test Status	ผ่าน

จากตารางที่ 4.12 ระบบจับคู่คีย์เวิร์ดได้ถูกต้องทั้งแบบตรงทั้งข้อความและแบบมีคำนี้ และหยุดตอบทันทีเมื่อร้านค้าปิดกฎนั้น ผลตรงกับที่คาดหวัง จึงผ่านการทดสอบ

4.13 ผลการทดสอบการตอบของแชทบอทจากฐานความรู้

การทดสอบนี้ตรวจการตอบของแชทบอทจากเอกสารความรู้ ดังตารางที่ 4.13

ตารางที่ 4.13 การทดสอบการตอบของแชทบอทจากฐานความรู้

หัวข้อ	คำอธิบาย
Test Case ID	TC 012
Test Scenario	ลูกค้าถามคำถามที่มีคำตอบอยู่ในเอกสารความรู้ของร้านค้า
Prerequisites	ตั้งโหมดแชทบอทเป็น Autopilot และเพิ่มเอกสารความรู้ของร้านแล้ว
Test Data	เอกสารความรู้ระบุเวลาเปิดร้านและค่าจัดส่ง
Test Steps	1. ลูกค้าส่งข้อความ “ร้านเปิดกี่โมง” 2. ลูกค้าส่งข้อความ 3 ข้อความติดกันภายในไม่กี่วินาทีเพื่อถามค่าจัดส่ง 3. ตรวจคำตอบในกล่องแชทและในแอปพลิเคชันของลูกค้า
Expected Results	แชทบอทตอบตรงกับข้อมูลในเอกสาร ขั้นที่ 2 แชทบอทตอบเพียงครั้งเดียวโดยรวมทั้ง 3 ข้อความเป็นคำถามเดียว
Actual Results	แชทบอทตอบเวลาเปิดร้านและค่าจัดส่งตรงกับข้อมูลในเอกสาร และตอบเพียงครั้งเดียวสำหรับข้อความ 3 ข้อความที่ส่งมาติดกัน
Test Status	ผ่าน

จากตารางที่ 4.13 แชทบอทตอบจากเอกสารความรู้ของร้านค้าได้ตรงตามข้อมูล และรวมข้อความที่ส่งมาติดกันเป็นคำถามเดียว ลูกค้าจึงไม่ได้รับคำตอบซ้ำ

4.14 ผลการทดสอบการส่งต่อบทสนทนาให้เจ้าหน้าที่

การทดสอบนี้ตรวจการส่งต่อบทสนทนาจากแชทบอทให้เจ้าหน้าที่ ดังตารางที่ 4.14

ตารางที่ 4.14 การทดสอบการส่งต่อบทสนทนาให้เจ้าหน้าที่

หัวข้อ	คำอธิบาย
Test Case ID	TC 013
Test Scenario	ลูกค้าขอคุยกับเจ้าหน้าที่ และส่งรูปสลิปหลังถามเรื่องการชำระเงิน
Prerequisites	ตั้งโหมดแชทบอทเป็น Autopilot
Test Data	ข้อความ “ขอคุยกับแอดมิน” และรูปสลิปโอนเงินตัวอย่าง
Test Steps	1. ลูกค้าส่งข้อความ “ขอคุยกับแอดมิน” 2. ตรวจแท็กและโหมดของบทสนทนา 3. เจ้าหน้าที่ตอบลูกค้า 4. ในบทสนทนาใหม่ ลูกค้าถามวิธีโอนเงินแล้วส่งรูปสลิป
Expected Results	ขั้นที่ 2 บทสนทนาถูกติดแท็ก “รอแอดมิน” และเปลี่ยนเป็นโหมด Manual ขั้นที่ 3 แท็กถูกถอดออก ขั้นที่ 4 ลูกค้าได้รับข้อความรับทราบอัตโนมัติ และบทสนทนาถูกส่งต่อให้เจ้าหน้าที่ตรวจสลิป
Actual Results	บทสนทนาถูกติดแท็ก “รอแอดมิน” และเปลี่ยนเป็นโหมด Manual แท็กถูกถอดเมื่อเจ้าหน้าที่ตอบ และเมื่อลูกค้าส่งรูปสลิป ลูกค้าได้รับข้อความรับทราบอัตโนมัติ และบทสนทนาถูกส่งต่อให้เจ้าหน้าที่
Test Status	ผ่าน

จากตารางที่ 4.14 แชทบอทหยุดตอบและส่งต่อบทสนทนาให้เจ้าหน้าที่ได้ทั้งกรณีที่ลูกค้าขอคุยกับแอดมินและกรณีที่ลูกค้าส่งสลิป จึงผ่านการทดสอบ

4.15 ผลการทดสอบการเปิดคำสั่งซื้อจากแชท

การทดสอบนี้ตรวจการเปิดคำสั่งซื้อและการเปลี่ยนสถานะ ดังตารางที่ 4.15

ตารางที่ 4.15 การทดสอบการเปิดคำสั่งซื้อจากแชท

หัวข้อ	คำอธิบาย
Test Case ID	TC 014
Test Scenario	เปิดคำสั่งซื้อจากบทสนทนา แล้วเปลี่ยนสถานะจนจัดส่ง
Prerequisites	มีสินค้าในแค็ตตาล็อกและมีบทสนทนากับลูกค้า

ตารางที่ 4.15 การทดสอบการเปิดคำสั่งซื้อจากแชท (ต่อ)

หัวข้อ	คำอธิบาย
Test Data	สินค้าราคา 150 บาท จำนวน 3 ชิ้น และที่อยู่จัดส่งของลูกค้า
Test Steps	1. เปิดบทสนทนาแล้วกดสร้างคำสั่งซื้อ 2. เลือกสินค้า กรอกจำนวน 3 และที่อยู่จัดส่ง แล้วบันทึก 3. เปลี่ยนสถานะเป็นชำระแล้ว และจัดส่งแล้วตามลำดับ 4. เปิดหน้าออเดอร์และหน้าลูกค้า
Expected Results	ยอดรวมเท่ากับ 450 บาท สถานะเปลี่ยนตามลำดับ หน้าออเดอร์แสดงคำสั่งซื้อ และที่อยู่ของลูกค้าถูกอัปเดตตามที่อยู่จัดส่ง
Actual Results	ยอดรวมเท่ากับ 450 บาท สถานะเปลี่ยนตามลำดับ หน้าออเดอร์แสดงคำสั่งซื้อ และที่อยู่ของลูกค้าถูกอัปเดตตามที่อยู่จัดส่ง
Test Status	ผ่าน

จากตารางที่ 4.15 ระบบคำนวณยอดรวมจากราคาและจำนวนได้ถูกต้อง เปลี่ยนสถานะคำสั่งซื้อได้ และอัปเดตที่อยู่ของลูกค้าให้ตรงกัน จึงผ่านการทดสอบ

4.16 ผลการทดสอบการส่งบรอดแคสต์แบบตั้งเวลา

การทดสอบนี้ตรวจการส่งข้อความตามกลุ่มเป้าหมายและเวลา ดังตารางที่ 4.16

ตารางที่ 4.16 การทดสอบการส่งบรอดแคสต์แบบตั้งเวลา

หัวข้อ	คำอธิบาย
Test Case ID	TC 015
Test Scenario	สร้างบรอดแคสต์ถึงลูกค้าที่ติดแท็กที่กำหนด และตั้งเวลาส่งล่วงหน้า
Prerequisites	ผู้ทดสอบมีสิทธิ์บรอดแคสต์ และ Scheduler ทำงานอยู่
Test Data	ข้อความโปรโมชัน กลุ่มเป้าหมายตามแท็ก “ลูกค้าประจำ” และเวลาส่งอีก 5 นาที
Test Steps	1. เปิดหน้าสร้างบรอดแคสต์ กรอกข้อความ เลือกช่องทางและกลุ่มเป้าหมาย 2. ตั้งเวลาส่งแล้วบันทึก 3. รอจนถึงเวลาที่กำหนด แล้วเปิดหน้ารายการบรอดแคสต์

ตารางที่ 4.16 การทดสอบการส่งบรอดแคสต์แบบตั้งเวลา (ต่อ)

หัวข้อ	คำอธิบาย
Expected Results	สถานะเปลี่ยนจากตั้งเวลาไว้เป็นส่งแล้ว ลูกค้าที่ติดแท็กได้รับข้อความ ลูกค้าที่ไม่ได้ติดแท็กไม่ได้รับ และแสดงจำนวนผู้รับกับจำนวนที่ส่งสำเร็จ
Actual Results	เมื่อถึงเวลาที่กำหนด สถานะเปลี่ยนเป็นส่งแล้ว ลูกค้าที่ติดแท็กได้รับข้อความ ลูกค้าที่ไม่ได้ติดแท็กไม่ได้รับ และหน้ารายการแสดงจำนวนผู้รับกับจำนวนที่ส่งสำเร็จ
Test Status	ผ่าน

จากตารางที่ 4.16 ระบบส่งบรอดแคสต์ตามเวลาและกลุ่มเป้าหมายที่กำหนดได้ถูกต้อง และแสดงผลการส่งให้ร้านค้าตรวจสอบได้ จึงผ่านการทดสอบ

4.17 ผลการทดสอบการนำเข้าประวัติแชท

การทดสอบนี้ตรวจการนำเข้าบทสนทนาเดิมของเพจ Facebook ดังตารางที่ 4.17

ตารางที่ 4.17 การทดสอบการนำเข้าประวัติแชท

หัวข้อ	คำอธิบาย
Test Case ID	TC 016
Test Scenario	นำเข้าประวัติแชทของเพจย้อนหลัง 30 วัน และเลือกช่วงเกิน 90 วัน
Prerequisites	เชื่อมต่อเพจที่มีบทสนทนาเดิมแล้ว
Test Data	ช่วงวันที่ 30 วันล่าสุด และช่วงวันที่ 120 วัน
Test Steps	1. เลือกนำเข้าประวัติแชท แล้วเลือกช่วง 30 วัน 2. ติดตามความคืบหน้าจนเสร็จ 3. เลือกช่วงวันที่ 120 วัน
Expected Results	ขั้นที่ 2 บทสนทนาเดิมแสดงในกล่องแชทตามลำดับเวลา ขั้นที่ 3 ระบบไม่ให้เลือกช่วงเกิน 90 วัน
Actual Results	บทสนทนาเดิมในช่วง 30 วันแสดงในกล่องแชทตามลำดับเวลา และระบบไม่ให้เลือกช่วงวันที่เกิน 90 วัน
Test Status	ผ่าน

จากตารางที่ 4.17 ร้านค้านำเข้าประวัติแชทเดิมของเพจได้ และระบบจำกัดช่วงวันที่ไม่ให้เกิน 90 วัน ผลที่ได้ตรงกับผลลัพธ์ที่คาดหวังทุกขั้นตอน จึงผ่านการทดสอบ

4.18 ผลการทดสอบการแจ้งเตือนแชทที่ค้างตอบ

การทดสอบนี้ตรวจการแจ้งเตือนเมื่อลูกค้ารอคำตอบนานเกินกำหนด ดังตารางที่ 4.18

ตารางที่ 4.18 การทดสอบการแจ้งเตือนแชทที่ค้างตอบ

หัวข้อ	คำอธิบาย
Test Case ID	TC 017
Test Scenario	ลูกค้าส่งข้อความแล้วไม่มีเจ้าหน้าที่ตอบเกินเวลาที่กำหนด
Prerequisites	ตั้งเวลาแจ้งเตือนเป็น 5 นาที และตั้งโหมดแชทบอทเป็น Manual
Test Data	ข้อความจากลูกค้าทดสอบ
Test Steps	1. ลูกค้าส่งข้อความ 2. รอ 5 นาทีโดยไม่ตอบ 3. สังเกตการแจ้งเตือนบนหน้าเว็บ 4. รอต่ออีก 5 นาที
Expected Results	ขั้นที่ 3 เจ้าหน้าที่ได้รับการแจ้งเตือนว่ามีแชทค้างตอบ ขั้นที่ 4 ระบบไม่แจ้งเตือนซ้ำสำหรับบทสนทนาเดิม
Actual Results	เจ้าหน้าที่ได้รับการแจ้งเตือนว่ามีแชทค้างตอบเมื่อครบ 5 นาที และระบบไม่แจ้งเตือนซ้ำสำหรับบทสนทนาเดิม
Test Status	ผ่าน

จากตารางที่ 4.18 ระบบแจ้งเตือนเจ้าหน้าที่เมื่อลูกค้ารอคำตอบนานเกินเวลาที่ร้านค้ากำหนด และไม่แจ้งเตือนซ้ำสำหรับบทสนทนาเดิม ผลตรงกับที่คาดหวัง จึงผ่านการทดสอบ

4.19 ผลการทดสอบ WebChat Widget

การทดสอบนี้ตรวจหน้าต่างแชทที่ร้านค้านำไปติดบนเว็บไซต์ของร้าน ดังตารางที่ 4.19

ตารางที่ 4.19 การทดสอบ WebChat Widget

หัวข้อ	คำอธิบาย
Test Case ID	TC 018
Test Scenario	ผู้เยี่ยมชมเว็บไซต์ของร้านเริ่มแชท และเจ้าหน้าที่ตอบกลับ
Prerequisites	สร้าง Widget และนำโค้ดฝังไปวางบนหน้าเว็บทดสอบแล้ว

ตารางที่ 4.19 การทดสอบ WebChat Widget (ต่อ)

หัวข้อ	คำอธิบาย
Test Data	หน้าเว็บทดสอบที่ติดโค้ดฝัง
Test Steps	1. เปิดหน้าเว็บทดสอบแล้วกดไอคอนแชท 2. ส่งข้อความ 3. เจ้าหน้าที่ตอบจากกล่องแชท 4. โหลดหน้าเว็บทดสอบใหม่
Expected Results	ข้อความจากขั้นที่ 2 แสดงในกล่องแชท คำตอบจากขั้นที่ 3 แสดงในหน้าต่างแชททันที และขั้นที่ 4 ยังเห็นประวัติข้อความเดิม
Actual Results	ข้อความจากผู้เยี่ยมชมแสดงในกล่องแชท คำตอบของเจ้าหน้าที่แสดงในหน้าต่างแชททันที และเมื่อโหลดหน้าเว็บใหม่ยังเห็นประวัติข้อความเดิม
Test Status	ผ่าน

จากตารางที่ 4.19 ผู้เยี่ยมชมเว็บไซต์ของร้านแชทกับเจ้าหน้าที่ผ่าน WebChat Widget ได้แบบทันที และประวัติข้อความไม่หายเมื่อโหลดหน้าใหม่

4.20 แบบประเมินความพึงพอใจในการใช้งานระบบเว็บ

ผู้จัดทำให้กลุ่มตัวอย่างจำนวน 16 คน ซึ่งเป็น ผู้ทดลองใช้งานระบบเว็บแล้วทำแบบประเมินความพึงพอใจ โดยแปลผลค่าเฉลี่ยตามเกณฑ์ในตารางที่ 4.20

ตารางที่ 4.20 เกณฑ์การแปลผลค่าเฉลี่ยความพึงพอใจ

ค่าเฉลี่ย	ระดับความพึงพอใจ
4.50–5.00	มากที่สุด
3.50–4.49	มาก
2.50–3.49	ปานกลาง
1.50–2.49	น้อย
1.00–1.49	น้อยที่สุด

ตารางที่ 4.21 ผลการประเมินความพึงพอใจในการใช้งานระบบเว็บ

รายการประเมิน	ค่าเฉลี่ย	S.D.	ระดับ
ความง่ายในการเชื่อมต่อช่องทางแชท	4.56	0.51	มากที่สุด
ความสะดวกในการตอบแชทจากหลายช่องทางในหน้าเดียว	4.31	0.60	มาก
ความรวดเร็วในการแสดงข้อความใหม่	3.94	0.68	มาก
ความถูกต้องของคำตอบจากแชทบอท	4.06	0.77	มาก
ความสะดวกในการเปิดคำสั่งซื้อจากแชท	4.44	0.63	มาก
ความเหมาะสมของการแบ่งสิทธิ์ตามบทบาท	4.25	0.58	มาก
ความชัดเจนของรายงานภาพรวม	4.12	0.62	มาก
ความพึงพอใจโดยรวมต่อระบบ	4.31	0.60	มาก
รวม	4.25	0.64	มาก

จากตารางที่ 4.21 ความพึงพอใจโดยรวมอยู่ในระดับมาก มีค่าเฉลี่ย 4.25 และส่วนเบี่ยงเบนมาตรฐาน 0.64 รายการที่ได้คะแนนสูงสุดคือความง่ายในการเชื่อมต่อช่องทางแชท มีค่าเฉลี่ย 4.56 อยู่ในระดับมากที่สุด เพราะร้านค้าเชื่อมต่อช่องทางได้จากหน้าตั้งค่าเพียงไม่กี่ขั้นตอน ส่วนรายการที่ได้คะแนนต่ำสุดคือความรวดเร็วในการแสดงข้อความใหม่ มีค่าเฉลี่ย 3.94

บทที่ 5
สรุปผลและข้อเสนอแนะ

โครงงานนี้พัฒนาระบบเว็บของ Chatshop ซึ่งรวมแชทจาก LINE Official Account, Facebook Messenger, Instagram และ WebChat ไว้ในกล่องข้อความเดียว พร้อมระบบทีม แชทบอท และระบบจัดการคำสั่งซื้อ บทนี้สรุปผลการดำเนินงานเทียบกับวัตถุประสงค์ ข้อจำกัดของระบบ ปัญหาที่พบระหว่างพัฒนา และข้อเสนอแนะสำหรับการพัฒนาต่อ โดยมีหัวข้อดังนี้

5.1 สรุปผลการดำเนินงาน

5.2 ประโยชน์ที่สถานประกอบการได้รับ

5.3 ข้อจำกัดของระบบ

5.4 ปัญหาและอุปสรรค

5.5 ข้อเสนอแนะ

ซึ่งแต่ละหัวข้อมีรายละเอียดดังต่อไปนี้

5.1 สรุปผลการดำเนินงาน

ผู้จัดทำพัฒนาระบบเว็บด้วย Laravel 10 ใช้ MySQL เก็บข้อมูลเชิงสัมพันธ์ ใช้ MongoDB เก็บบทสนทนา ข้อความ และฐานความรู้ ประมวลผลงานเบื้องหลังผ่านคิวงาน และส่งข้อความใหม่ไปยังหน้าเว็บผ่าน Laravel Reverb ผลการดำเนินงานเทียบกับวัตถุประสงค์ในหัวข้อ 1.3 มีดังนี้

5.1.1 ระบบรับข้อความจากทั้ง 4 ช่องทางผ่าน Webhook ที่ตรวจลายเซ็นทุกครั้ง แล้วแสดงในกล่องข้อความเดียวโดยไม่ต้องโหลดหน้าใหม่ เจ้าหน้าที่ตอบได้ทั้งข้อความ สติกเกอร์ และไฟล์ และค้นหาบทสนทนาย้อนหลังได้ ซึ่งเป็นไปตามวัตถุประสงค์ข้อ 1.3.1

5.1.2 ร้านค้าหลายร้านใช้ระบบเดียวกันได้โดยข้อมูลไม่ปนกัน เจ้าของร้านเชิญสมาชิกมอบหมายบทสนทนา และกำหนดสิทธิ์ตามบทบาท 13 รายการ ซึ่งเป็นไปตามวัตถุประสงค์ข้อ 1.3.2

5.1.3 แชทบอทตอบคำถามจากเอกสารความรู้ของร้านค้าด้วยเทคนิค RAG รวมข้อความที่ลูกค้าส่งมาติดกันเป็นคำถามเดียว และส่งต่อบทสนทนาให้เจ้าหน้าที่เมื่อตอบไม่ได้หรือเมื่อลูกค้าส่งรูปภาพ เสียง วิดีโอ หรือไฟล์ ซึ่งเป็นไปตามวัตถุประสงค์ข้อ 1.3.3

5.1.4 เจ้าหน้าที่จัดการสินค้าและข้อมูลลูกค้า และเปิดคำสั่งซื้อได้จากหน้าแชทโดยตรง ระบบคำนวณยอดรวมและติดตามสถานะคำสั่งซื้อได้ 4 สถานะ ซึ่งเป็นไปตามวัตถุประสงค์ข้อ 1.3.4

5.1.5 หน้าภาพรวมแสดงเวลาตอบครั้งแรก ปริมาณแชทแยกตามช่องทาง และผลงานของเจ้าหน้าที่แต่ละคน และส่งออกเป็นไฟล์ CSV ได้ ซึ่งเป็นไปตามวัตถุประสงค์ข้อ 1.3.5

ผลการทดสอบตามกรณีทดสอบในบทที่ 4 ทั้ง 18 กรณี ผ่านทุกกรณี และผลการประเมินความพึงพอใจของกลุ่มตัวอย่างจำนวน 16 คน มีค่าเฉลี่ยรวมเท่ากับ 4.25 อยู่ในระดับมาก

5.2 ประโยชน์ที่สถานประกอบการได้รับ

ระบบอยู่ระหว่างรอการอนุมัติแอปจาก Meta และรอการตรวจโค้ดก่อนเปิดให้ร้านค้าใช้งานจริง จึงยังไม่มีข้อมูลจากการใช้งานของร้านค้า จึงแบ่งการวัดประโยชน์เป็น 3 ส่วน ได้แก่ ผลที่วัดได้จากการพัฒนาและการทดสอบ มูลค่าที่ประมาณการ และประโยชน์เชิงคุณภาพ ดังนี้

5.2.1 ผลที่วัดได้จากการพัฒนาและการทดสอบ ผู้จัดทำวัดผลจากระบบในสภาพแวดล้อมทดสอบ โดยใช้บัญชีและข้อมูลสำหรับทดสอบชุดเดียวกับบทที่ 4 ดังตารางที่ 5.1

ตารางที่ 5.1 ผลที่วัดได้จากการพัฒนาและการทดสอบ

ตัวชี้วัด	วิธีวัด	ผล
เวลาที่แชทบอทใช้สร้างคำตอบ	ค่าเฉลี่ยจากการเรียกใช้ qwen3:14b ผ่าน Ollama 28 ครั้ง	11.8 วินาทีต่อคำตอบ
ความถูกต้องของคำตอบ	ถามคำถามที่มีในเอกสารความรู้ของร้าน และคำถามที่ไม่มีในเอกสาร	ตอบตรงตามเอกสารความรู้ทุกครั้ง ส่วนคำถามที่ไม่มีในเอกสารส่งต่อให้เจ้าหน้าที่
การตรวจสลิปโอนเงิน	เทียบกับการตรวจสลิปด้วยตนเอง	ระบบตรวจยอดเงิน บัญชีผู้รับ และสลิปซ้ำให้ทันที โดยไม่ต้องรอเจ้าหน้าที่
การย่อขนาดไฟล์	ขอบเขตที่ระบบทำ	รูปภาพย่อด้านยาวไม่เกิน 2,000 พิกเซล วิดีโอแปลงเป็น MP4 ด้านยาวไม่เกิน 1,280 พิกเซล และเก็บไฟล์ที่เล็กกว่าเสมอ
ผลการทดสอบระบบ	ทดสอบตามกรณีทดสอบในบทที่ 4	ผ่าน 18 จาก 18 กรณี

จากตารางที่ 5.1 ตัวชี้วัดทั้งหมดวัดจากระบบในสภาพแวดล้อมทดสอบ ซึ่งยังไม่มีร้านค้าใช้งานจริง ผู้จัดทำจึงใช้ผลเหล่านี้เป็นข้อมูลตั้งต้นในการประมาณการมูลค่าในหัวข้อถัดไป

5.2.2 ประมาณการมูลค่าเมื่อเปิดใช้งานจริง ผู้จัดทำประมาณการมูลค่าจากจำนวนธุรกิจที่คาดว่าจะใช้งานระบบ ตามสมมติฐานในตารางที่ 5.2

ตารางที่ 5.2 สมมติฐานที่ใช้ประมาณการมูลค่า

สมมติฐาน	ค่าที่ใช้	ที่มา
ธุรกิจที่ใช้งานระบบ	10–12 ธุรกิจ	ประมาณการของสถานประกอบการ
เพจต่อธุรกิจ	2–3 เพจ	ประมาณการของสถานประกอบการ
บทสนทนาต่อเพจต่อวัน	20 บทสนทนา	สมมติฐานของผู้จัดทำ
เงินเดือนแอดมินตอบแชท	13,000–18,000 บาทต่อเดือน หรือ 62.50–86.54 บาทต่อชั่วโมง	ประมาณการของสถานประกอบการ คิด 26 วันต่อเดือน วันละ 8 ชั่วโมง
เวลาทำงานของแอดมิน	1 กะ 8 ชั่วโมงต่อวัน	เวลาทำงานปกติ

จากตารางที่ 5.2 ทั้ง 10–12 ธุรกิจมีเพจรวม 20–36 เพจ คิดเป็นบทสนทนาประมาณ 12,000–21,600 บทสนทนาต่อเดือน หากร้านค้าต้องการตอบลูกค้าได้ตลอด 24 ชั่วโมงด้วยเจ้าหน้าที่ ต้องจ้างแอดมินเพิ่มอีก 2 กะเพื่อดูแลช่วงนอกเวลาทำงาน 16 ชั่วโมงต่อวัน แชทบอทตอบคำถามที่พบบ่อยในช่วงเวลานั้นแทนได้ จึงคิดมูลค่าเทียบเท่าจากเงินเดือนแอดมิน 2 คน ดังตารางที่ 5.3

ตารางที่ 5.3 มูลค่าเทียบเท่าค่าจ้างแอดมินช่วงนอกเวลาทำงาน

รายการ	ต่อธุรกิจ	รวม 10–12 ธุรกิจ
ต่อเดือน	26,000–36,000 บาท	260,000–432,000 บาท
ต่อปี	312,000–432,000 บาท	3,120,000–5,184,000 บาท

จากตารางที่ 5.3 แชทบอทมีมูลค่าเทียบเท่าค่าจ้างแอดมินประมาณ 3.1–5.2 ล้านบาทต่อปีเมื่อรวมทุกธุรกิจ ตัวเลขนี้เป็นค่าสูงสุด เพราะแชทบอทตอบได้เฉพาะคำถามที่มีในเอกสารความรู้ คำถามอื่นยังต้องรอเจ้าหน้าที่ในเวลาทำงาน จึงควรวัดซ้ำเมื่อระบบเปิดใช้งานจริง

5.2.3 ประโยชน์เชิงคุณภาพ ประโยชน์ที่แปลงเป็นตัวเงินได้ยาก แต่เกิดจากความสามารถของระบบโดยตรง และเกิดขึ้นได้ทันทีเมื่อร้านค้าเริ่มใช้งาน มีดังนี้

5.2.3.1 ร้านค้าตอบคำถามที่พบบ่อยได้ตลอด 24 ชั่วโมงผ่านแชทบอท ลูกค้าที่ทักมานอกเวลาทำการจึงได้รับคำตอบทันทีโดยไม่ต้องรอเจ้าหน้าที่

5.2.3.2 แชทตกหล่นและการตอบซ้ำลดลง เพราะเจ้าหน้าที่มอบหมายและล็อกบทสนทนาได้ และระบบแจ้งเตือนแชทที่ค้างตอบนานเกินกำหนด

5.2.3.3 ตรวจสลิปได้เร็วกว่าการตรวจด้วยตนเอง และความเสี่ยงจากสลิปปลอมหรือสลิปที่ใช้ซ้ำลดลง เพราะระบบตรวจยอดเงิน บัญชีผู้รับ และการใช้สลิปซ้ำก่อนเปลี่ยนสถานะคำสั่งซื้อ โดยมีค่าตรวจจากบริการตรวจสลิปเพียง 0.40 บาทต่อรายการ

5.2.3.4 เจ้าหน้าที่ไม่ต้องออกจากระบบไปไล่ตอบแชทในแอปพลิเคชันของแต่ละช่องทาง เพราะข้อมูลลูกค้า ประวัติแชท และคำสั่งซื้อจากทุกช่องทางอยู่ในระบบเดียว

5.2.3.5 แชทบอทใช้ LLM บนเซิร์ฟเวอร์ของบริษัท ข้อมูลบทสนทนาของลูกค้าจึงไม่ถูกส่งไปยังผู้ให้บริการภายนอก และไม่มีค่าบริการที่เพิ่มขึ้นตามจำนวนข้อความ

5.2.3.6 บริษัทได้ระบบที่พร้อมนำไปให้บริการร้านค้าออนไลน์ และพัฒนาต่อยอดได้จากโครงสร้างที่รองรับหลายร้านค้าในระบบเดียว

5.3 ข้อจำกัดของระบบ

ระบบที่พัฒนาขึ้นยังมีข้อจำกัดที่ควรทราบก่อนนำไปใช้งานจริง ดังนี้

5.3.1 คำสั่งซื้อหนึ่งรายการมีสินค้าได้เพียง 1 ชนิด โดยกำหนดจำนวนได้ หากลูกค้าซื้อสินค้าหลายชนิด เจ้าหน้าที่ต้องเปิดคำสั่งซื้อแยกตามชนิดสินค้า

5.3.2 ระบบไม่มีการรับชำระเงินออนไลน์ ลูกค้าต้องโอนเงินแล้วส่งสลิปในแชท หากร้านค้าไม่ได้เปิดการตรวจสลิปอัตโนมัติ หรือแชทนั้นมีคำสั่งซื้อที่รอชำระมากกว่า 1 รายการ เจ้าหน้าที่ต้องตรวจสลิปและเปลี่ยนสถานะคำสั่งซื้อด้วยตนเองจากหน้าแชท

5.3.3 การค้นคืนเอกสารความรู้เปรียบเทียบเวกเตอร์ของคำถามกับทุกส่วนย่อยของเอกสาร ซึ่งเหมาะกับฐานความรู้ขนาดเล็กถึงกลางของร้านค้าทั่วไป หากเอกสารมีจำนวนมาก เวลาที่ใช้ค้นคืนจะเพิ่มขึ้นตามจำนวนส่วนย่อยที่ต้องเปรียบเทียบทั้งหมด

5.3.4 การนำเข้าประวัติแชทรองรับเฉพาะ Facebook และ Instagram ย้อนหลังได้ไม่เกิน 90 วัน เพราะ LINE Messaging API ไม่มีช่องทางให้ดึงประวัติข้อความเดิม

5.3.5 Meta อนุญาตให้ส่งข้อความหาลูกค้าได้ภายใน 24 ชั่วโมงหลังข้อความล่าสุดของลูกค้า หลังจากนั้นระบบส่งได้เฉพาะคำตอบของเจ้าหน้าที่ แชทบอทและบรอดแคสต์จึงส่งถึงลูกค้าที่เงียบไปนานบน Facebook และ Instagram ไม่ได้

5.3.6 ระบบทำงานโดยอาศัย API ของ LINE และ Meta หากแพลตฟอร์มเปลี่ยนเงื่อนไขหรือเกิดเหตุขัดข้อง การรับส่งข้อความของช่องทางนั้นจะได้รับผลกระทบ

5.4 ปัญหาและอุปสรรค

ระหว่างการพัฒนาระบบ ผู้จัดทำพบปัญหาและอุปสรรค ดังนี้

5.4.1 การขออนุมัติแอปจาก Meta สิทธิ์ที่ใช้เชื่อมต่อเพจและ Instagram เป็นสิทธิ์ระดับ Advanced Access ระหว่างที่แอปอยู่ในโหมดพัฒนา ระบบรับข้อความได้เฉพาะจากบัญชีที่มีบทบาทในแอป การทดสอบจึงต้องเพิ่มบัญชีผู้ทดสอบทีละบัญชี และการเปิดให้ร้านค้าทั่วไปใช้งานต้องผ่าน App Review และ Business Verification ซึ่งใช้เอกสารนิติบุคคลของบริษัท

5.4.2 การเข้าสู่ระบบด้วย Facebook ในช่วงแรกไม่ได้ขอสิทธิ์ email ระบบจึงไม่ได้รับอีเมลของผู้ใช้และสร้างร้านค้าใหม่ทุกครั้งที่เข้าสู่ระบบ ผู้จัดทำแก้ไขโดยเพิ่มสิทธิ์ email ในแอป และแยกแอปที่ใช้เข้าสู่ระบบออกจากแอปที่ใช้เชื่อมต่อเพจ

5.4.3 รูปภาพและไฟล์ที่ลูกค้าส่งผ่าน LINE ไม่มี URL สาธารณะ หน้าเว็บจึงแสดงรูปจาก LINE โดยตรงไม่ได้ ผู้จัดทำแก้ไขโดยให้เซิร์ฟเวอร์ดึงไฟล์มาเก็บไว้ในระบบ และย่อขนาดรูปภาพและวิดีโอก่อนจัดเก็บ เพื่อประหยัดพื้นที่จัดเก็บและให้หน้าเว็บโหลดรูปได้เร็วขึ้น

5.4.4 ลูกค้ามักพิมพ์คำถามแยกเป็นหลายข้อความติดกัน ทำให้แชทบอทตอบทีละข้อความและตอบซ้ำกัน ผู้จัดทำแก้ไขโดยให้ระบบรอสักครู่เพื่อรวมข้อความที่ส่งมาติดกันเป็นคำถามเดียว และป้องกันการตอบซ้ำเมื่องานเดียวกันถูกประมวลผลมากกว่าหนึ่งครั้ง

5.5 ข้อเสนอแนะ

จากข้อจำกัดและปัญหาที่พบ ผู้จัดทำมีข้อเสนอแนะสำหรับการพัฒนาต่อ ดังนี้

5.5.1 พัฒนาคำสั่งซื้อให้รองรับสินค้าหลายชนิดในรายการเดียว และเชื่อมต่อระบบรับชำระเงินออนไลน์ เพื่อลดงานตรวจสลิปของเจ้าหน้าที่

5.5.2 เปลี่ยนการค้นคืนเอกสารความรู้ไปใช้ฐานข้อมูลเวกเตอร์ เมื่อร้านค้ามีเอกสารความรู้จำนวนมาก เพื่อให้เวลาค้นคืนไม่เพิ่มขึ้นตามขนาดของเอกสาร

5.5.3 เพิ่มช่องทางการขายอื่นที่ร้านค้าออนไลน์ในประเทศไทยนิยมใช้ เช่น TikTok และแพลตฟอร์มตลาดออนไลน์ เพื่อให้กล่องข้อความครอบคลุมทุกช่องทางของร้าน

5.5.4 ให้บรอดแคสต์ส่งรูปภาพและข้อความแบบมีปุ่มได้ นอกเหนือจากข้อความธรรมดา เพื่อให้ร้านค้าประชาสัมพันธ์สินค้าได้น่าสนใจขึ้น

5.5.5 แปลงไฟล์ JSX ของหน้ากล่องข้อความล่วงหน้าในขั้นตอน build แทนการแปลงบนเบราว์เซอร์ทุกครั้งที่เปิดหน้า เพื่อให้หน้ากล่องข้อความโหลดเร็วขึ้น

5.5.6 วัดผลซ้ำเมื่อระบบเปิดใช้งานจริง โดยเทียบเวลาตอบครั้งแรกและจำนวนข้อความที่แชทบอทตอบแทนเจ้าหน้าที่ก่อนและหลังใช้งาน เพื่อยืนยันมูลค่าที่ประมาณการไว้ในหัวข้อ 5.2.2

บรรณานุกรม

[1] สำนักงานพัฒนาธุรกรรมทางอิเล็กทรอนิกส์, “ETDA เผยมูลค่าอีคอมเมิร์ซไทย ปี 66 พุ่งแตะ 5.96 ล้านล้านบาท อุตสาหกรรมประกันภัย โตมากสุด ร้อยละ 31,” [ออนไลน์]. ETDA เว็บไซต์: https://www.etda.or.th/th/pr-news/Dgt_ecom_survey2024.aspx (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[2] LY Corporation, “Messaging API overview,” [ออนไลน์]. LINE Developers เว็บไซต์: https://developers.line.biz/en/docs/messaging-api/overview/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[3] LY Corporation, “Receive messages (webhook),” [ออนไลน์]. LINE Developers เว็บไซต์: https://developers.line.biz/en/docs/messaging-api/receiving-messages/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[4] Meta Platforms, “ภาพรวม - API กราฟ,” [ออนไลน์]. Meta for Developers เว็บไซต์: https://developers.facebook.com/docs/graph-api/overview (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[5] Meta Platforms, “Meta Webhooks for Messenger Platform,” [ออนไลน์]. Meta for Developers เว็บไซต์: https://developers.facebook.com/docs/messenger-platform/webhooks (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[6] M. Stauffer, Laravel: Up & Running, 3rd ed. Sebastopol, CA: O'Reilly Media, 2023.

[7] Laravel, “Eloquent: Getting Started,” [ออนไลน์]. Laravel 10.x Documentation เว็บไซต์: https://laravel.com/docs/10.x/eloquent (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[8] Laravel, “Blade Templates,” [ออนไลน์]. Laravel 10.x Documentation เว็บไซต์: https://laravel.com/docs/10.x/blade (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[9] Meta Open Source, “Thinking in React,” [ออนไลน์]. React เว็บไซต์: https://react.dev/learn/thinking-in-react (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[10] Laravel, “Queues,” [ออนไลน์]. Laravel 10.x Documentation เว็บไซต์: https://laravel.com/docs/10.x/queues (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[11] Laravel, “Task Scheduling,” [ออนไลน์]. Laravel 10.x Documentation เว็บไซต์: https://laravel.com/docs/10.x/scheduling (เข้าถึงเมื่อ: 13 ตุลาคม2569).

บรรณานุกรม (ต่อ)

[12] Laravel, “Broadcasting,” [ออนไลน์]. Laravel 10.x Documentation เว็บไซต์: https://laravel.com/docs/10.x/broadcasting (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[13] I. Fette and A. Melnikov, “The WebSocket Protocol,” Internet Engineering Task Force (IETF), RFC 6455, 2011. https://doi.org/10.17487/RFC6455

[14] Amazon Web Services, “MySQL คืออะไร - อธิบายฐานข้อมูลแบบเชิงสัมพันธ์ MySQL,” [ออนไลน์]. AWS เว็บไซต์: https://aws.amazon.com/th/rds/mysql/what-is-mysql/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[15] E. F. Codd, “A relational model of data for large shared data banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970. https://doi.org/10.1145/362384.362685

[16] โอภาส เอี่ยมสิริวงศ์, ระบบฐานข้อมูล (Database Systems) ฉบับปรับปรุงเพิ่มเติม. กรุงเทพฯ: ซีเอ็ดยูเคชั่น, 2558.

[17] MongoDB, “Documents,” [ออนไลน์]. MongoDB Docs เว็บไซต์: https://www.mongodb.com/docs/manual/core/document/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[18] M. Fowler, “Polyglot Persistence,” [ออนไลน์]. martinfowler.com เว็บไซต์: https://martinfowler.com/bliki/PolyglotPersistence.html (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[19] Amazon Web Services, “ฐานข้อมูล NoSQL คืออะไร - คำอธิบายฐานข้อมูลที่ไม่ใช่เชิงสัมพันธ์,” [ออนไลน์]. AWS เว็บไซต์: https://aws.amazon.com/th/nosql/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[20] MongoDB, “What is MongoDB?,” [ออนไลน์]. MongoDB Docs เว็บไซต์: https://www.mongodb.com/docs/manual/introduction/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[21] Microsoft, “Multitenant SaaS Patterns - Azure SQL Database,” [ออนไลน์]. Microsoft Learn เว็บไซต์: https://learn.microsoft.com/en-us/azure/azure-sql/database/saas-tenancy-app-design-patterns (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

บรรณานุกรม (ต่อ)

[22] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” Computer, vol. 29, no. 2, pp. 38–47, 1996. https://doi.org/10.1109/2.485845

[23] D. Hardt, “The OAuth 2.0 Authorization Framework,” Internet Engineering Task Force (IETF), RFC 6749, 2012. https://doi.org/10.17487/RFC6749

[24] Amazon Web Services, “RESTful API คืออะไร - คำอธิบายเกี่ยวกับ RESTful API,” [ออนไลน์]. AWS เว็บไซต์: https://aws.amazon.com/th/what-is/restful-api/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[25] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, CA, 2000.

[26] Amazon Web Services, “แชทบอทคืออะไร - อธิบายแชทบอท AI,” [ออนไลน์]. AWS เว็บไซต์: https://aws.amazon.com/th/what-is/chatbot/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[27] J. Weizenbaum, “ELIZA—a computer program for the study of natural language communication between man and machine,” Communications of the ACM, vol. 9, no. 1, pp. 36–45, 1966. https://doi.org/10.1145/365153.365168

[28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in Advances in Neural Information Processing Systems, vol. 30, 2017, pp. 5998–6008.

[29] T. Brown et al., “Language models are few-shot learners,” in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 1877–1901.

[30] Amazon Web Services, “LLM คืออะไร - คำอธิบายโมเดลภาษาขนาดใหญ่,” [ออนไลน์]. AWS เว็บไซต์: https://aws.amazon.com/th/what-is/large-language-model/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[31] Ollama, “OpenAI compatibility,” [ออนไลน์]. Ollama Docs เว็บไซต์: https://docs.ollama.com/api/openai-compatibility (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[32] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

บรรณานุกรม (ต่อ)

[33] Amazon Web Services, “RAG คืออะไร - คำอธิบาย Retrieval-Augmented Generation AI,” [ออนไลน์]. AWS เว็บไซต์: https://aws.amazon.com/th/what-is/retrieval-augmented-generation/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[34] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” arXiv:1301.3781, 2013. https://doi.org/10.48550/arXiv.1301.3781

[35] Amazon Web Services, “การฝังคืออะไร - อธิบายการฝังในแมชชีนเลิร์นนิง,” [ออนไลน์]. AWS เว็บไซต์: https://aws.amazon.com/th/what-is/embeddings-in-machine-learning/ (เข้าถึงเมื่อ: 13 ตุลาคม 2569).

[36] พัชรา วงค์ตาผา, “การพัฒนาระบบแชทบอทสำหรับการค้นคว้าปริญญานิพนธ์ ระดับปริญญาตรี ของห้องสมุดคณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี,” รายงานการวิจัย, คณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี, ปทุมธานี, 2566.

[37] ธนพล วุฒิธาดา และ วชิรวิทย์ ชื่นจิตร, “ระบบจัดการสินค้าสำหรับผู้ผลิตรายย่อยกรณีศึกษาบ้านฟาร์มขนม,” ปริญญานิพนธ์ วศ.บ. (วิศวกรรมคอมพิวเตอร์), มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตขอนแก่น, ขอนแก่น, 2569.

[38] มนต์ทิชา รัตนพันธ์ และ ฉัตรวดี สายใยทอง, “การพัฒนา Line Chatbot สำหรับงานบริการด้านงานวิจัย กรณีศึกษาสถาบันวิจัยและพัฒนา มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี,” Journal of Applied Research on Science and Technology, ปีที่ 22, ฉบับที่ 1, หน้า 78–89, 2566. https://doi.org/10.14456/jarst.2023.8

[39] ชรัช แสวงเจริญ และ ชุติมา พิศาลย์, “การใช้ LINE Chatbot สำหรับการแจ้งซ่อมโสตทัศนูปกรณ์ งานเทคโนโลยีการศึกษา มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ,” วารสารวิทยาศาสตร์และเทคโนโลยี มทร.สุวรรณภูมิ, ปีที่ 9, ฉบับที่ 3, หน้า 76–84, 2568.

[40] วิสิฏฐา ชุนสิทธิ์ และ รังสรรค์ เกียรติ์ภานนท์, “การออกแบบแชทบอทให้บริการลูกค้าเพื่อรองรับการขยายตลาดด้วยการตลาดดิจิทัล กรณีศึกษา บริษัท ฟอร์มูลา ทีเคดี จำกัด,” วารสารมหาวิทยาลัยราชภัฏมหาสารคาม, ปีที่ 15, ฉบับที่ 3, หน้า 107–120, 2564.

บรรณานุกรม (ต่อ)

[41] สิปปกร พงศ์เดโชธนบดี, พิชัย ระเวงวัน, ดนัย ศรีษะนอก และ วิจิตร เดชทะสอน, “การพัฒนาระบบแชทบอทเพื่อการสื่อสารและให้บริการข้อมูลแก่ลูกค้า: กรณีศึกษา บริษัท บัณฑิตเวิลด์ จำกัด,” วารสารวิชาการและวิจัย มหาวิทยาลัยภาคตะวันออกเฉียงเหนือ, ปีที่ 16, ฉบับที่ 2, หน้า 1–15, 2569.

[42] ณัฐภัทร สุวรรณศิลป์, ปิยะดา กันตรัตนากุล และ กฤติกา สังขวดี, “การพัฒนาระบบซื้อขายสินค้าออนไลน์ กรณีศึกษา: ร้านกลับตานีสาขาพิษณุโลก,” วารสารร่มยูงทอง, ปีที่ 1, ฉบับที่ 2, หน้า 36–52, 2566.

[43] S. Ghosh, S. Ness, and S. Salunkhe, “The role of AI enabled chatbots in omnichannel customer service,” Journal of Engineering Research and Reports, vol. 26, no. 6, pp. 327–345, 2024. https://doi.org/10.9734/jerr/2024/v26i61184

ภาคผนวก

ภาคผนวก ก
คู่มือการใช้งานระบบเว็บ Chatshop

ภาคผนวก ก
คู่มือการใช้งานระบบเว็บ Chatshop

คู่มือนี้อธิบายขั้นตอนการใช้งานระบบเว็บของ Chatshop สำหรับเจ้าของร้านและเจ้าหน้าที่ ตั้งแต่การสมัครใช้งาน การเชื่อมต่อช่องทาง การตอบแชท การตั้งค่าแชทบอท ไปจนถึงการดูรายงานสรุปผล โดยแต่ละหัวข้อมีภาพหน้าจอประกอบ ดังนี้

ก.1 ข้อกำหนดและผู้ใช้งานระบบ

ก.2 การสมัครใช้งานและเข้าสู่ระบบ

ก.3 การเชื่อมต่อช่องทางแชท

ก.4 การตอบแชทในกล่องแชท

ก.5 ทีมงานและสิทธิ์การใช้งาน

ก.6 การตั้งค่าการตอบกลับ

ก.7 การตั้งค่าแชทบอท

ก.8 สินค้า ลูกค้า และคำสั่งซื้อ

ก.9 การส่งบรอดแคสต์

ก.10 การนำเข้าประวัติแชท

ก.11 การแจ้งเตือนและรายงานสรุปผล

ซึ่งแต่ละหัวข้อมีรายละเอียดดังต่อไปนี้

ก.1 ข้อกำหนดและผู้ใช้งานระบบ

ระบบเว็บของ Chatshop ใช้งานผ่านเว็บเบราว์เซอร์ เช่น Google Chrome, Microsoft Edge หรือ Safari บนคอมพิวเตอร์ที่เชื่อมต่ออินเทอร์เน็ต โดยไม่ต้องติดตั้งโปรแกรมเพิ่ม ผู้ใช้งานแบ่งตามบทบาทที่ร้านค้ากำหนด ได้แก่ Owner เข้าถึงได้ทุกส่วน Manager ตอบแชท มอบหมายแชท และดูรายงาน Admin ตอบแชท Agent ตอบแชทและสร้างคำสั่งซื้อ และ Viewer ดูรายงานได้อย่างเดียว ผู้ใช้หนึ่งคนมีบทบาทเดียวในแต่ละร้านค้า

เมนูหลักอยู่ที่แถบด้านซ้ายของหน้าจอ ได้แก่ กล่องแชท ออเดอร์ ภาพรวม ทีมงาน ตั้งค่า สินค้า ลูกค้า และบรอดแคสต์ โดยเมนูที่ผู้ใช้ไม่มีสิทธิ์จะไม่แสดง แถบเมนูหลักแสดงดังรูปที่ ก.1



รูปที่ ก.1 แถบเมนูหลักของระบบเว็บ

ก.2 การสมัครใช้งานและเข้าสู่ระบบ

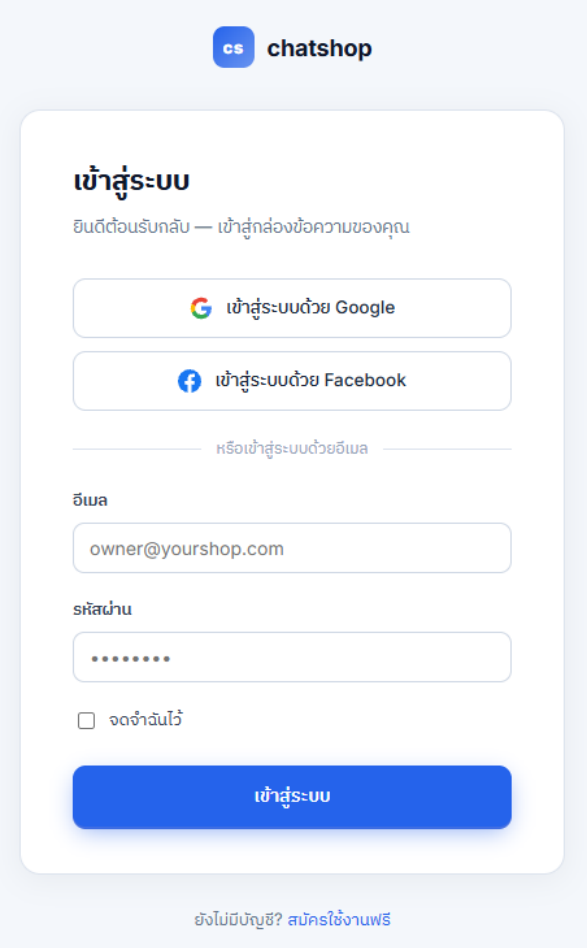
ก.2.1 การสมัครใช้งานด้วยอีเมล มีขั้นตอนดังนี้

1) เปิดหน้าเว็บของระบบ แล้วคลิก “สมัครใช้งานฟรี” ใต้ฟอร์มเข้าสู่ระบบ

2) กรอกชื่อร้านค้า อีเมล รหัสผ่านอย่างน้อย 8 ตัวอักษร และยืนยันรหัสผ่าน

3) คลิก “สร้างบัญชี” ระบบจะสร้างร้านค้าและพาเข้าสู่หน้ากล่องแชท

หน้าสมัครใช้งานแสดงดังรูปที่ ก.2



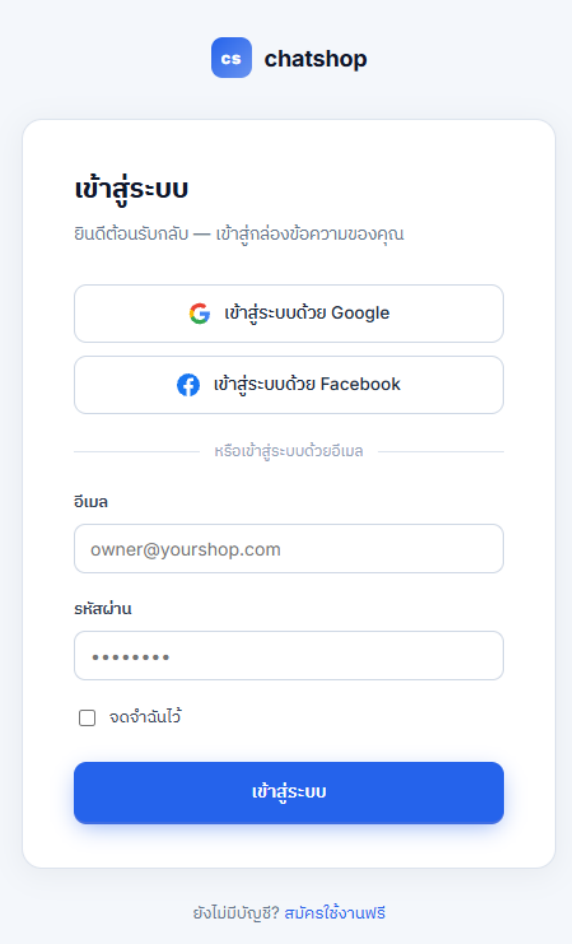
รูปที่ ก.2 หน้าสมัครใช้งาน

ก.2.2 การเข้าสู่ระบบ มีขั้นตอนดังนี้

1) กรอกอีเมลและรหัสผ่าน แล้วคลิก “เข้าสู่ระบบ” หากต้องการให้ระบบจำการเข้าสู่ระบบไว้ ให้เลือก “จดจำฉันไว้” ก่อนคลิก ครั้งต่อไปจะไม่ต้องกรอกรหัสผ่านใหม่

2) หากใช้บัญชี Google หรือ Facebook ให้คลิก “เข้าสู่ระบบด้วย Google” หรือ “เข้าสู่ระบบด้วย Facebook” แล้วอนุญาตสิทธิ์ในหน้าของผู้ให้บริการ ผู้ใช้ใหม่จะได้ร้านค้าที่ตั้งชื่อตามชื่อบัญชีโดยอัตโนมัติ ซึ่งแก้ไขชื่อได้ภายหลังในหน้าตั้งค่า

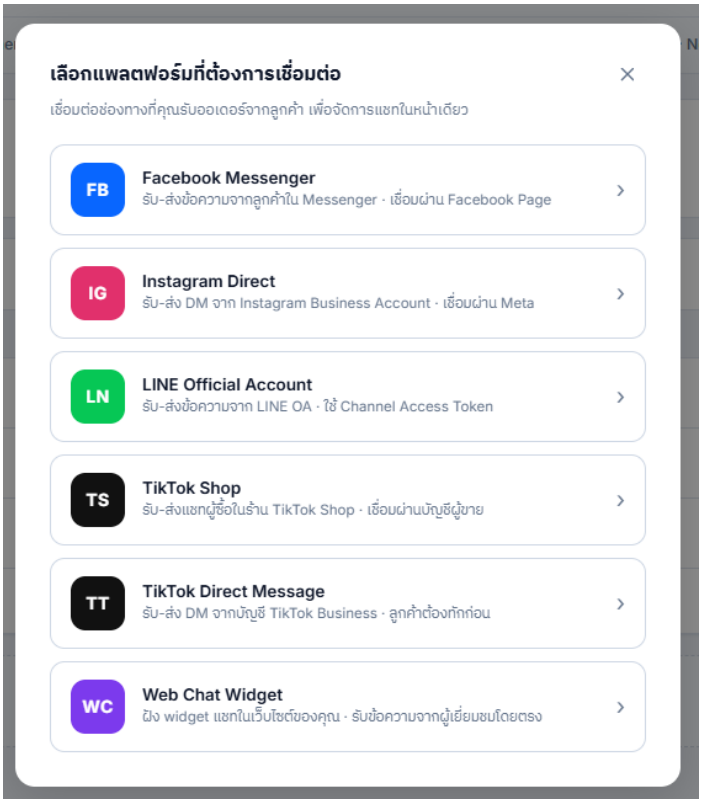
3) หากกรอกข้อมูลผิด ระบบจะแสดงข้อความ “อีเมลหรือรหัสผ่านไม่ถูกต้อง” หน้าเข้าสู่ระบบแสดงดังรูปที่ ก.3



รูปที่ ก.3 หน้าเข้าสู่ระบบ

ก.3 การเชื่อมต่อช่องทางแชท

การเชื่อมต่อช่องทางทำที่เมนู ตั้งค่า แท็บ “ช่องทาง · Channels” โดยคลิก “+ เพิ่มช่องทาง” แล้วเลือกแพลตฟอร์มที่ต้องการ ช่องทางที่เชื่อมต่อแล้วแสดงในตาราง “ช่องทางที่เชื่อมต่อ” พร้อมสถานะการใช้งาน และยกเลิกการเชื่อมต่อได้ด้วยปุ่ม × ในแถวของช่องทาง ร้านค้าหนึ่งร้านเชื่อมต่อได้หลายช่องทาง หน้าต่างเลือกแพลตฟอร์มแสดงดังรูปที่ ก.4



รูปที่ ก.4 หน้าต่างเลือกแพลตฟอร์มที่ต้องการเชื่อมต่อ

ก.3.1 การเชื่อมต่อ LINE Official Account มีขั้นตอนดังนี้

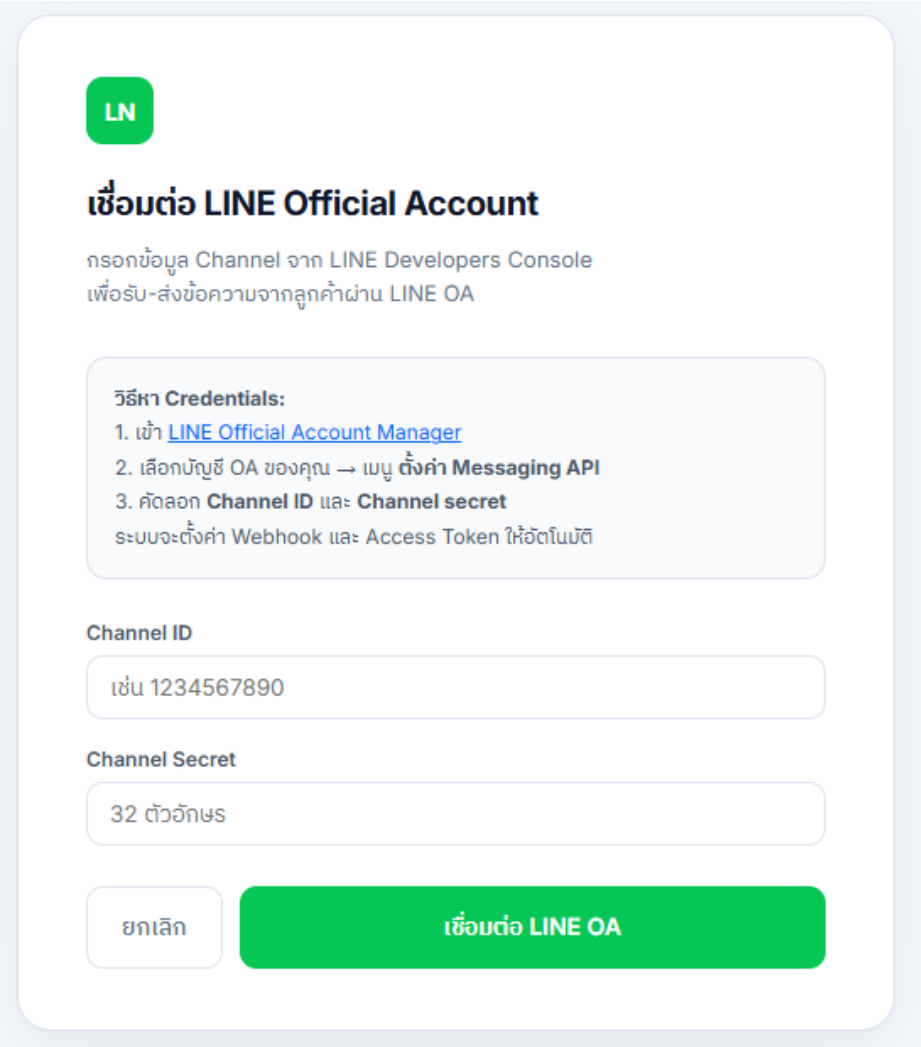
1) เปิด LINE Official Account Manager เลือกบัญชีที่ต้องการ แล้วไปที่เมนู “ตั้งค่า Messaging API” เพื่อคัดลอก Channel ID และ Channel secret

2) เลือก LINE Official Account ในระบบ แล้วกรอก Channel ID และ Channel Secret ที่คัดลอกไว้ลงในฟอร์มให้ครบถ้วน

3) คลิก “เชื่อมต่อ LINE OA” ระบบจะตั้งค่า Webhook ให้อัตโนมัติ

4) ดู Webhook URL ได้ภายหลังจากปุ่ม “Credential” ในแถวของ LINE

หน้าเชื่อมต่อ LINE Official Account แสดงดังรูปที่ ก.5

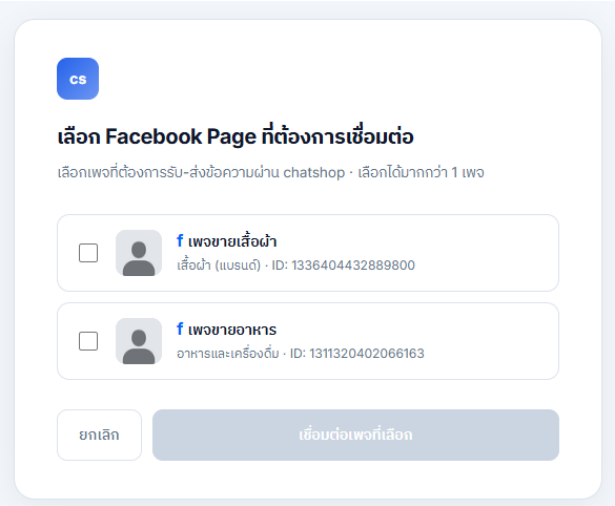


รูปที่ ก.5 หน้าเชื่อมต่อ LINE Official Account

ก.3.2 การเชื่อมต่อ Facebook Page มีขั้นตอนดังนี้

1) เลือก Facebook Messenger ระบบจะเปิดหน้าขอสิทธิ์ของ Facebook ให้เข้าสู่ระบบและอนุญาตสิทธิ์ที่ระบบร้องขอ

2) เลือกเพจที่ต้องการ ได้มากกว่า 1 เพจ แล้วคลิก “เชื่อมต่อเพจที่เลือก” หน้าเลือกเพจแสดงดังรูปที่ ก.6

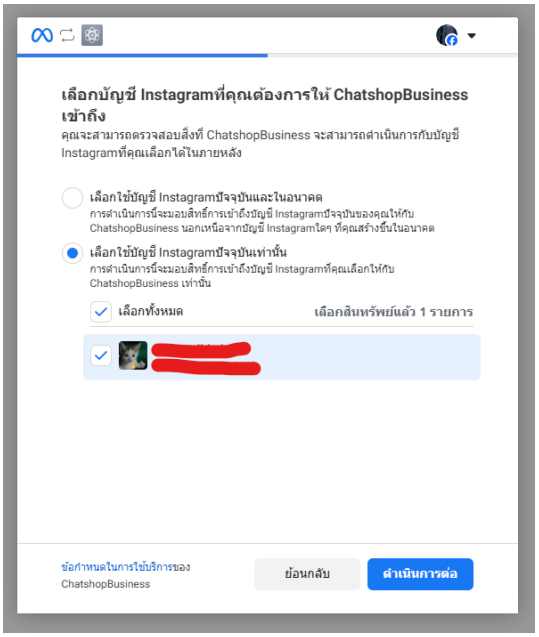


รูปที่ ก.6 หน้าเลือก Facebook Page ที่ต้องการเชื่อมต่อ

ก.3.3 การเชื่อมต่อ Instagram มีขั้นตอนดังนี้

1) เลือก Instagram Direct แล้วอนุญาตสิทธิ์ในหน้าของ Meta

2) เลือกบัญชี Instagram Business ที่ผูกกับ Facebook Page แล้วคลิก “เชื่อมต่อบัญชีที่เลือก” หากไม่พบบัญชี ให้ผูก Instagram กับ Facebook Page ก่อน หน้าเลือกบัญชี Instagram แสดงดังรูปที่ ก.7



รูปที่ ก.7 หน้าเลือก Instagram ที่ต้องการเชื่อมต่อ

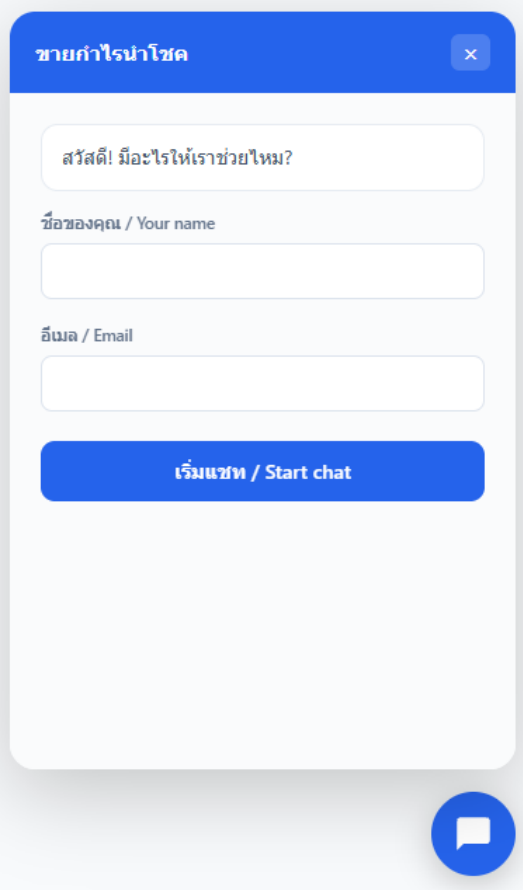
ก.3.4 การติดตั้ง WebChat บนเว็บไซต์ มีขั้นตอนดังนี้

1) เลือก Web Chat Widget กรอกชื่อหน้าต่าง ข้อความต้อนรับ และสีหลัก

2) คลิก “สร้าง Widget” ระบบจะแสดงโค้ดสำหรับฝังในเว็บไซต์

3) คลิก “คัดลอกโค้ด” แล้ววางโค้ดก่อนแท็กปิด body ของหน้าเว็บไซต์ร้านค้า หากต้องการคัดลอกโค้ดอีกครั้ง ให้คลิกปุ่ม “Embed” ในแถวของช่องทาง

หน้าต่างโค้ดฝัง WebChat แสดงดังรูปที่ ก.8



รูปที่ ก.8 หน้าต่างโค้ดฝัง Web Chat Widget

ก.4 การตอบแชทในกล่องแชท

เมนูกล่องแชทรวมบทสนทนาจากทุกช่องทางไว้ในหน้าเดียว ประกอบด้วยรายการแชทด้านซ้าย หน้าบทสนทนาตรงกลาง และแผงข้อมูลลูกค้าด้านขวา แสดงดังรูปที่ ก.9



รูปที่ ก.9 หน้ากล่องแชท

ก.4.1 การค้นหาและกรองแชท มีขั้นตอนดังนี้

1) พิมพ์ชื่อลูกค้า ข้อความ หรือเลขออเดอร์ในช่องค้นหาด้านบน

2) กรองแชทตามแพลตฟอร์ม หรือตามเพจด้วย “เพจ · Pages”

3) คลิก “ดูแชทที่เก็บถาวร” เพื่อดูแชทที่ปิดไปแล้ว

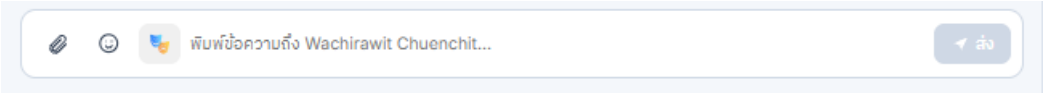
ก.4.2 การตอบข้อความ มีขั้นตอนดังนี้

1) คลิกแชทที่ต้องการ แล้วพิมพ์ข้อความในช่องพิมพ์ด้านล่าง หรือคลิก “ตอบเร็ว” ที่ส่วนหัวของแชทเพื่อเลือกข้อความตอบกลับสำเร็จรูป

2) แนบรูปภาพ วิดีโอ หรือเอกสารด้วยไอคอนคลิปหนีบกระดาษ หรือลากไฟล์มาวางในหน้าแชท และส่งสติกเกอร์ด้วยปุ่มสติกเกอร์

3) คลิก “ส่ง” ระบบจะส่งข้อความไปยังลูกค้าผ่านช่องทางเดิมที่ลูกค้าทักมา

4) หากแชทมอบหมายให้คนอื่นแล้ว ต้องคลิก “ยินยอมเพื่อตอบ” ก่อนตอบช่องพิมพ์ข้อความและการแนบไฟล์แสดงดังรูปที่ ก.10



รูปที่ ก.10 ช่องพิมพ์ข้อความและการแนบไฟล์

ก.4.3 การติดแท็ก มอบหมาย และปิดแชท มีขั้นตอนดังนี้

1) ที่แผงข้อมูลลูกค้า คลิก “+ เพิ่มแท็ก” แล้วเลือกแท็กที่ต้องการ โดยแท็กใหม่ต้องสร้างก่อนที่เมนู ตั้งค่า แท็บ “แท็ก · Tags” แล้วจึงนำมาติดกับแชทได้

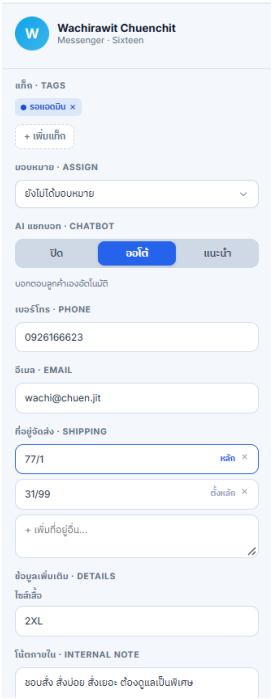
2) ในส่วน “มอบหมาย · Assign” เลือกผู้ดูแลแชท หรือ “ยกเลิกมอบหมาย”

3) ในส่วน “AI แชทบอท · Chatbot” เลือก “ปิด” เพื่อให้เจ้าหน้าที่ตอบเอง “ออโต้” เพื่อให้แชทบอทตอบอัตโนมัติ หรือ “แนะนำ” เพื่อให้แชทบอทร่างคำตอบ

4) บันทึกเบอร์โทร อีเมล ที่อยู่จัดส่ง และโน้ตภายในของลูกค้าได้ในแผงเดียวกัน

5) เมื่อจบการสนทนา คลิกเมนู ••• แล้วเลือก “เก็บถาวร · Archive”

แผงข้อมูลลูกค้าแสดงดังรูปที่ ก.11



รูปที่ ก.11 แผงข้อมูลลูกค้าในหน้ากล่องแชท

ก.5 ทีมงานและสิทธิ์การใช้งาน

ก.5.1 การเชิญสมาชิกเข้าทีม มีขั้นตอนดังนี้

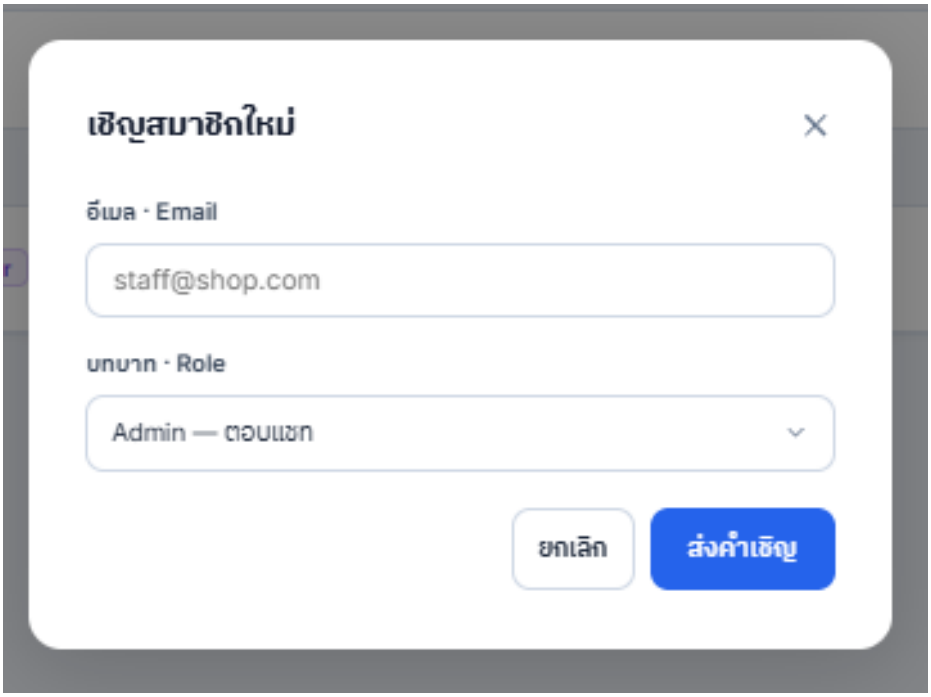
1) ไปที่เมนู ทีมงาน แท็บ “ทีมงาน · People” แล้วคลิก “เชิญผู้ใช้”

2) กรอกอีเมลและเลือกบทบาท แล้วคลิก “ส่งคำเชิญ” ลิงก์คำเชิญมีอายุ 7 วัน

3) ผู้ได้รับเชิญคลิกลิงก์ในอีเมล กรอกชื่อและรหัสผ่าน แล้วคลิก “เข้าร่วมทีม”

4) คำเชิญที่ยังไม่ตอบรับส่งซ้ำหรือยกเลิกได้ที่ “คำเชิญที่รอตอบรับ”

หน้าต่างเชิญสมาชิกแสดงดังรูปที่ ก.12



รูปที่ ก.12 หน้าต่างเชิญสมาชิกใหม่

ก.5.2 การกำหนดบทบาทและสิทธิ์ มีขั้นตอนดังนี้

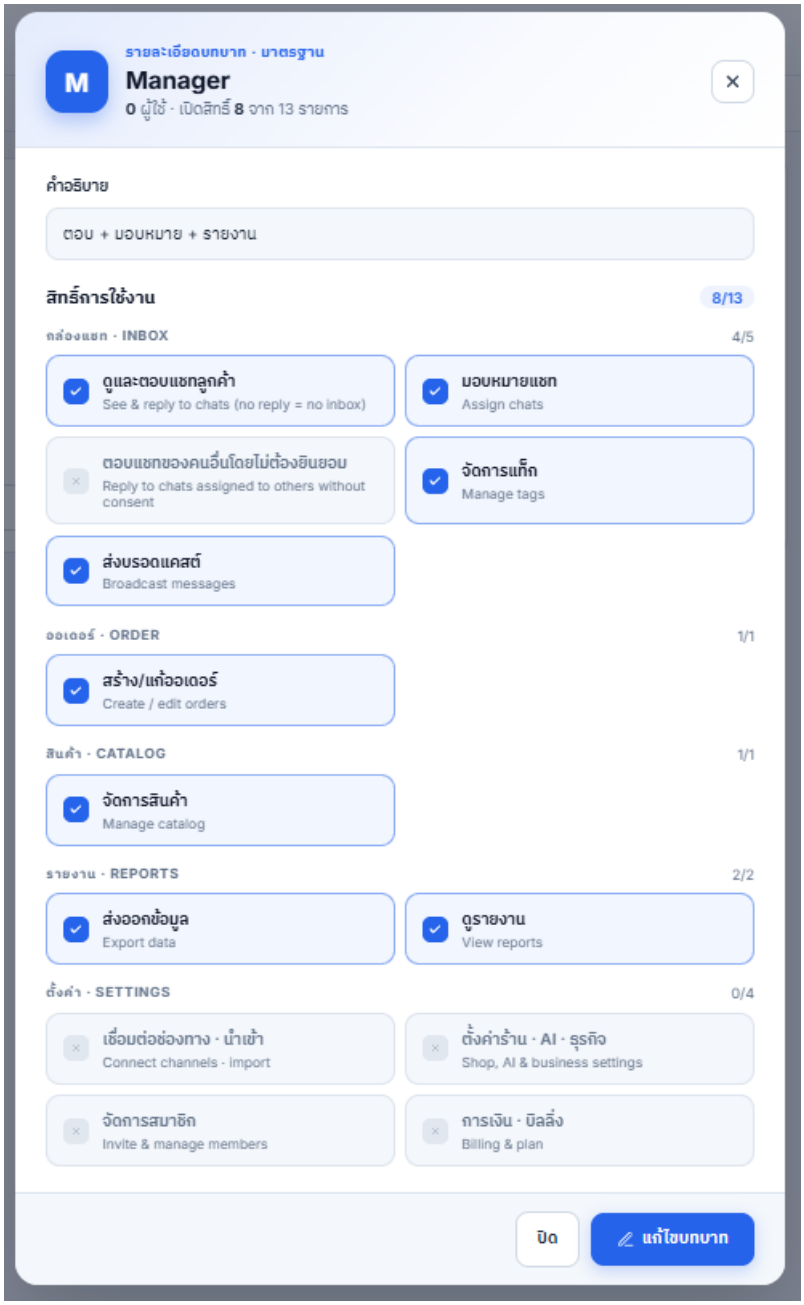
1) ไปที่แท็บ “บทบาท · Roles” แล้วคลิก “แก้ไขบทบาท” เพื่อแก้บทบาทเดิม หรือ “สร้างบทบาทเอง” เพื่อสร้างบทบาทใหม่

2) กรอกชื่อบทบาท สีประจำบทบาท และคำอธิบาย แล้วเลือกสิทธิ์การใช้งาน ซึ่งแบ่งเป็นกลุ่มกล่องแชท ออเดอร์ สินค้า รายงาน และตั้งค่า

3) คลิก “สร้างบทบาท” หรือ “บันทึกการเปลี่ยนแปลง”

4) เปลี่ยนบทบาทของสมาชิกได้จากเมนูในแถวของสมาชิกแต่ละคน

หน้าแก้ไขบทบาทแสดงดังรูปที่ ก.13



รูปที่ ก.13 หน้าแก้ไขบทบาทและสิทธิ์การใช้งาน

ก.6 การตั้งค่าการตอบกลับ

ก.6.1 การตั้งกฎตอบกลับอัตโนมัติตามคีย์เวิร์ด มีขั้นตอนดังนี้

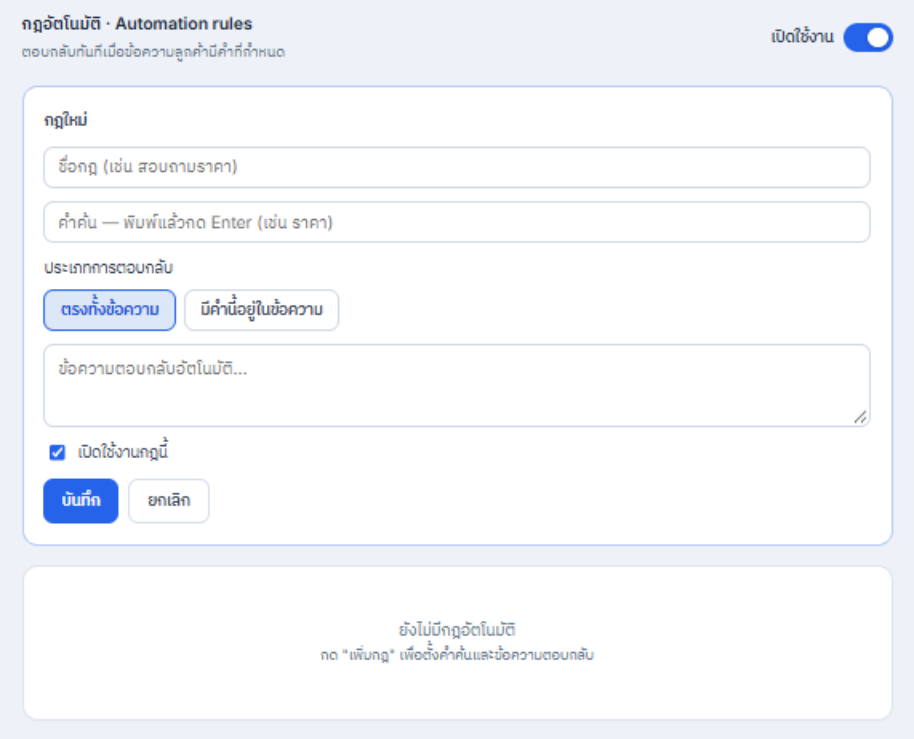
1) ไปที่เมนู ตั้งค่า แท็บ “อัตโนมัติ · Automation” แล้วเปิดสวิตช์

2) คลิก “+ เพิ่มกฎ” กรอกชื่อกฎ แล้วพิมพ์คีย์เวิร์ดและกด Enter ทีละคำ

3) เลือกประเภทการตอบกลับเป็น “ตรงทั้งข้อความ” หรือ “มีคำนี้อยู่ในข้อความ” แล้วพิมพ์ข้อความที่ต้องการให้ระบบตอบกลับลูกค้าเมื่อพบคีย์เวิร์ด

4) เลือก “เปิดใช้งานกฎนี้” แล้วคลิก “บันทึก”

หน้าเพิ่มกฎอัตโนมัติแสดงดังรูปที่ ก.14



รูปที่ ก.14 หน้าเพิ่มกฎตอบกลับอัตโนมัติ

ก.6.2 การตั้งค่าข้อความสำเร็จรูป แท็ก และข้อมูลลูกค้า มีขั้นตอนดังนี้

1) แท็บ “ตอบสำเร็จรูป · Saved Replies” คลิก “Add reply” กรอกชื่อและข้อความ แล้วคลิก “Save” เพื่อใช้ในช่องพิมพ์ของกล่องแชท

2) แท็บ “แท็ก · Tags” กรอกชื่อแท็ก แล้วคลิก “เพิ่มแท็ก”

3) แท็บ “ข้อมูลลูกค้า · Customer fields” กรอกชื่อหัวข้อ เลือกชนิดเป็นพิมพ์ข้อความหรือเลือกจากรายการ แล้วคลิก “เพิ่มหัวข้อ”

ก.7 การตั้งค่าแชทบอท

การตั้งค่าแชทบอททำที่เมนู ตั้งค่า แท็บ “AI Chatbot” ซึ่งมีรายการขั้นตอนเริ่มต้น 3 ขั้น ได้แก่ เปิดโหมดบอท ตั้งบุคลิก และเพิ่มคลังความรู้ โดยแต่ละขั้นมีปุ่ม “ไปตั้งค่า”

ก.7.1 การตั้งค่าด้วยตัวช่วย มีขั้นตอนดังนี้

1) คลิก “ตั้งค่าด้วยตัวช่วย” ในการ์ด “ผู้ช่วยตอบแชท AI”

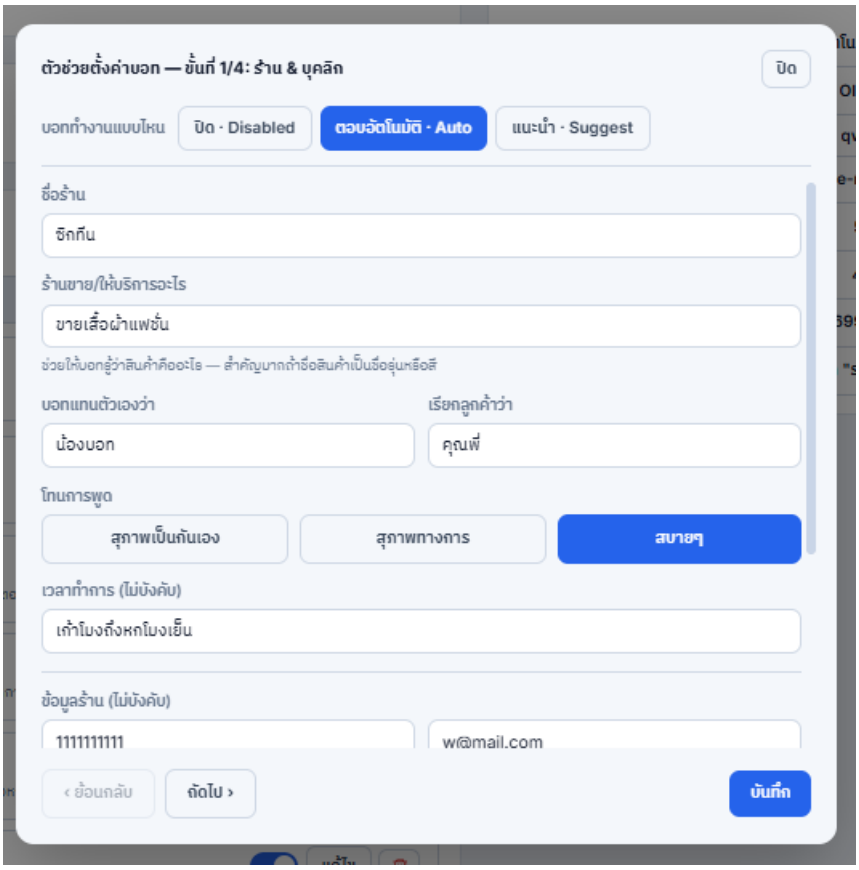
2) ขั้นร้านและบุคลิก กรอกชื่อร้าน สินค้าหรือบริการ คำที่บอทใช้แทนตัวเองและเรียกลูกค้า โทนการพูด เวลาทำการ ข้อมูลติดต่อ และช่องทางชำระเงิน

3) ขั้นนโยบาย กรอกนโยบายการสั่งซื้อ การจัดส่ง และการรับประกัน แล้วคลิก “ดึงจากระบบสินค้า” เพื่อนำรายการสินค้าและราคามาใช้

4) ขั้นกฎของบอท กรอกกฎบรรทัดละ 1 ข้อ และข้อความที่บอทไม่ต้องตอบ

5) ขั้นตรวจสอบ แก้ไข System prompt ได้ และทดลองพิมพ์ข้อความในส่วน “ทดสอบบอท” เพื่อดูคำตอบของบอทก่อนใช้งานจริง แล้วคลิก “บันทึก”

ตัวช่วยตั้งค่าบอทแสดงดังรูปที่ ก.15



รูปที่ ก.15 ตัวช่วยตั้งค่าบอท

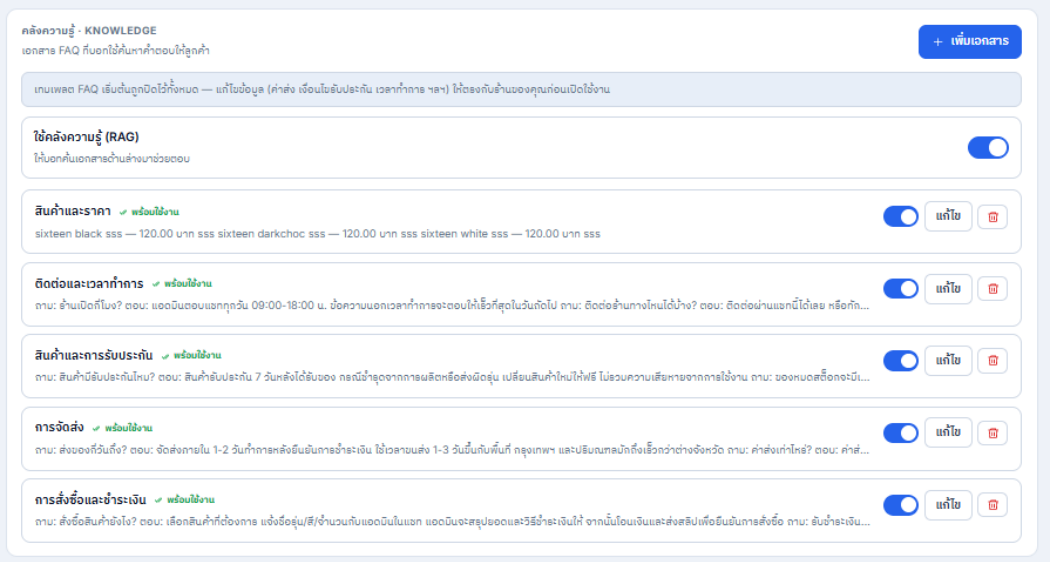
ก.7.2 การเพิ่มคลังความรู้และเลือกโหมดบอท มีขั้นตอนดังนี้

1) ในส่วน “คลังความรู้” เปิด “ใช้คลังความรู้ (RAG)” แล้วคลิก “+ เพิ่มเอกสาร”

2) กรอกหัวข้อและเนื้อหา เลือก “เปิดใช้งาน” แล้วคลิก “บันทึก” เอกสารพร้อมใช้เมื่อสถานะเปลี่ยนจาก “กำลังประมวลผล” เป็น “พร้อมใช้งาน”

3) ในส่วน “ตั้งค่าขั้นสูง · Advanced” เลือกโหมดบอทเป็นปิด ตอบอัตโนมัติหรือแนะนำ กำหนดแท็กที่ใช้เมื่อส่งต่อแชทให้แอดมิน แล้วคลิก “บันทึกขั้นสูง”

คลังความรู้แสดงดังรูปที่ ก.16



รูปที่ ก.16 ส่วนคลังความรู้ของแชทบอท

ก.7.3 การเพิ่มผู้ให้บริการแบบจำลองภาษา แชทบอทใช้ AI ของระบบเป็นค่าเริ่มต้น หากร้านค้าต้องการใช้แบบจำลองอื่น มีขั้นตอนดังนี้

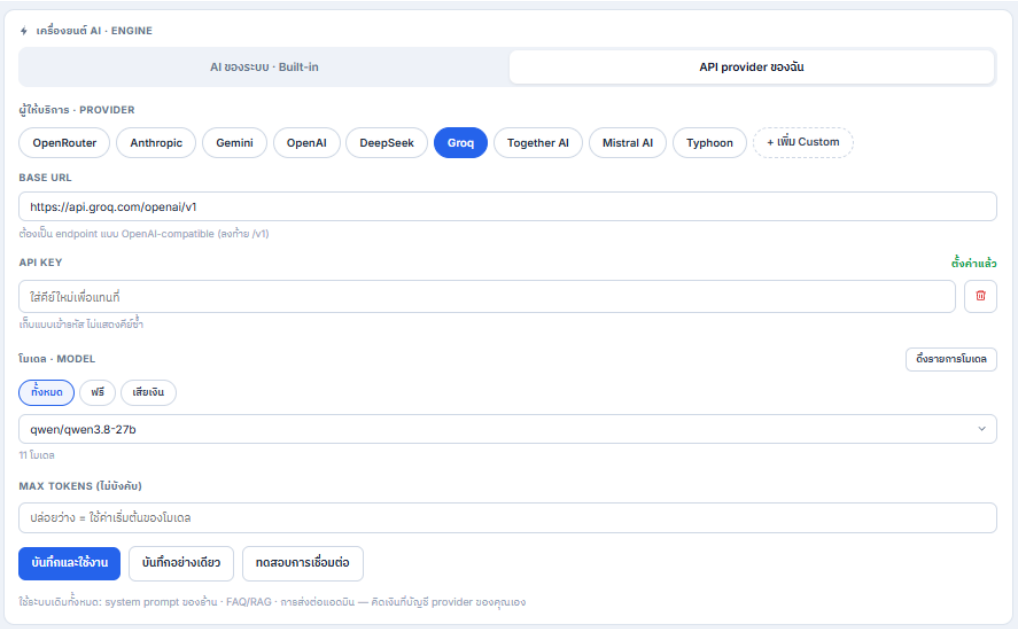
1) ในส่วน “เครื่องยนต์ AI · Engine” เลือก “API provider ของฉัน”

2) เลือกผู้ให้บริการ เช่น OpenRouter, OpenAI, Gemini หรือ Typhoon หรือคลิก “+ เพิ่ม Custom” สำหรับผู้ให้บริการอื่นที่รองรับรูปแบบ OpenAI API

3) กรอก Base URL และ API key แล้วคลิก “ดึงรายการโมเดล” เพื่อเลือกโมเดล

4) คลิก “ทดสอบการเชื่อมต่อ” เมื่อทดสอบผ่านแล้ว คลิก “บันทึกและใช้งาน”

ฟอร์มเพิ่มผู้ให้บริการแสดงดังรูปที่ ก.17



รูปที่ ก.17 ฟอร์มเพิ่มผู้ให้บริการแบบจำลองภาษา

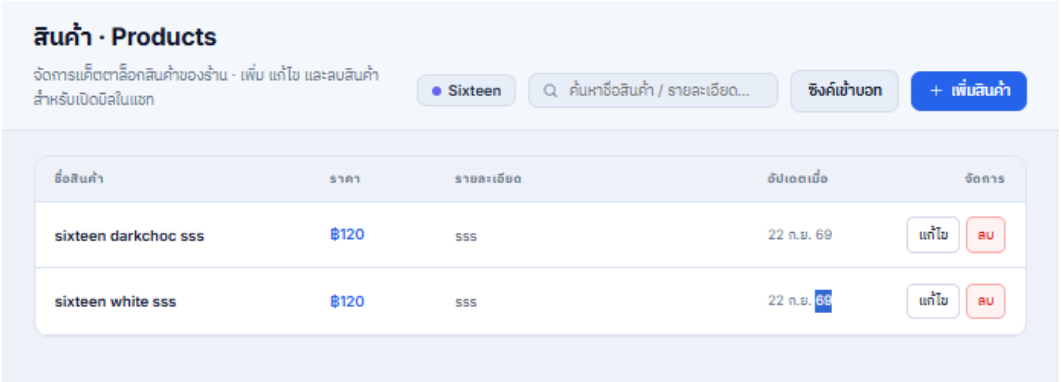
ก.8 สินค้า ลูกค้า และคำสั่งซื้อ

ก.8.1 การจัดการสินค้า มีขั้นตอนดังนี้

1) ไปที่เมนู สินค้า แล้วคลิก “+ เพิ่มสินค้า”

2) กรอกชื่อสินค้า ราคา และรายละเอียดสินค้า แล้วคลิก “เพิ่มสินค้า” โดยแก้ไขหรือลบสินค้าได้จากปุ่มในแถวของสินค้า และค้นหาสินค้าได้จากช่องค้นหาด้านบน

3) คลิก “ซิงค์เข้าบอท” เพื่อให้แชทบอทใช้ข้อมูลสินค้าล่าสุดในการตอบลูกค้า หน้าจัดการสินค้าแสดงดังรูปที่ ก.18



รูปที่ ก.18 หน้าจัดการสินค้า

ก.8.2 การแก้ไขข้อมูลลูกค้า มีขั้นตอนดังนี้

1) ไปที่เมนู ลูกค้า ค้นหาชื่อลูกค้า แล้วคลิกแถวของลูกค้าที่ต้องการ

2) แก้ไขเบอร์โทร ที่อยู่ และโน้ต แล้วคลิก “บันทึก”

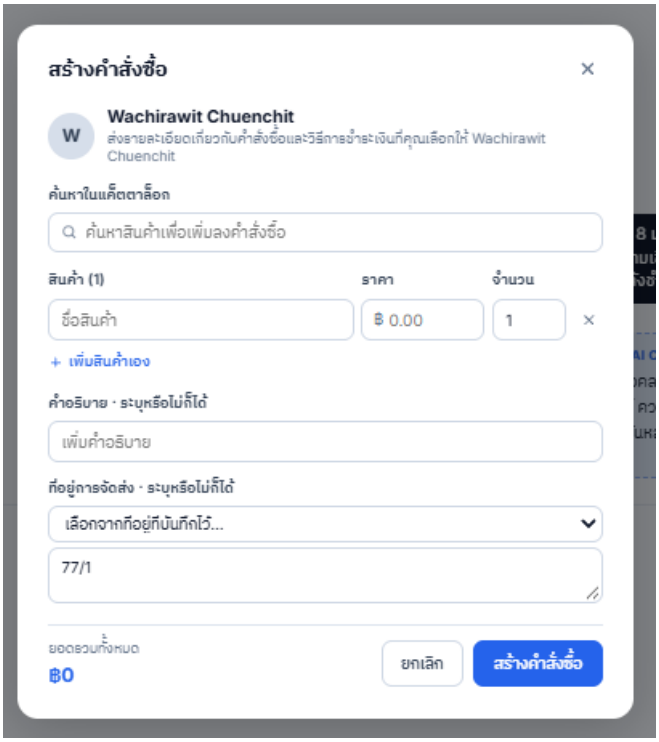
ก.8.3 การเปิดคำสั่งซื้อจากหน้าแชท มีขั้นตอนดังนี้

1) ในแผงข้อมูลลูกค้าของแชท คลิก “+ สร้างออเดอร์” ในส่วน “ออเดอร์ล่าสุด”

2) เลือกสินค้าจากแค็ตตาล็อก หรือ “+ เพิ่มสินค้าเอง” แล้วกำหนดจำนวน

3) เลือกที่อยู่จัดส่ง ตรวจยอดรวม แล้วคลิก “สร้างคำสั่งซื้อ”

หน้าต่างสร้างคำสั่งซื้อแสดงดังรูปที่ ก.19



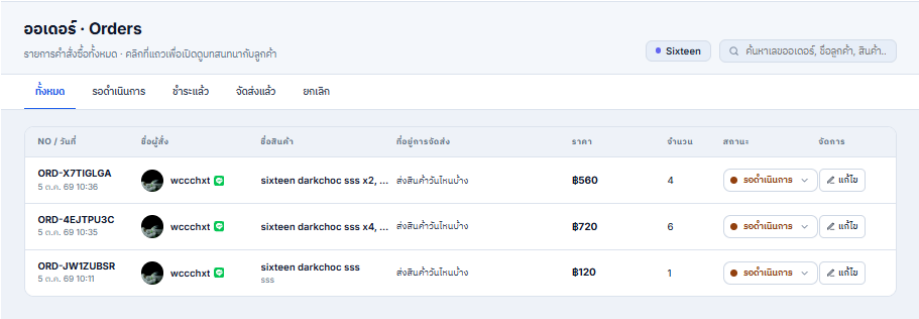
รูปที่ ก.19 หน้าต่างสร้างคำสั่งซื้อจากหน้าแชท

ก.8.4 การติดตามคำสั่งซื้อ มีขั้นตอนดังนี้

1) ไปที่เมนู ออเดอร์ เลือกแท็บสถานะ ได้แก่ ทั้งหมด รอดำเนินการ ชำระแล้ว จัดส่งแล้ว และยกเลิก หรือค้นหาจากเลขออเดอร์ ชื่อลูกค้า หรือชื่อสินค้า

2) เปลี่ยนสถานะใน “แก้ไขคำสั่งซื้อ” แล้วคลิก “บันทึกการแก้ไข”

หน้ารายการคำสั่งซื้อแสดงดังรูปที่ ก.20



รูปที่ ก.20 หน้ารายการคำสั่งซื้อ

ก.9 การส่งบรอดแคสต์

การสร้างบรอดแคสต์เพื่อส่งข้อความถึงลูกค้าหลายคนพร้อมกัน มีขั้นตอนดังนี้

1) ไปที่เมนู บรอดแคสต์ แล้วคลิก “สร้าง Broadcast”

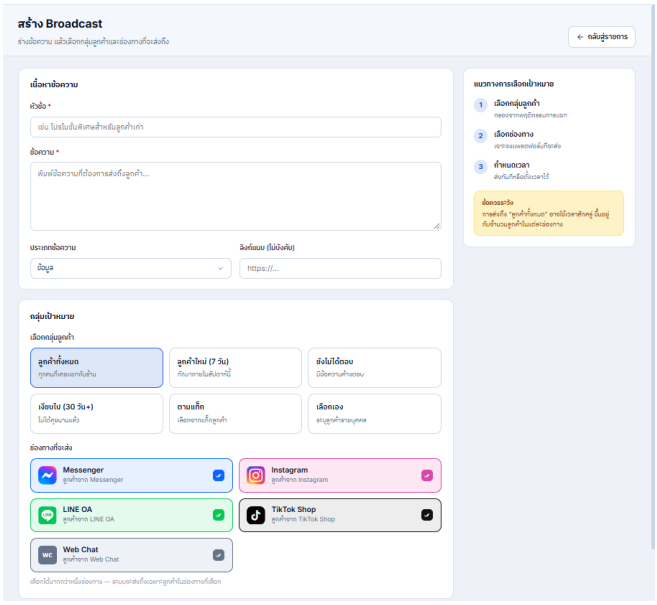
2) กรอกหัวข้อ ข้อความ ประเภทข้อความ และลิงก์แนบซึ่งไม่บังคับ

3) เลือกกลุ่มลูกค้า ได้แก่ ลูกค้าทั้งหมด ลูกค้าใหม่ใน 7 วัน ลูกค้าที่ยังไม่ได้ตอบ ลูกค้าที่เงียบไปเกิน 30 วัน ลูกค้าตามแท็ก หรือเลือกลูกค้าเอง แล้วเลือกช่องทางที่จะส่ง

4) กำหนดวันเวลาส่ง หรือเว้นว่างเพื่อส่งทันที แล้วคลิก “ดูจำนวนผู้รับ”

5) เปิด “พร้อมส่ง” แล้วคลิก “สร้าง Broadcast”

หน้าสร้างบรอดแคสต์แสดงดังรูปที่ ก.21



รูปที่ ก.21 หน้าสร้างบรอดแคสต์

ก.10 การนำเข้าประวัติแชท

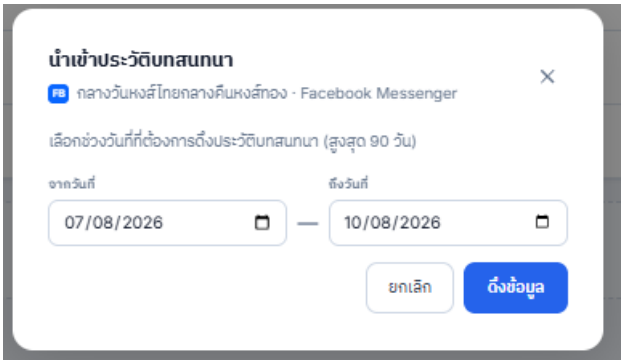
การนำเข้าประวัติแชทรองรับเฉพาะ Facebook และ Instagram มีขั้นตอนดังนี้

1) ไปที่เมนู ตั้งค่า แท็บ “ช่องทาง” แล้วคลิก “นำเข้า” ในแถวของเพจที่ต้องการ

2) เลือกช่วงวันที่ไม่เกิน 90 วัน แล้วคลิก “ดึงข้อมูล”

3) รอจนระบบแจ้งจำนวนบทสนทนาที่นำเข้าสำเร็จ แล้วคลิก “ปิด”

หน้าต่างนำเข้าประวัติแชทแสดงดังรูปที่ ก.22



รูปที่ ก.22 หน้าต่างนำเข้าประวัติบทสนทนา

ก.11 การแจ้งเตือนและรายงานสรุปผล

ก.11.1 การตั้งค่าการแจ้งเตือน มีขั้นตอนดังนี้

1) ไปที่เมนู ตั้งค่า แท็บ “การแจ้งเตือน · Notifications” แล้วเปิดหรือปิดการแจ้งเตือนของระบบปฏิบัติการและเสียงแจ้งเตือน

2) เลือกให้แจ้งเตือนเมื่อมีข้อความใหม่ทุกรายการ หรือเฉพาะแชทที่มอบหมายให้ตนเอง และเลือกช่องทางที่ต้องการรับแจ้งเตือน

ส่วนแชทที่ค้างตอบนานเกินเวลาที่กำหนด ระบบจะแจ้งเตือนเจ้าหน้าที่โดยอัตโนมัติโดยไม่ต้องตั้งค่าเพิ่ม และแสดงจำนวนแชทที่ค้างตอบไว้ในหน้ารายงานสรุปผล

ก.11.2 การดูรายงานสรุปผล มีขั้นตอนดังนี้

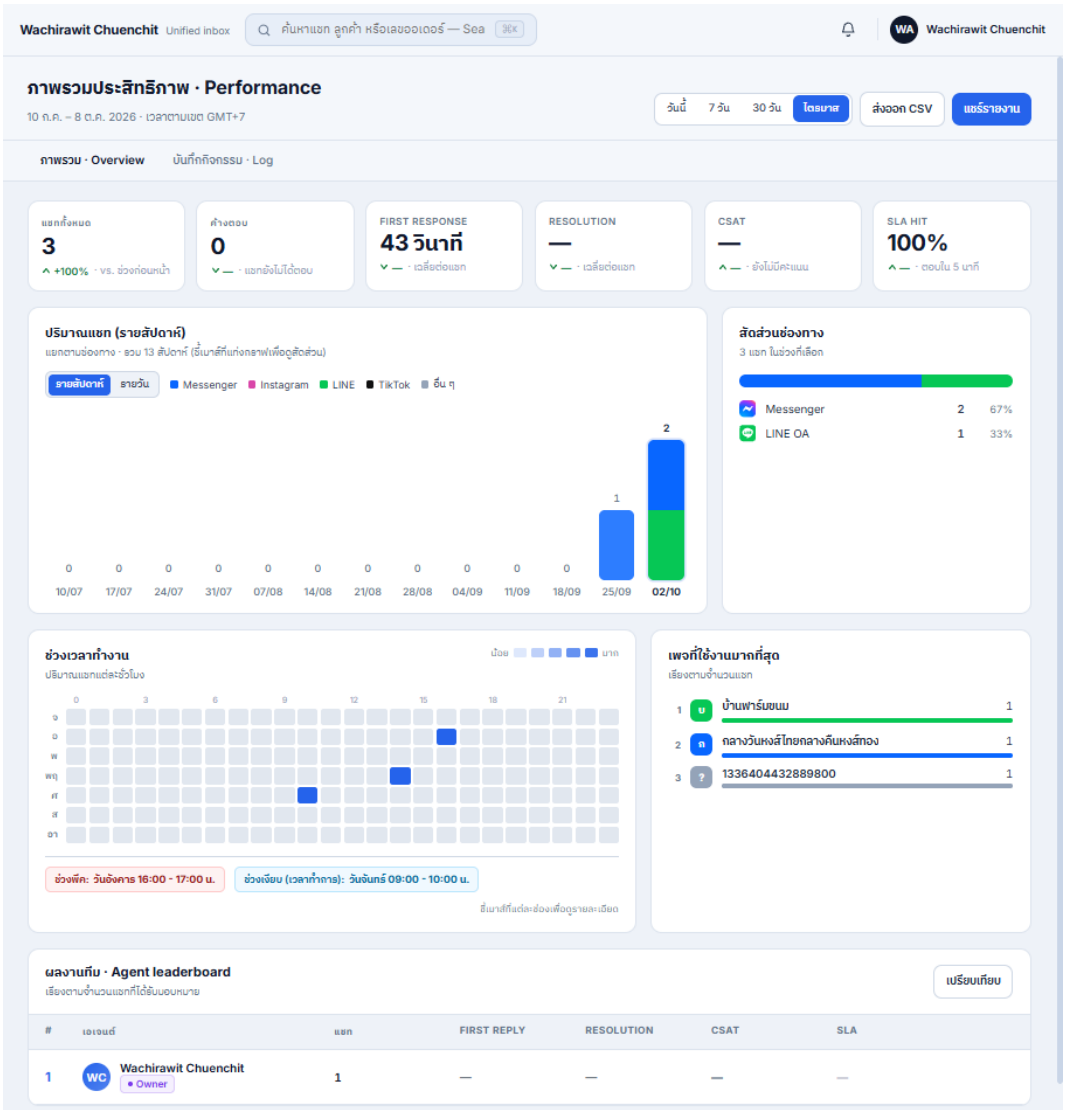
1) ไปที่เมนู ภาพรวม แล้วเลือกช่วงเวลา ได้แก่ วันนี้ 7 วัน 30 วัน หรือไตรมาส

2) ดูตัวชี้วัด ได้แก่ จำนวนแชททั้งหมด แชทที่ค้างตอบ เวลาตอบครั้งแรก เวลาปิดแชท และสัดส่วนแชทที่ตอบภายใน 5 นาที รวมทั้งกราฟปริมาณแชทต่อวัน สัดส่วนช่องทาง ช่วงเวลาที่มีแชทมาก เพจที่ใช้งานมากที่สุด และผลงานของเจ้าหน้าที่แต่ละคน

3) คลิก “ส่งออก CSV” เพื่อดาวน์โหลดข้อมูล หรือ “แชร์รายงาน” เพื่อคัดลอกข้อความสรุปไปส่งต่อ คัดลอกลิงก์หน้ารายงาน หรือพิมพ์รายงานเป็นไฟล์ PDF

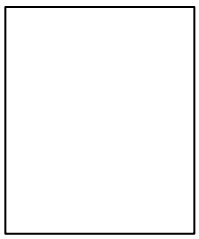
4) เปิดแท็บ “บันทึกกิจกรรม · Log” เพื่อดูประวัติการตอบกลับและการมอบหมายแชทของสมาชิก โดยกรองตามสมาชิกหรือประเภทการกระทำได้

หน้าภาพรวมประสิทธิภาพแสดงดังรูปที่ ก.23



รูปที่ ก.23 หน้าภาพรวมประสิทธิภาพ

ประวัติผู้เขียน



ชื่อ-นามสกุล	นายวชิรวิทย์ ชื่นจิตร
วัน-เดือน-ปีเกิด	21 มีนาคม 2546
ที่อยู่	31/8 อ.เมืองเลย ต.กุดป่อง จ.เลย 42000
อีเมล	wachirawit.ce@rmuti.ac.th
เบอร์โทรศัพท์	0926166623

ประวัติการศึกษา

มัธยมศึกษาตอนต้น	โรงเรียนเลยพิทยาคม ปีการศึกษา 2558
ประกาศนียบัตรวิชาชีพ	วิทยาลัยเทคนิคเลย ปีการศึกษา 2561
ประกาศนียบัตรวิชาชีพชั้นสูง	วิทยาลัยเทคนิคเลย ปีการศึกษา 2564